

TOYOTA - 32Bit FİNANS PORTALI PROJESİ

YAZILIM TASARIM DÖKÜMANI (SDD)



Hazırlayan : Berkan Gülyağcı

E-mail: berkan.gulyagci@ogr.sakarya.edu.tr



İçindekiler

Doküman Kontrol Bilgileri

1. GİRİŞ

- 1.1 Amaç
- 1.2 Kapsam
- 1.3 Hedef Kitle
- 1.4 Tanımlar ve Kısaltmalar
- 1.5 Referanslar
- 1.6 Notasyon ve Tasarım Dili
- 1.7 Döküman Genel Bakışı

2. TASARIM BAĞLAMİ VE BAKIŞ AÇILARI

- 2.1 İlgili Taraflar ve Kaygıları
- 2.2 Mimari Sürücüler ve Kalite Hedefleri
- 2.3 Tasarım Bakış Açıları (Viewpoints)
- 2.4 Teknoloji Yığını Özeti
- 2.5 Mimari Başarı Kriterleri

3. MİMARİ GENEL BAKIŞ

- 3.1 Sistem Bağlam Görünümü (C4 — Level 1)
- 3.2 Konteyner Görünümü (C4 — Level 2)
- 3.3 Bileşen Görünümü (C4 — Level 3)
 - 3.3.1 Backend Bileşenleri
 - 3.3.2 Frontend Bileşenleri
- 3.4 Clean Architecture Katmanları ve Bağımlılık Kuralı
 - 3.4.1 Katmanlar
 - 3.4.2 Bağımlılık Kuralı (Dependency Rule)
- 3.5 Tasarım İlkeleri ve Desenleri (SOLID ve Patterns)
 - 3.5.1 SOLID İlkeleri
 - 3.5.2 Kullanılan Tasarım Desenleri
 - 3.5.3 Tasarım Gerekçesi

4. BACKEND TASARIMI

4.1 Katmanlı Yapı

4.2 Modül Tasarım Şablonu ve Modüller Arası Bağımlılıklar

4.2.1 Modül Tasarım Şablonu

4.2.2 Modüller Arası Bağımlılıklar

4.3 Dış Veri Entegrasyonu ve Sembol Yönlendirme

4.3.1 Provider Adapter Deseni

4.3.2 Varlık Türüne Göre Yönlendirme (Asset-Type Routing)

4.3.3 Sembol Çözümleme (Symbol Resolution)

4.4 Grafik Veri Stratejisi (Chart Strategy)

4.4.1 Varlık Türüne Özel Grafik Servisleri

4.4.2 Kripto Grafiği için Kaynak Zinciri ve Toplama

4.4.3 Tasarım Gerekçesi

4.5 Dayanıklılık Tasarımı (Resilience)

4.5.1 Last Known Good (LKG) — Birincil Dayanıklılık

4.5.2 Resilience4j — Yeniden Deneme ve Devre Kesici

4.5.3 Önbellek Isıtma (Cache Warm-up)

4.6 Hata Yönetimi ve İstisnalar

4.6.1 İstisna → HTTP Durum Eşlemesi

4.6.2 Özel İstisna Sınıfları

4.6.3 Standart Hata Yanıtı ve Kimlik/Yetki Hataları

4.7 Önbellekleme ve Zamanlanmış İşler

4.7.1 Önbellek Tasarımı (Redis)

4.7.2 Zamanlanmış Görevler (Schedulers)

4.7.3 Dağıtık Ortamda Görev Kilitleme (ShedLock)

4.7.4 Zamanlayıcı İş Parçacığı Havuzu (Thread Pool)

4.8 Konfigürasyon ve Sır (Secret) Yönetimi

4.8.1 Profil Tabanlı Yapılandırma

4.8.2 Sır (Secret) Yönetimi

4.8.3 Üretimde Güvenli Başlatma (Fail-Loud)

5. FRONTEND TASARIMI

5.1 Uygulama Mimarisi

5.1.1 Katman Organizasyonu

- 5.1.2 Özellik Alanları (Features)
- 5.1.3 Uygulama Kabuğu ve Sağlayıcı Zinciri
- 5.2 Yönlendirme ve Sayfa Haritası
 - 5.2.1 Erişim Koruma Katmanları
 - 5.2.2 Genel (Public) Rotalar
 - 5.2.3 Korumalı (Kayıtlı Kullanıcı) Rotalar
 - 5.2.4 Yönetici (Admin) Rotalar
 - 5.2.5 Geriye Dönük Uyumluluk
- 5.3 Durum Yönetimi ve Veri Çekme Katmanı
 - 5.3.1 Global Durum (Context)
 - 5.3.2 Veri Çekme Katmanı (API)
 - 5.3.3 Kimlik ve Token Yönetimi (Interceptor)
- 5.4 Grafik Bileşenleri Mimarisi
 - 5.4.1 Grafik Kütüphaneleri ve Kullanım Alanları
 - 5.4.2 Finansal Grafikler (klinecharts)
 - 5.4.3 Grafik Çizim Kalıcılığı (useChartDrawings)
 - 5.4.4 İstatistiksel ve Karşılaştırmalı Grafikler
- 5.5 Tema Sistemi
 - 5.5.1 Tema Uygulama Mekanizması
 - 5.5.2 Tasarım Token'ları (Design Tokens)
 - 5.5.3 Flaş Önleme (FOUC) ve Kalıcılık
- 5.6 Uluslararasılaştırma (i18n) Mimarisi
 - 5.6.1 Çeviri Organizasyonu
 - 5.6.2 Yerel Biçimlendirme (Locale Formatting)
- 5.7 Temsili Ekran Görüntüleri
 - 5.7.1 Kontrol Paneli (Dashboard)
 - 5.7.2 Portföy Görünümü
 - 5.7.3 Varlık Detay ve Grafik
 - 5.7.4 Yapay Zekâ Portföy Analizi
- 5.8 Dışa Aktarma (Export)
- 5.9 Sayfalama (Pagination)
- 6. VERİ TASARIMI

6.1 Mantıksal Veri Modeli (ER)

6.1.1 Varlıklar ve İlişkiler

6.1.2 Gerçek Yabancı Anahtar (FK) İlişkileri

6.2 Veritabanı Şeması

6.2.1 Tasarım Standartları

6.2.2 Örnek Tablo Şeması (`portfolio\_transaction`)

6.2.3 İndeksler

6.3 Veri Bütünlüğü Kuralları

6.3.1 Değer Kısıtları (CHECK)

6.3.2 Benzersizlik Kısıtları (UNIQUE)

6.3.3 İki Katmanlı Doğrulama

6.4 Şema Sürümleme ve Migration (Flyway)

6.4.1 Migration Yaklaşımı

6.4.2 Şema Evrimi

6.5 Önbellek Tasarımı (Redis)

6.5.1 Önbellek Yapılandırması

6.5.2 Anahtar (Key) Desenleri

6.5.3 İki Katmanlı Önbellek (Normal + LKG)

6.5.4 Önbellek Geçersiz Kılma (Invalidation)

7. ARAYÜZ VE API TASARIMI

7.0 API Standartları ve Yanıt Formatı

7.0.1 Genel İlkeler

7.0.2 Standart Yanıt Sarmalayıcısı (ApiResponse)

7.0.3 HTTP Durum Kodları

7.0.4 API Dokümantasyonu (OpenAPI / Swagger UI)

7.1 REST API Sözleşmesi (Endpoint Kataloğu)

7.1.1 Piyasa ve Haber Uç Noktaları (Genel Erişim)

7.1.2 Kullanıcı Verisi Uç Noktaları (Kimlik Gerekli)

7.1.3 Yönetici Uç Noktaları (Admin Gerekli)

7.1.4 Diğer Uç Noktalar

7.2 Kimlik Doğrulama Akışı (OAuth2/OIDC)

7.2.1 Giriş Akışı (Authorization Code + PKCE)

7.2.2 Belirteç Doğrulama (Backend — Yerel JWKS)

7.2.3 Belirteç Yenileme

7.2.4 Keycloak Giriş Ekranı

7.3 Önemli Etkileşim Senaryoları

7.3.1 Cache-First Veri Sunumu

7.3.2 Portföy Değerleme

7.3.3 Alarm → Bildirim → E-posta

7.3.4 Yapay Zekâ Sohbet Asistanı (Tool-Calling)

7.3.5 Senaryo Simülasyonu (What-If)

7.3.6 Kullanıcı Kayıt Akışı

7.3.7 Kullanıcı Banlama (Cascade)

7.3.8 Bülten Dijesti (Newsletter Digest)

7.3.9 ViOP Vade Kapatma

7.4 Dış Entegrasyon Sözleşmeleri ve Degrade Davranışı

7.4.1 Entegrasyon Koruma Katmanı

7.4.2 Degrade (Aşamalı Bozulma) Davranışı

7.4.3 Entegrasyon Loglama ve Gözlemlenebilirlik

7.4.4 Rate Limiting (Hız Sınırlama)

8. GÜVENLİK TASARIMI

8.1 Kimlik ve Yetki Modeli

8.1.1 Kimlik Yönetimi (Keycloak + LDAP)

8.1.2 Yetkilendirme Modeli (RBAC)

8.1.3 Merkezi Yetkilendirme (SecurityConfig)

8.1.4 Kaynak Sahipliği Kontrolü

8.2 Ban Mekanizması Tasarımı

8.2.1 Ban Türleri ve Durum Saklama

8.2.2 Banlama Akışı (Cascade)

8.2.3 Ban Kaldırma (Cascade Geri Alma)

8.3 Güvenlik Önlemleri

8.3.1 Aktarım Güvenliği (HTTPS/TLS)

8.3.2 Köken Politikası (CORS)

8.3.3 Girdi Doğrulama ve Temizleme (Validation & Sanitization)

8.3.4 Hız Sınırlama (Rate Limiting)

8.3.5 Sır (Secret) Yönetimi

8.3.6 Durumsuz Oturum ve CSRF

8.4 Tehdit Modeli (STRIDE) ve ASVS Uyumu

8.4.1 STRIDE Tehdit Analizi

8.4.2 OWASP ASVS Referansı

8.4.3 Bilinçli Kapsam Sınırları

9. DAĞITIM TASARIMI

9.1 Konteynerizasyon (Docker)

9.1.1 Backend İmajı

9.1.2 Frontend İmajı

9.1.3 Diğer İmajlar

9.2 Kubernetes Topolojisi (GKE)

9.2.1 Katmanlı Manifest Organizasyonu

9.2.2 İş Yüğü Türleri (Deployment vs StatefulSet)

9.2.3 Servis Keşfi ve İç İletişim

9.2.4 Backend Yapılandırması (BackendConfig)

9.3 Ağ, Ingress ve TLS

9.3.1 Ingress Yönlendirme (Same-Origin)

9.3.2 TLS Sonlandırma

9.4 Ölçekleme, Yüksek Erişilebilirlik ve Kaynaklar

9.4.1 Yatay Otomatik Ölçekleme (HPA)

9.4.2 Kesinti Bütçesi ve Yedeklilik

9.4.3 Kaynak Talepleri ve Sağlık Kontrolleri

9.4.4 Kalıcı Depolama (PVC)

9.5 Sürekli Entegrasyon ve Dağıtım (CI/CD)

9.5.1 Sürekli Entegrasyon (CI)

9.5.2 Sürekli Dağıtım (CD)

9.5.3 Anahtarsız Kimlik Doğrulama (Workload Identity Federation)

9.5.4 İzlenebilir Dağıtım

10. GÖZLEMLENEBİLİRLİK TASARIMI

10.1 Üç Sütun Mimarisi (Metrik, İz, Log)

10.2 Veri Akışı ve Bileşenler

10.2.1 İz ve Metrik Akışı

10.2.2 Log Akışı (Ayrı Hat)

10.2.3 Sinyal Korelasyonu (Trace ↔ Log)

10.3 İzleme Panoları (Grafana Dashboards)

10.3.1 APM Panosu (Uygulama Performansı)

10.3.2 Dış Veri Kaynakları Panosu (En Kritik)

10.3.3 Loglar Panosu

10.3.4 Bildirimler ve Alarmlar Panosu

10.4 İş ve Entegrasyon Loglaması (AUDIT/INTEGRATION)

10.5 Uçtan Uca İzleme (Distributed Tracing)

10.6 Gerçek İzleme Ekranları

11. ÇAPRAZ KESEN KONULAR (Cross-Cutting Concerns)

11.1 Standart Yanıt Sarmalayıcı (Response Wrapper)

11.2 Merkezi Hata Yönetimi (Global Exception Handling)

11.3 Girdi Doğrulama (Validation)

11.4 İstek Filtreleri (Filter Zinciri)

11.5 Loglama ve İz Kancaları (Logging & Tracing Hooks)

11.6 Görsel Proxy (Image Proxy)

11.7 Performans Çapraz Konuları

12. MİMARİ KARAR KAYITLARI (ADR)

ADR-01: Modüler Monolit Mimari

ADR-02: Clean Architecture ve Port/Adapter

ADR-03: Dayanıklılık için LKG (Stale-if-Error)

ADR-04: Kimlik Yönetiminin Dışsallaştırılması (Keycloak)

ADR-05: Kalıcı Olmayan Piyasa/Haber Verisi (Cache-Only)

ADR-06: Standart Yanıt Sarmalayıcı (ApiResponse)

ADR-07: Şema Sürümleme (Flyway)

ADR-08: Yapay Zekâ Hesaplama Ayrımı (AI Tasarlar, Kod Hesaplar)

ADR-09: Elle Onaylı Dağıtım (Manuel CD Trigger)

ADR-10: Anahtarsız CI/CD Kimlik Doğrulama (WIF)

ADR-11: Çoklu Sağlayıcı Yapay Zekâ Zinciri

13. RİSKLER VE ÖNLEMLER

13.1 Risk Kategorileri ve Önlemler

13.1.1 Entegrasyon Riskleri (Dış Kaynak)

13.1.2 Performans ve Yük Riskleri

13.1.3 Güvenilirlik Riskleri

13.1.4 Güvenlik Riskleri

13.1.5 Operasyonel ve Veri-Kalitesi Riskleri

13.2 Risk Öncelik Matrisi

14. TASARIM DOĞRULAMA (V&V) VE NFR ÖZETİ

14.1 Test Stratejisi

14.2 Kod Kalitesi Araçları

14.3 Sürekli Doğrulama (CI)

14.4 NFR Özet Tablosu (Gereksinim → Tasarım Karşılığı)

14.5 Kalite Ekranları

14.5.1 SonarQube Kalite Kapısı

15. EKLER

Ek A — Gereksinim İzlenebilirlik Matrisi (RTM)

Ek B — Görsel ve Diyagram Dizini

Ek C — Teknik Terimler ve Kısaltmalar

Ek D — İlgili Dökümanlar

Ek E — Kapsam Dışı ve Gelecek Çalışmalar

Proje Adı: Toyota 32Bit Finans Portalı (Portiva)

Şirket: Toyota 32Bit

Döküman Türü: Teknik Analiz Dökümanı – Yazılım Tasarım Tanımı (SDD)

Dayanılan Standartlar: IEEE 1016-2009, ISO/IEC/IEEE 42010:2022, C4 Model, OWASP ASVS

Tarih: 7 Haziran 2026

Hazırlayan: Berkan Gülyağcı

1. GİRİŞ

1.1 Amaç

Bu döküman, Toyota 32Bit Finans Portalı (Portiva) yazılımının teknik tasarımını ve mimari yapısını tanımlamak amacıyla hazırlanmıştır. Analiz Dökümanında (SRS) "ne yapılacağı" iş ve kullanıcı perspektifinden ortaya konmuşken, bu Teknik Analiz Dökümanı (Yazılım Tasarım Tanımı / SDD) bu gereksinimlerin "**nasıl gerçekleştirildiğini**" teknik düzeyde açıklar.

Dökümanın başlıca hedefleri şunlardır:

- Sistemin mimari yapısını (katmanlar, modüller, bileşenler) ve bunlar arasındaki ilişkileri tanımlamak.
- Backend, frontend, veri, arayüz, güvenlik, dağıtım ve gözlemlenebilirlik tasarımını ayrıntılı biçimde belgelemek.
- Mimari kararları ve gerekçelerini (ADR) kayıt altına alarak tasarımın "neden böyle" olduğunu açıklamak.
- Geliştirici, yazılım lideri ve bakım ekibi için sistemin teknik bir referansını ve yol haritasını sağlamak.
- Gereksinim–tasarım–test izlenebilirliğini (RTM) sağlayarak Analiz Dökümanı ile teknik tasarım arasında bağ kurmak.

Döküman; yazılım tasarım tanımları için IEEE 1016-2009, mimari tanımı için ISO/IEC/IEEE 42010:2022, mimari görselleştirme için C4 modeli ve güvenlik tasarımı için OWASP ASVS çerçeveleri temel alınarak yapılandırılmıştır.

1.2 Kapsam

Bu döküman, Toyota 32Bit Finans Portalı (Portiva) sisteminin uçtan uca teknik tasarımını kapsar. Sistem; React tabanlı bir ön yüz (frontend) ile Spring Boot tabanlı bir arka uçtan (backend) oluşan, konteyner tabanlı ve bulut yerel (cloud-native) bir web uygulamasıdır.

Dökümanın kapsadığı teknik alanlar:

- **Mimari tasarım:** Sistemin C4 modeli ile katmanlı (bağlam, konteyner, bileşen) gösterimi ve Clean Architecture yaklaşımı.
- **Backend tasarımı:** İşlevsel alanlara (domain) göre modüler yapı, katmanlı mimari (sunum, uygulama, alan, altyapı), dış veri entegrasyonu, dayanıklılık (LKG) ve zamanlanmış görevler.
- **Frontend tasarımı:** Sayfa/bileşen mimarisi, yönlendirme, durum yönetimi, grafik bileşenleri, tema ve uluslararasılaştırma (i18n).
- **Veri tasarımı:** İlişkisel veritabanı (PostgreSQL) şeması, varlık-ilişki modeli, şema yönetimi ve önbellekleme (Redis) tasarımı.
- **Arayüz/API tasarımı:** İç REST API sözleşmesi, kimlik doğrulama akışı (OAuth2/OIDC) ve önemli etkileşim senaryoları.
- **Güvenlik tasarımı:** Kimlik ve erişim yönetimi (Keycloak/LDAP/JWT), tehdit modeli ve OWASP ASVS uyumu.
- **Dağıtım tasarımı:** Konteynerizasyon (Docker), Kubernetes (GKE) topolojisi ve sürekli entegrasyon/dağıtım (CI/CD) hattı.
- **Gözlemlenebilirlik tasarımı:** Dağıtık izleme, metrik ve log toplama (OpenTelemetry, Prometheus, Tempo, OpenSearch, Grafana).
- **Çapraz kesen konular, mimari kararlar (ADR), riskler, doğrulama ve izlenebilirlik matrisi.**

Sistem ölçek olarak, arka uçta 12 ana işlevsel alan (ve piyasa alanı altında yaklaşık 20 varlık sınıfı modülü) ile yaklaşık 557 Java sınıfı; ön yüzde 9 ana özellik alanı ile yaklaşık 329 bileşen/modülden oluşmaktadır.

Kapsam dışı: Bu döküman, iş ve kullanıcı gereksinimlerinin tanımını içermez (bunlar Analiz Dökümanında ele alınmıştır). Ayrıca üçüncü taraf servislerin (Keycloak, dış veri sağlayıcıları vb.) iç tasarımları kapsam dışıdır; yalnızca bu servislerle entegrasyon tasarımı ele alınır.

1.3 Hedef Kitle

Bu döküman, sistemin teknik tasarımıyla ilgilenen aşağıdaki teknik paydaş grupları için hazırlanmıştır:

- **Yazılım Lideri / Mimar:** Sistemin mimari yapısını, tasarım kararlarını ve bunların gerekçelerini (ADR) değerlendirmek, tasarımın standartlara ve kalite hedeflerine uygunluğunu denetlemek için.
- **Backend Geliştirici:** Katmanlı mimariyi, domain modüllerini, dış veri entegrasyonlarını, dayanıklılık mekanizmalarını ve API tasarımını anlamak ve geliştirmek için.

- **Frontend Geliştirici:** Sayfa/bileşen mimarisini, durum yönetimini, grafik bileşenlerini, tema ve i18n yapısını anlamak ve geliştirmek için.
- **DevOps / Sistem Mühendisi:** Konteynerizasyon, Kubernetes (GKE) topolojisi, CI/CD hattı ve gözlemlenebilirlik altyapısını kurmak ve işletmek için.
- **Test ve Kalite Mühendisi:** Tasarımın test edilebilirliğini değerlendirmek ve doğrulama (V&V) stratejisini tasarımıyla ilişkilendirmek için.
- **Bakım Ekibi:** Mevcut sistemi sürdürmek, sorun gidermek ve yeni özellikler eklemek için tasarım referansı olarak.

Dökümanın okuyucularının; yazılım mimarisi, REST API'ler, ilişkisel veritabanları, konteyner teknolojileri ve temel bulut/dağıtım kavramları konusunda bilgi sahibi olduğu varsayılmaktadır.

1.4 Tanımlar ve Kısaltmalar

Bu bölüm, dökümanda kullanılan teknik terim, teknoloji ve kısaltmaları tanımlar. Finansal terimler için Analiz Dökümanının (SRS) sözlüğüne başvurulabilir.

Mimari ve Tasarım Terimleri

- **SDD:** Yazılım Tasarım Tanımı (Software Design Description) — bu döküman.
- **SRS:** Yazılım Gereksinim Spesifikasyonu (Analiz Dökümanı).
- **Clean Architecture:** Bağımlılıkların dıştan içe (altyapı → uygulama → alan) yönlendiği, çekirdek iş mantığını çerçevelerden bağımsız tutan katmanlı mimari yaklaşımı.
- **Domain (Alan):** Belirli bir işlevsel sorumluluğu kapsayan modül (örn. portföy, alarm, haber).
- **C4 Modeli:** Yazılım mimarisini dört soyutlama seviyesinde (Context, Container, Component, Code) gösteren görselleştirme yaklaşımı.
- **ADR:** Mimari Karar Kaydı (Architecture Decision Record) — bir mimari kararın bağlamı, seçenekleri ve gerekçesini belgeleyen kayıt.
- **RTM:** Gereksinim İzlenebilirlik Matrisi (Requirement Traceability Matrix).
- **DTO:** Veri Aktarım Nesnesi (Data Transfer Object) — katmanlar arası veri taşıyan nesne.
- **Port / Adapter:** Clean Architecture'da uygulama çekirdeğinin dış dünyayla soyutlanmış sözleşmesi (port) ve bu sözleşmenin somut gerçekleştirimi (adapter).

Backend Teknolojileri

- **Spring Boot:** Java tabanlı uygulama çatısı (framework); sistemin arka uç temeli (sürüm 3.2.1, Java 21).
- **Spring Security:** Kimlik doğrulama ve yetkilendirme çatısı (OAuth2 Resource Server).
- **Spring Data JPA / Hibernate:** Nesne-ilişkisel eşleme (ORM) ve veri erişim katmanı.
- **JPA:** Java Persistence API — ilişkisel veri kalıcılığı standardı.
- **Flyway:** Veritabanı şema değişikliklerinin (DDL) sürüm kontrollü ve otomatik biçimde yönetilmesini sağlayan migration aracı.
- **Lombok:** Java'da tekrar eden kalıp kodu (getter/setter/constructor vb.) derleme zamanında üreten kütüphane.

- **springdoc-openapi (Swagger UI):** REST API'lerin OpenAPI standardında otomatik dokümantasyonunu ve etkileşimli arayüzünü sağlayan kütüphane.
- **ShedLock:** Dağıtık ortamda zamanlanmış görevlerin yalnızca tek bir örnekte çalışmasını sağlayan kilitleme kütüphanesi.
- **Resilience4j:** Hata toleransı (yeniden deneme, zaman aşımı vb.) kütüphanesi.
- **Micrometer / Prometheus:** Uygulama metriklerinin toplanması ve sunulması.
- **Jsoup:** HTML ayrıştırma kütüphanesi (web içeriği çekimi için).
- **Jackson:** JSON/XML serileştirme kütüphanesi.

Frontend Teknolojileri

- **React:** Bileşen tabanlı kullanıcı arayüzü kütüphanesi (sürüm 19).
- **Vite:** Modern ön yüz derleme ve geliştirme aracı.
- **React Router:** İstemci tarafı yönlendirme (routing) kütüphanesi.
- **Tailwind CSS:** Yardımcı sınıf tabanlı (utility-first) CSS çatısı.
- **Axios:** HTTP istek istemcisi (REST API çağrıları için).
- **klinecharts / ECharts / Recharts:** Finansal ve istatistiksel grafik kütüphaneleri.
- **react-grid-layout:** Sürükle-bırak ızgara tabanlı kontrol paneli düzeni.

Altyapı ve Operasyon

- **PostgreSQL:** İlişkisel veritabanı yönetim sistemi (sürüm 17).
- **Redis:** Bellek içi önbellek ve dağıtık kilit deposu (sürüm 7).
- **Apache Kafka:** Dağıtık mesaj kuyruğu (log aktarımı için).
- **Keycloak:** Açık kaynak kimlik ve erişim yönetimi sunucusu (sürüm 26).
- **LDAP / ApacheDS:** Dizin tabanlı kullanıcı yönetim protokolü ve sunucusu.
- **Docker:** Konteynerizasyon platformu.
- **Kubernetes / GKE:** Konteyner orkestrasyon platformu / Google Kubernetes Engine.
- **OpenTelemetry (OTel):** Dağıtık izleme, metrik ve log toplama standardı.
- **Tempo:** Dağıtık iz (trace) depolama sistemi.
- **OpenSearch:** Arama ve log analiz motoru.
- **Grafana:** Metrik, iz ve log görselleştirme paneli.
- **CI/CD:** Sürekli Entegrasyon / Sürekli Dağıtım.
- **JWT / OAuth2 / OIDC:** İmzalı erişim belgesi / yetkilendirme protokolü / OpenID Connect kimlik katmanı.
- **LKG (Last Known Good):** Dış kaynak erişilemediğinde son başarılı veriyi sunan dayanıklılık (stale-if-error) yaklaşımı.
- **TTL:** Önbellek kaydı geçerlilik süresi (Time To Live).

1.5 Referanslar

Bu döküman aşağıdaki standart ve kaynaklara dayanılarak hazırlanmıştır:

- **IEEE 1016-2009** – IEEE Standard for Information Technology — Systems Design — Software Design Descriptions (Yazılım tasarım tanımı yapısı için).
- **ISO/IEC/IEEE 42010:2022** – Software, systems and enterprise — Architecture description (Mimari tanımı, bakış açıları ve ilgili taraflar için).
- **C4 Model** – Context, Container, Component, Code; yazılım mimarisi görselleştirme yaklaşımı (Simon Brown).
- **OWASP ASVS** – Application Security Verification Standard (Güvenlik tasarımı ve doğrulama çerçevesi).
- **ISO/IEC/IEEE 29148:2018** ve **ISO/IEC 25010:2011** – Analiz Dökümanının (SRS) dayandığı gereksinim ve kalite standartları (çapraz referans).
- **Toyota 32Bit Finans Portalı (Portiva) Analiz Dökümanı (SRS)** – Bu dökümanda tasarımı yapılan gereksinimlerin tanımlandığı doküman.

1.6 Notasyon ve Tasarım Dili

Dökümanda mimari ve tasarım gösterimi için aşağıdaki notasyonlar kullanılmıştır:

- **C4 Modeli diyagramları:** Sistem bağlamı (Level 1), konteyner (Level 2) ve bileşen (Level 3) seviyelerinde mimari gösterim.
- **UML benzeri diyagramlar:** Sınıf/varlık ilişkileri, sıralı (sequence) etkileşim diyagramları ve durum/akış diyagramları için.
- **Varlık-İlişki (ER) diyagramı:** Veritabanı tablolarının ve ilişkilerinin gösterimi.
- **Dağıtım (deployment) diyagramı:** Konteyner ve Kubernetes topolojisinin fiziksel yerleşimi.

Diyagramlarda; düz çizgiler doğrudan/senkron çağrılar, kesikli çizgiler mantıksal veya asenkron ilişkileri, oklar veri/çağrı yönünü gösterir. Diyagramlar gerektiğinde Türkçe etiketlerle sunulmuştur.

1.7 Döküman Genel Bakışı

Bu döküman aşağıdaki ana bölümlerden oluşur:

- **Bölüm 1 (Giriş):** Amaç, kapsam, hedef kitle, tanımlar, referanslar ve notasyon.
- **Bölüm 2 (Tasarım Bağlamı ve Bakış Açıları):** İlgili taraflar, kaygılar, mimari sürücüler ve teknoloji yığını.
- **Bölüm 3 (Mimari Genel Bakış):** C4 modeli (bağlam, konteyner, bileşen) ve Clean Architecture.
- **Bölüm 4 (Backend Tasarımı):** Katmanlı yapı, domain modülleri, dış veri entegrasyonu, dayanıklılık ve zamanlanmış görevler.

- **Bölüm 5 (Frontend Tasarımı):** Uygulama mimarisi, yönlendirme, durum yönetimi, grafikler, tema ve i18n.
- **Bölüm 6 (Veri Tasarımı):** Veri modeli, veritabanı şeması, şema yönetimi ve önbellekleme.
- **Bölüm 7 (Arayüz/API Tasarımı):** REST API sözleşmesi, kimlik doğrulama akışı ve etkileşim senaryoları.
- **Bölüm 8 (Güvenlik Tasarımı):** Kimlik/yetki modeli, güvenlik önlemleri ve tehdit modeli.
- **Bölüm 9 (Dağıtım Tasarımı):** Konteynerizasyon, Kubernetes topolojisi ve CI/CD.
- **Bölüm 10 (Gözlemlenebilirlik):** Metrik, iz ve log mimarisi.
- **Bölüm 11–15:** Çapraz kesen konular, mimari kararlar (ADR), riskler, doğrulama ve ekler (izlenebilirlik matrisi dahil).

2. TASARIM BAĞLAMı VE BAKIŞ AÇILARI

Bu bölüm, ISO/IEC/IEEE 42010 mimari tanımı standardına uygun olarak; sistemin ilgili taraflarını (stakeholders), bu tarafların kaygılarını (concerns), tasarımı yönlendiren mimari sürücülerini ve dökümanda kullanılan tasarım bakış açılarını tanımlar.

2.1 İlgili Taraflar ve Kaygıları

Bir mimari tanımı, ilgili tarafların kaygılarını karşılamak üzere oluşturulur. Aşağıdaki tablo, sistemin teknik ilgili taraflarını ve her birinin başlıca mimari kaygılarını özetler.

İlgili Taraf	Başlıca Kaygıları
Yazılım Lideri / Mimar	Mimarinin sürdürülebilirliği, modülerliği, tasarım kararlarının tutarlılığı ve standartlara uygunluk
Backend Geliştirici	Katmanlı yapının netliği, dış entegrasyonların yönetilebilirliği, hata yönetimi ve test edilebilirlik
Frontend Geliştirici	Bileşen mimarisi, durum yönetimi, performans ve yeniden kullanılabilirlik
DevOps / Sistem Mühendisi	Dağıtım kolaylığı, ölçeklenebilirlik, gözlemlenebilirlik ve operasyonel dayanıklılık
Test / Kalite Mühendisi	Test edilebilirlik, doğrulanabilirlik ve sürekli entegrasyon uyumu
Güvenlik Sorumlusu	Kimlik/erişim yönetimi, veri güvenliği, sır yönetimi ve tehdit yüzeyi
Son Kullanıcı (dolaylı)	Yanıt hızı, kullanılabilirlik ve hizmet sürekliliği

2.2 Mimari Sürücüler ve Kalite Hedefleri

Sistemin mimari tasarımını yönlendiren başlıca kalite sürücülerini ve bunların tasarıma yansıması aşağıda özetlenmiştir:

- **Çok Sayıda Dış Entegrasyon (Bütünleştirilebilirlik):** Sistem 20'den fazla dış veri kaynağıyla bütünleşir. Bu, her dış kaynak için **port/adapter** soyutlamasını (kod tabanında 47 port arayüzü) zorunlu kılmış; dış servisler uygulama çekirdeğinden yalıtılmıştır.
- **Dış Kaynak Kırılabilirliğine Karşı Dayanıklılık:** Dış kaynakların kesinti veya kota sınırı riskine karşı, **Last Known Good (LKG)** dayanıklılık deseni 18 farklı noktada uygulanır; böylece bir kaynak erişilemediğinde sistem son geçerli veriyi sunarak çalışmaya devam eder.
- **Performans ve Yanıt Hızı:** Sık erişilen ve maliyetli verilerin tekrar tekrar çekilmemesi için **çok katmanlı önbellekleme** (50'den fazla önbellek ad alanı, Redis tabanlı) ve periyodik önbellek ısıtma (cache warm-up) uygulanmıştır.
- **Operasyonel Otomasyon:** Alarm kontrolü, vade kapatma, bülten gönderimi ve önbellek tazeleme gibi periyodik işler **zamanlanmış görevlerle** (22 scheduler) otomatikleştirilmiş; dağıtık ortamda çift çalışmayı önlemek için ShedLock kullanılmıştır.
- **Sürdürülebilirlik ve Modülerlik:** Sistem, işlevsel alanlara (domain) göre ayrılmış ve her domain Clean Architecture katmanlarına (sunum, uygulama, alan, altyapı) bölünmüştür; bu, bir alandaki değişikliğin diğerlerini en az düzeyde etkilemesini sağlar.
- **Güvenlik:** Kimlik yönetimi uzman bir sisteme (Keycloak/LDAP) devredilmiş; tüm korumalı kaynaklar JWT tabanlı yetkilendirmeye korunmuştur.
- **Gözlemlenebilirlik:** Üretim ortamında sorunların hızlı teşhisi için OpenTelemetry tabanlı dağıtık izleme, metrik ve yapılandırılmış log altyapısı tasarlanmıştır.

2.3 Tasarım Bakış Açıları (Viewpoints)

IEEE 1016 ve 42010 yaklaşımına uygun olarak, sistem tasarımı farklı ilgili taraf kaygılarını ele alan çoklu bakış açılarıyla sunulmuştur:

Bakış Açısı	İlgili Bölüm	Ele Aldığı Kaygı
Bağlam Görünümü (Context)	Bölüm 3	Sistemin dış aktör ve servislerle ilişkisi
Mantıksal/Bileşen Görünümü (Logical/Component)	Bölüm 3, 4, 5	Modül yapısı, katmanlar ve sorumluluklar
Bilgi Görünümü (Information)	Bölüm 6	Veri modeli, şema ve önbellek
Arayüz Görünümü (Interface)	Bölüm 7	API sözleşmeleri ve entegrasyon noktaları
Etkileşim Görünümü (Interaction)	Bölüm 7	Sıralı (sequence) çağrı ve veri akışları
Güvenlik Görünümü (Security)	Bölüm 8	Kimlik, yetki ve tehdit modeli
Dağıtım Görünümü (Deployment)	Bölüm 9	Konteyner ve Kubernetes topolojisi
Operasyonel Görünüm (Operational)	Bölüm 10	Gözlemlenebilirlik ve izleme

2.4 Teknoloji Yığını Özeti

Sistemin dayandığı başlıca teknoloji yığını aşağıda özetlenmiştir:

Katman	Teknolojiler
Ön Yüz (Frontend)	React 19, Vite, React Router, Tailwind CSS, Axios; klinecharts, ECharts, Recharts; react-grid-layout; html-to-image, jsPDF, xlsx (grafik/PDF/Excel dışı aktarma); lucide-react (ikonlar)
Arka Uç (Backend)	Java 21, Spring Boot 3.2.1 (Web, Security, Data JPA, Data Redis, Kafka, Mail, OAuth2 Resource Server, Cache, Validation, Actuator); Lombok
API Dokümantasyonu	springdoc-openapi (Swagger UI / OpenAPI)
Dayanıklılık / Görev	Resilience4j, ShedLock (Redis sağlayıcı), Last Known Good (LKG) önbellek
Veri	PostgreSQL 17, Redis 7; Flyway (veritabanı şema migration); Hibernate / JPA (ORM)
Mesajlaşma	Apache Kafka
Kimlik	Keycloak 26, LDAP (ApacheDS), JWT / OAuth2 / OIDC
İçerik İşleme	Jsoup (HTML ayrıştırma), Jackson (JSON/XML serileştirme)
Gözlemlenebilirlik	OpenTelemetry, Micrometer + Prometheus, Tempo, OpenSearch, Grafana; log4j2 (JSON şablonlu log)
Dağıtım	Docker, Kubernetes (GKE), GitHub Actions (CI/CD)
Kod Kalitesi / Test	SonarQube, JUnit Jupiter, Testcontainers, WireMock, Vitest, JaCoCo ,React Testing Library,

2.5 Mimari Başarı Kriterleri

Mimari tasarımın başarılı kabul edilmesi için aşağıdaki ölçülebilir kriterlerin sağlanması hedeflenir. Bu kriterler, Bölüm 2.2'deki mimari sürücülerin somut karşılığıdır.

Kriter	Hedef / Ölçüt	Tasarımdaki Karşılığı
Yanıt Hızı	Normal yük altında okuma uç noktalarında düşük gecikme	Redis önbellek (50'den fazla ad alanı) + cache-first okuma; önbellek ısıtma (warm-up)
Soğuk Yük Yönetimi	İlk erişimde büyük listelerin (tahvil/fon) hızlı sunulması	Periyodik warm-up; önbellek TTL > ısıtma periyodu invaryantı

Kriter	Hedef / Ölçüt	Tasarımdaki Karşılığı
Otomatik Ölçeklenme	Artan yükte yatay ölçekleme	Kubernetes HPA: 2–6 kopya arası, %50 CPU eşiğinde otomatik ölçekleme
Dış Kaynak Dayanıklılığı	Bir dış kaynak kesildiğinde sistemin çalışmaya devam etmesi	Last Known Good (LKG) deseni 18 noktada; degrade davranışı
Operasyonel Otomasyon	Periyodik işlerin elle müdahale olmadan çalışması	22 zamanlanmış görev (alarm kontrolü 60 sn, fon ısıtma ~8 sn, vade/bülten cron'ları); ShedLock ile dağıtık kilit
Test Kapsamı	Kritik işlevlerin otomatik testlerle doğrulanması	228 backend + 100 frontend test dosyası; CI'da `mvn verify` + Vitest
Gözlemlenebilirlik	İsteklerin uçtan uca izlenebilmesi	OpenTelemetry ile dağıtık iz; yapılandırılmış (JSON) log; Grafana panoları
Güvenli Erişim	Korumalı kaynaklara yalnızca yetkili erişim	JWT tabanlı yetkilendirme; tüm kişisel uç noktalar kimlik doğrulamalı
Dağıtım Tekrarlanabilirliği	Aynı yapının farklı ortamlarda kurulabilmesi	Docker imajları; ortam bağımsız yapılandırma; otomatik CI/CD dağıtımı

Bu kriterler, sistemin kalite hedeflerine ne ölçüde ulaştığını değerlendirmek için referans noktalarıdır ve Bölüm 14'teki doğrulama (V&V) faaliyetleriyle ilişkilendirilir.

3. MİMARİ GENEL BAKIŞ

Bu bölüm, sistemin mimari yapısını C4 modeli (Bağlam → Konteyner → Bileşen) kullanarak üç soyutlama seviyesinde sunar ve uygulanan Clean Architecture yaklaşımını açıklar. Mimari stil, **modüler monolit** (modular monolith) olarak özetlenebilir: tek bir dağıtılabilir arka uç uygulaması içinde, işlevsel alanlara göre net biçimde ayrılmış modüller.

3.1 Sistem Bağlam Görünümü (C4 — Level 1)

Bağlam görünümü, sistemi tek bir kutu olarak ele alır ve onu kullanan aktörler ile etkileşimde bulunduğu dış sistemleri gösterir. Bu seviye, "sistem kimlerle ve nelerle konuşuyor?" sorusunu yanıtlar.

Aktörler (kullanıcılar):

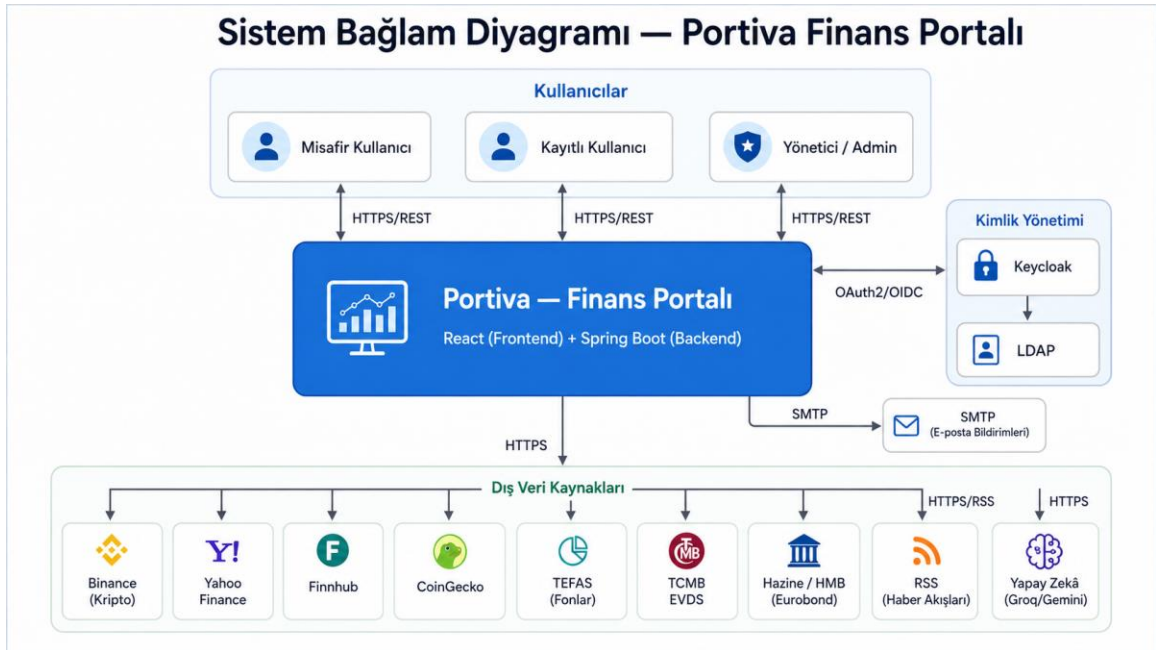
- **Misafir (Anonim) Kullanıcı:** Giriş yapmadan sisteme erişen ziyaretçi. Piyasa verilerini ve haberleri görüntüleyebilir, sohbet asistanını sınırlı biçimde kullanabilir. Bu erişim, herkese açık (`permitAll`) uç noktalarla sağlanır ve kişiye özel veri içermez.

- **Kayıtlı Kullanıcı:** Kimlik doğrulamasından geçmiş kullanıcı (Keycloak'ta `USER` rolü). Misafir yeteneklerine ek olarak portföy oluşturur, izleme listesi ve alarm tanımlar, bülten aboneliği ve destek talebi yönetir.
- **Yönetici (Admin):** Kullanıcı yönetimi, ban işlemleri ve destek talebi yönetimi yapan yetkili kullanıcı (Keycloak'ta `ADMIN` rolü).

Dış sistemler:

- **Keycloak (+ LDAP/ApacheDS):** Kimlik doğrulama, yetkilendirme ve iki adımlı doğrulama (TOTP tabanlı 2FA) sağlayıcısı; arkasında LDAP dizini ile federasyon.
- **Piyasa ve Finans Veri Kaynakları:** TCMB EVDS, TCMB Kurlar, Yahoo Finance, Binance, CoinGecko, TEFAS, Midas, İş Yatırım, Akbank Yatırım, Business Insider, HMB, Borsa İstanbul, FRED, Hesapkurdu, open.er-api, alternative.me, Rasyonet, Halkarz ve diğerleri (toplam 20'den fazla kaynak — *Analiz Dökümanı* 5.4).
- **Haber Kaynakları:** RSS yayınları (AA, NTV, Dünya, Kriptofoni vb.) ve Finnhub.
- **Yapay Zekâ Sağlayıcıları:** Sohbet asistanı yanıtları için çoklu sağlayıcı zinciri (Groq, Gemini).
- **SMTP Sunucusu:** Bildirim, alarm ve bülten e-postalarının gönderimi için.

Sistem, bu aktör ve dış sistemlerle; kullanıcı tarafında HTTPS/REST, kimlik tarafında OAuth2/OIDC, dış veri kaynaklarında HTTPS/REST (JSON, XML, RSS) ve e-posta tarafında SMTP protokolleri üzerinden etkileşir.



Şekil 1: Sistem Bağlam Diyagramı (C4 — Seviye 1)

3.2 Konteyner Görünümü (C4 — Level 2)

Konteyner görünümü, sistemi oluşturan çalıştırılabilir/dağıtılabilir birimleri (konteynerleri) ve aralarındaki iletişimi gösterir. Sistem, mantıksal olarak beş bölgeye (zone) ayrılmıştır.

İstemci Bölgesi (Client Zone)

Konteyner	Teknoloji / Port	Sorumluluk
Web Ön Yüz	React 19 (Nginx ile statik servis), 80/5173	Kullanıcı arayüzü; backend REST API'sini çağırır

Çekirdek Uygulama Bölgesi (Core Zone)

Konteyner	Teknoloji / Port	Sorumluluk
Backend API	Spring Boot 3.2.1 / Java 21, 8080	İş mantığı, dış veri entegrasyonu, önbellek yönetimi, REST API
PostgreSQL	postgres:17, 5432/5433	Kullanıcıya ait kalıcı veriler (portföy, alarm, bildirim vb.)
Redis	redis:7, 6379	Önbellek (50+ ad alanı) ve dağıtık kilit (ShedLock) deposu
Kafka	apache/kafka:4.1.1 (KRaft), 9092	Log akışının asenkron taşınması

Güvenlik Bölgesi (Security Zone)

Konteyner	Teknoloji / Port	Sorumluluk
Keycloak	keycloak:26.0, 8081	Kimlik doğrulama, yetkilendirme, 2FA (OIDC/OAuth2)
ApacheDS (LDAP)	openmicroscopy/apacheds, 10389	Kullanıcı dizini (Keycloak federasyonu)

Gözlemlenebilirlik Bölgesi (Observability Zone)

Konteyner	Teknoloji / Port	Sorumluluk
OTel Collector	otel-collector-contrib:0.96.0, 4317/4318	Metrik ve iz (trace) verilerini toplar, hedeflere iletir
Tempo	grafana/tempo:2.4.1, 3200	Dağıtık iz (trace) depolama
Prometheus	prometheus:v2.51.0, 9090	Metrik toplama ve depolama
OpenSearch	opensearch:2.11.0, 9200	Merkezi log depolama ve analiz

Konteyner	Teknoloji / Port	Sorumluluk
OpenSearch Dashboards	opensearch-dashboards:2.11.0, 5601	Log görselleştirme arayüzü
Log Consumer	(özel servis)	Kafka'dan log tüketip OpenSearch'e yazar
Grafana	grafana:10.4.0, 3000	Metrik/iz/log panoları

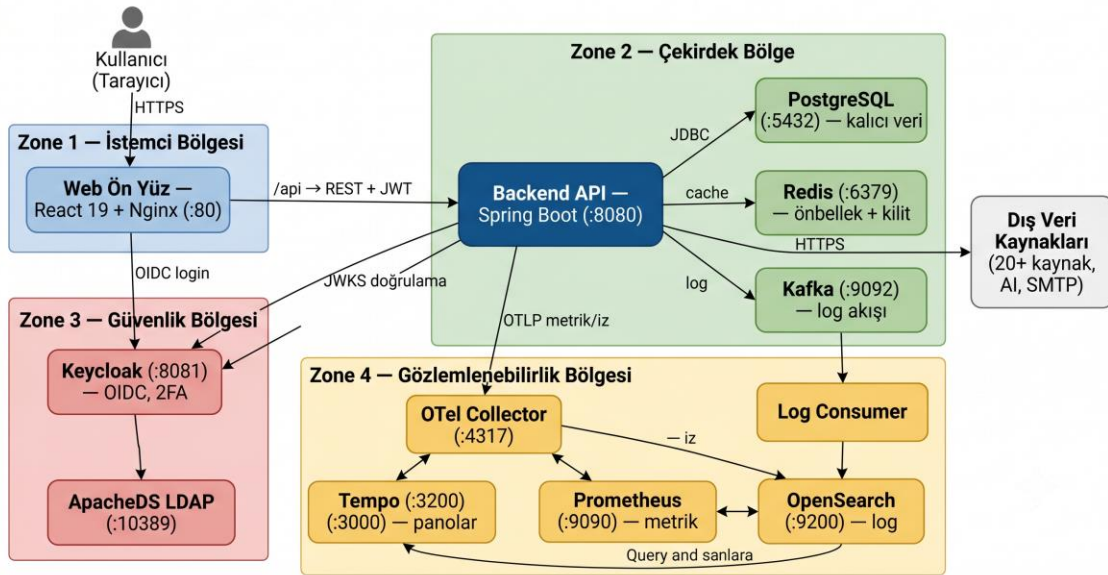
Kod Kalitesi Bölgesi (yalnızca geliştirme)

Konteyner	Teknoloji / Port	Sorumluluk
SonarQube	sonarqube:community, 9000	Statik kod analizi ve kalite kapısı (yalnızca geliştirme ortamı)

Başlıca konteyner iletişimleri:

- Web Ön Yüz → Backend API: HTTPS/REST (JWT Bearer)
- Web Ön Yüz / Backend → Keycloak: OIDC/OAuth2 (token alma, JWKS doğrulama)
- Keycloak → ApacheDS: LDAP federasyonu
- Backend → PostgreSQL: JDBC (kalıcı veri)
- Backend → Redis: önbellek ve dağıtık kilit
- Backend → Kafka → Log Consumer → OpenSearch: log akış hattı
- Backend → OTEL Collector → Tempo / Prometheus: metrik ve iz
- Backend → Dış veri kaynakları: HTTPS/REST (piyasa, haber, AI)
- Grafana → Prometheus / Tempo / OpenSearch: pano veri kaynakları

Şekil 2: Konteyner Diyagramı (C4 — Seviye 2)



Şekil 2: Konteyner Diyagramı (C4 — Seviye 2).

3.3 Bileşen Görünümü (C4 — Level 3)

Bileşen görünümü, iki ana konteynerin (Backend API ve Web Ön Yüz) iç yapısını, yani bunları oluşturan başlıca bileşenleri gösterir.

3.3.1 Backend Bileşenleri

Backend API, işlevsel alanlara (domain) göre **12 ana modülden** oluşur. Her modül, kendi içinde Clean Architecture katmanlarına bölünmüştür.

Modül	Sorumluluk
`market`	Piyasa verileri (hisse, döviz, kripto, fon, tahvil, VIOP, emtia, endeks, ekonomi vb. ~20 alt-alan)
`portfolio`	Portföy ve işlem yönetimi, değerlendirme, kâr/zarar, izleme listesi, what-if analizi
`alarm`	Fiyat/değişim/hacim alarmları ve değerlendirme
`notification`	Uygulama içi bildirimler ve e-posta gönderim durumu
`news`	Çok kaynaklı haber toplama, sınıflandırma ve kişiselleştirme
`assistant`	Yapay zekâ sohbet asistanı (çoklu sağlayıcı + araç çağırma)
`newsletter`	Bülten aboneliği ve periyodik özet gönderimi
`support`	Kullanıcı destek talepleri yönetimi
`preferences`	Kullanıcı arayüz tercihleri (cihazlar arası senkron) ve margin alarm ayarı
`admin`	Kullanıcı yönetimi, ban işlemleri (Keycloak entegrasyonu)
`auth`	Kimlik doğrulama yardımcıları, kayıt ve token akışı
`common`	Çapraz kesen bileşenler (güvenlik, loglama, önbellek, hata yönetimi, yapılandırma)

Her domain modülü, aşağıdaki Clean Architecture katmanlarını içerir:

- **presentation:** REST denetleyicileri (`controller`) ve veri aktarım nesneleri (`dto`).
- **application:** Uygulama servisleri (iş akışları) ve dış dünya soyutlamaları (`port` arayüzleri).
- **domain:** İş varlıkları (entity) ve alan kuralları.
- **infrastructure:** Port'ların somut gerçekleştirmeleri — veritabanı erişimi (`persistence`/`repository`), önbellek (`cache`) ve dış servis istemcileri (`external`).

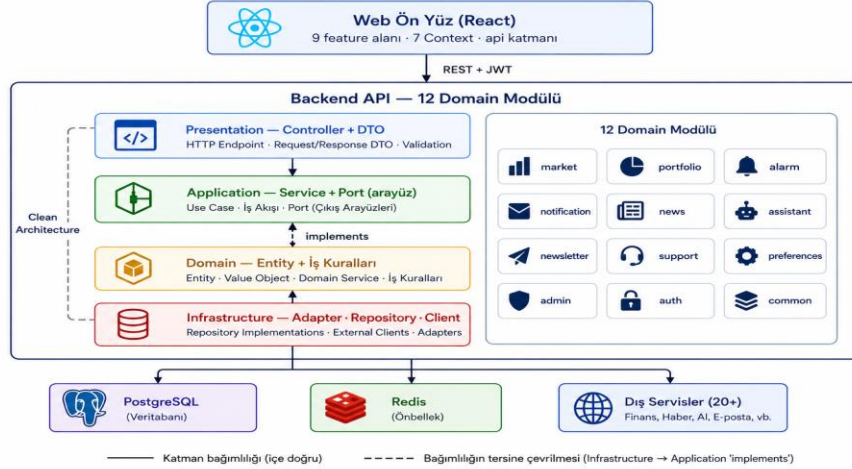
3.3.2 Frontend Bileşenleri

Web Ön Yüz, özellik alanlarına (feature) göre **9 modülden** oluşur: `dashboard`, `market`, `portfolio`, `alarms`, `news`, `notifications`, `profile`, `admin`, `auth`. Bunlara ek olarak çapraz kesen katmanlar bulunur:

Katman	Sorumluluk
`features`	Özellik alanlarına göre sayfalar ve bileşenler (9 alan)
`components`	Paylaşılan/ortak UI bileşenleri (Pagination, WatchlistStar vb.)
`context`	Global durum yönetimi (React Context) — kimlik, tema, dil, tercihler, izleme listesi, bildirim/onay
`api`	Backend REST çağrı katmanı (Axios tabanlı istemci)
`router`	Sayfa yönlendirme ve erişim koruması (`ProtectedRoute`)
`hooks`	Yeniden kullanılabilir özel React kancaları
`i18n`	Çok dilli metin yönetimi (TR/EN)
`lib` / `utils`	Yardımcı fonksiyonlar ve biçimlendiriciler

Global durum, 7 React Context ile yönetilir: `AuthContext`, `ThemeContext`, `LanguageContext`, `PreferencesContext`, `WatchlistContext`, `ToastContext`, `ConfirmContext`.

Şekil 3: Bileşen Diyagramı (C4 — Seviye 3)



Şekil 3: Bileşen Diyagramı (C4 — Seviye 3)

3.4 Clean Architecture Katmanları ve Bağımlılık Kuralı

Backend, her domain modülü içinde **Clean Architecture** ilkelerini uygular. Temel amaç; iş mantığını (domain ve application) çerçeve, veritabanı ve dış servis ayrıntılarından (infrastructure) yalıtaktır.

3.4.1 Katmanlar

Katman	İçerik	Bağımlılık Yönü
Domain	İş varlıkları (entity), değer nesneleri, alan kuralları	Hiçbir dış katmana bağımlı değildir (en içte)
Application	Uygulama servisleri (iş akışları), `port` arayüzleri	Yalnızca domain'e bağımlıdır
Infrastructure	Veritabanı erişimi, önbellek, dış servis istemcileri (port gerçekleştirimleri)	Application ve domain'e bağımlıdır
Presentation	REST denetleyicileri, DTO'lar	Application'a bağımlıdır

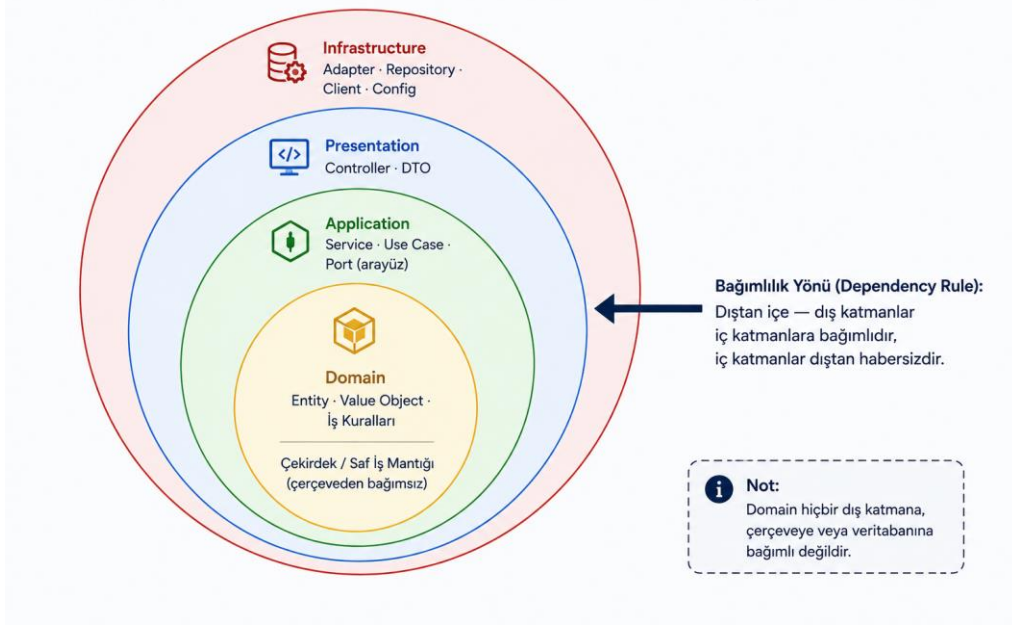
3.4.2 Bağımlılık Kuralı (Dependency Rule)

Clean Architecture'in temel kuralı, **bağımlılıkların yalnızca dışarıdan içeriye doğru** akmasıdır. Bu kural, kod tabanında doğrulanmıştır:

- **Bağımlılığın Tersine Çevrilmesi (Dependency Inversion):** Application katmanı, ihtiyaç duyduğu dış yetenekleri **`port` arayüzleri** olarak tanımlar (örn. `AssistantChatPort`, `AlarmMarketDataPort`, `KeycloakUserAdminPort`). Bu arayüzlerin somut gerçekleştirimleri (adapter) **infrastructure** katmanında yer alır (örn. `CompositeAssistantChatPort`, `AlarmMarketDataAdapter`, `KeycloakUserAdminAdapter`). Böylece iş mantığı, dış servisin "nasıl" çalıştığını bilmez; yalnızca "ne yaptığını" tanımlayan sözleşmeye bağlıdır.
- **Saf Domain:** Domain katmanı çerçeveden tamamen bağımsızdır; bu katmandaki sınıfların hiçbirisi Spring çatısına bağımlılık içermez. Bu, iş kurallarının test edilebilirliğini ve uzun ömürlülüğünü artırır.
- **Kararlılık Yönü:** En sık değişen katman (infrastructure — dış servisler, kütüphaneler) en dışta; en kararlı katman (domain — iş kuralları) en içte konumlanır. Dış katmanlardaki değişiklikler çekirdeği etkilemez.

Bu yapı; bir dış servisin (örn. bir piyasa veri sağlayıcısı) değiştirilmesinin yalnızca ilgili adapter'ı etkilemesini, iş mantığının değişmeden kalmasını sağlar.

Şekil 4: Clean Architecture Katmanları ve Bağımlılık Kuralı



Şekil 4:

Clean Architecture Katmanları ve Bağımlılık Kuralı.

3.5 Tasarım İlkeleri ve Desenleri (SOLID ve Patterns)

Sistemin nesne yönelimli tasarımı, Clean Architecture'ın yanı sıra SOLID ilkeleri ve yerleşik tasarım desenleri (design patterns) üzerine kuruludur. Bu, sürdürülebilir ve genişletilebilir bir kod tabanı sağlar.

3.5.1 SOLID İlkeleri

İlke	Sistemdeki Uygulama
S — Tek Sorumluluk (SRP)	Her servis tek bir işlevsel sorumluluğa odaklanır (kod tabanında ~69 odaklı servis); denetleyici/servis/repository ayrımı bunu pekiştirir
O — Açık/Kapalı (OCP)	Yeni bir dış kaynak veya grafik türü eklemek, mevcut kodu değiştirmeden yeni bir adapter/strateji eklemekle yapılır (port/adapter sayesinde)
L — Liskov Yerine Geçme (LSP)	Bir port arayüzünün farklı gerçekleştirmeleri (örn. farklı sağlayıcı adapter'ları) birbirinin yerine sorunsuz kullanılabilir
I — Arayüz Ayrımı (ISP)	Geniş, çok amaçlı arayüzler yerine küçük ve amaca özel arayüzler tercih edilir (~61 arayüz; her port dar bir yetenek tanımlar)

İlke	Sistemdeki Uygulama
D — Bağımlılığın Tersine Çevrilmesi (DIP)	Üst katmanlar somut sınıflara değil, soyut port arayüzlerine bağımlıdır (47 port arayüzü; gerçekleştirmeler infrastructure'da)

3.5.2 Kullanılan Tasarım Desenleri

Desen	Uygulama Yeri
Port/Adapter (Hexagonal)	Tüm dış entegrasyonlar — uygulama çekirdeğinin dış dünyayla soyutlanması (33 adapter)
Strategy (Strateji)	Varlık türüne özel grafik servisleri ve fiyat sorgu yönlendirmesi (4.3, 4.4)
Chain of Responsibility (Sorumluluk Zinciri)	Dış kaynak fallback zinciri ve yapay zekâ sağlayıcı zinciri (4.4.2, ADR-11)
Composite (Bileşik)	Birden çok sağlayıcıyı/kaynağı tek arayüz altında birleştirme (`CompositeAssistantChatPort`, `CryptoChartAggregator`)
Builder (İnşacı)	Karmaşık nesnelerin adım adım kurulması (`PortfolioHoldingsBuilder`)
Repository	Veri erişiminin soyutlanması (JPA repository'leri)
Filter Chain (Filtre Zinciri)	Çapraz kesen istek işleme (loglama, ban kontrolü — Bölüm 11.4)

3.5.3 Tasarım Gerekçesi

Bu ilke ve desenler tesadüfi değil, sistemin temel zorluğuna (çok sayıda değişken dış kaynak) yanıt olarak seçilmiştir: yeni bir veri kaynağı, grafik türü veya yapay zekâ sağlayıcısı eklemek, mevcut iş mantığını değiştirmeden yalnızca yeni bir küçük bileşen (adapter/strateji) eklemeyi gerektirir. Bu, sistemin **değişime kapalı ama genişlemeye açık** (OCP) kalmasını sağlar.

4. BACKEND TASARIMI

Bu bölüm, arka uç (backend) uygulamasının iç tasarımını; katmanlı yapısını, modül organizasyonunu, dış veri entegrasyonunu, dayanıklılık mekanizmalarını, önbellekleme ve zamanlanmış görev tasarımını ayrıntılı biçimde açıklar.

4.1 Katmanlı Yapı

Backend, Bölüm 3.4'te tanımlanan Clean Architecture katmanlarını somut bileşenlerle gerçekleştirir. Bir HTTP isteği, katmanlar arasında aşağıdaki sırayla ilerler:

İstek Akışı:

...

HTTP İsteği

↓

[Presentation] Controller (REST uç noktası) → DTO doğrulama

↓

[Application] Service (iş akışı) → Port arayüzü (dış yetenek sözleşmesi)

↓

[Domain] Entity / iş kuralları

↓

[Infrastructure] Adapter / Repository / Client (port gerçekleştirimi)

↓

PostgreSQL · Redis · Dış Servis

...

Katman Bileşenleri (kod tabanındaki gerçek sayılar):

Katman	Bileşen Türü	Sayı	Sorumluluk
Presentation	Controller	36	REST uç noktalarını sunar, istek/yanıt eşlemesi yapar
Application	Service	69	İş akışlarını yürütür, port'lar üzerinden dış yetenekleri çağırır
Infrastructure	Adapter	33	Port arayüzlerinin somut gerçekleştirmeleri (dış servis/Keycloak vb.)
Infrastructure	Repository	10	JPA tabanlı veritabanı erişimi

Örnek Akış (Portföy): `PortfolioController`, gelen isteği alır ve `PortfolioService` (iş akışı), `PortfolioAnalysisService` (analiz) gibi application servislerine yönlendirir. Bu servisler, güncel

fiyat gibi dış verilere `PortfolioHistoricalPricePort` gibi **port arayüzleri** üzerinden erişir; bu port'ların gerçekleştirimi infrastructure katmanındaki adapter'larda yer alır. Kalıcı veriye erişim ise `PortfolioRepository` (JPA) üzerinden yapılır.

Bu katmanlı ayırım; her bileşenin tek bir sorumluluğa sahip olmasını (Separation of Concerns), bağımlılıkların yönetilebilir kalmasını ve her katmanın bağımsız test edilebilmesini sağlar.

4.2 Modül Tasarım Şablonu ve Modüller Arası Bağımlılıklar

4.2.1 Modül Tasarım Şablonu

Sistemdeki her domain modülü, tutarlılık ve sürdürülebilirlik için aynı standart şablonla tasarlanmıştır. Bu, dökümanda her modülün ayrı ayrı tekrarlanmasını önler ve yeni modül eklemeyi kolaylaştırır. Her modül şu öğelerden oluşur:

Öge	Açıklama
Amaç ve Kapsam	Modülün ne yaptığı / ne yapmadığı
Sunum (Controller)	Dışarıya açtığı REST uç noktaları ve DTO'lar
Uygulama Servisleri	İş akışlarını yürüten servis(ler)
Port Arayüzleri	Modülün ihtiyaç duyduğu dış yeteneklerin sözleşmeleri
Domain Varlıkları	(Kalıcı veri varsa) iş varlıkları
Altyapı Gerçekleştirmeleri	Repository, cache ve dış servis adapter'ları
Önbellek ve Dayanıklılık	Cache ad alanı, TTL ve LKG yaklaşımı (varsa)

4.2.2 Modüller Arası Bağımlılıklar

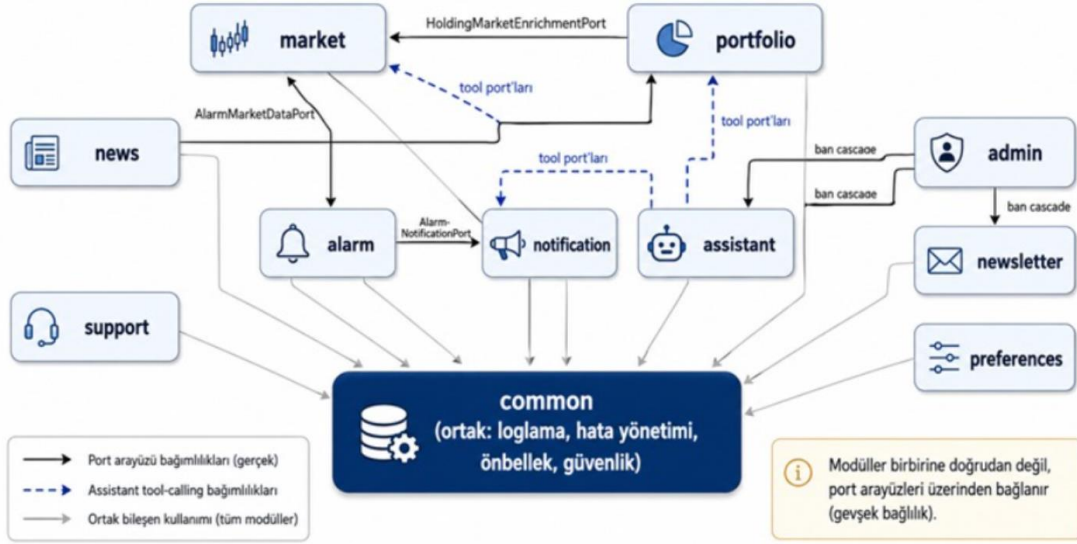
Modüler monolit yaklaşımının temel ilkesi, modüllerin birbirine **doğrudan değil**, **`port` arayüzleri üzerinden** bağlanmasıdır. Bir modül, başka bir modülün iç yapısını bilmez; yalnızca ihtiyaç duyduğu yeteneği tanımlayan bir port arayüzü tanımlar ve hedef modül bunu gerçekleştirir. Bu, gevşek bağlilik (loose coupling) sağlar.

Başlıca modüller arası bağımlılıklar:

Kaynak Modül	Hedef Yetenek	Port Arayüzü	Amaç
`alarm`	`market` (fiyat)	`AlarmMarketDataPort`	Alarm koşulunu güncel fiyatla değerlendirmek
`alarm`	`notification`	`AlarmNotificationPort`	Tetiklenen alarmda bildirim üretmek
`portfolio`	`market` (fiyat)	`HoldingMarketEnrichmentPort`, `PortfolioHistoricalPricePort`, `WatchlistMarketEnrichmentPort`	Portföy/izleme değerlemesi ve geçmiş fiyat

Kaynak Modül	Hedef Yetenek	Port Arayüzü	Amaç
`news`	`portfolio` (varlıklar)	`UserHoldingsPort`	Kullanıcının varlıklarına göre kişiselleştirilmiş haber
`assistant`	birden çok modül	araç (tool) port'ları	Sohbet asistanının fiyat/portföy/alarm araçlarını çağırması
Tüm modüller	`common`	(ortak bileşenler)	Loglama, hata yönetimi, önbellek, güvenlik

Bu yapı sayesinde; örneğin alarm modülü, market modülünün fiyatı hangi dış kaynaktan ve nasıl aldığını bilmeden yalnızca `AlarmMarketDataPort` sözleşmesine güvenir. Hedef modüldeki değişiklikler, port sözleşmesi korunduğu sürece kaynak modülü etkilemez.



4.3 Dış Veri Entegrasyonu ve Sembol Yönlendirme

Sistemin en karmaşık tasarım alanlarından biri, 20'den fazla heterojen dış veri kaynağının tek tip bir iç modele entegrasyonudur. Bu, üç katmanlı bir yaklaşımla çözülür: **provider adapter deseni**, **varlık türüne göre yönlendirme** ve **sembol çözümleme**.

4.3.1 Provider Adapter Deseni

Her dış kaynak, kendine özgü bir HTTP istemcisi (client) ve bunu bir port arayüzüne bağlayan bir adapter ile temsil edilir. Kod tabanında **29 dış servis istemcisi** bulunur. Örnekler:

İstemci (Client)	Kaynak	Veri
`BinanceKlineClient`	Binance	Kripto mum (kline) verisi
`CoinGeckoClient`	CoinGecko	Kripto fiyat/piyasa
`YahooChartClient`	Yahoo Finance	Hisse/endeks/grafik
`MidasStockClient`	Midas	BIST hisse listesi/detay
`EvdsBondClient` / `EvdsEconomyClient`	TCMB EVDS	Tahvil ve makro veri
`RasyonetFundClient`	Rasyonet	Yatırım fonu
`IsYatirimViopChartClient` / `AkbankViopClient`	İş Yatırım / Akbank	ViOP grafik/sözleşme
`BusinessInsiderBondClient`	Business Insider	Eurobond
`FredEconomyClient`	FRED	ABD makro
`FearGreedClient`	alternative.me	Kripto Korku & Hırs endeksi
`GroqChatClient` / `GeminiChatClient`	Groq / Gemini	Yapay zekâ sohbet
`HesapkurduBankCurrencyClient` / `OpenFxClient`	Hesapkurdu / open.er-api	Döviz kuru

Her istemci; standart bir HTTP istemcisi (Spring `RestClient`), yapılandırılabilir taban adres (env üzerinden), entegrasyon loglama ve hata yönetimi içerir. Bu sayede yeni bir kaynak eklemek, yalnızca yeni bir istemci/adapter yazmayı gerektirir; iş mantığı değişmez.

4.3.2 Varlık Türüne Göre Yönlendirme (Asset-Type Routing)

Sistem **8 varlık türünü** destekler: `STOCK`, `FUND`, `FX`, `FUTURE`, `CRYPTO`, `GOLD`, `COMMODITY`, `BOND`. Bir fiyat sorgusu geldiğinde, varlık türüne göre doğru sağlayıcı zincirine yönlendirilir. Bu yönlendirme, `AssetPriceQueryService` içinde merkezi bir `switch` yapısıyla gerçekleştirilir:

...

assetType'a göre:

STOCK → hisse fiyat servisi (Yahoo/Midas)

CRYPTO → kripto fiyat servisi (Binance/CoinGecko/Yahoo)

FX → döviz kuru servisi (TCMB/Hesapkurdu/open.er-api)

FUND → fon fiyat servisi (TEFAS/Rasyonet)

FUTURE → ViOP fiyat servisi (İş Yatırım/Akbank)

...

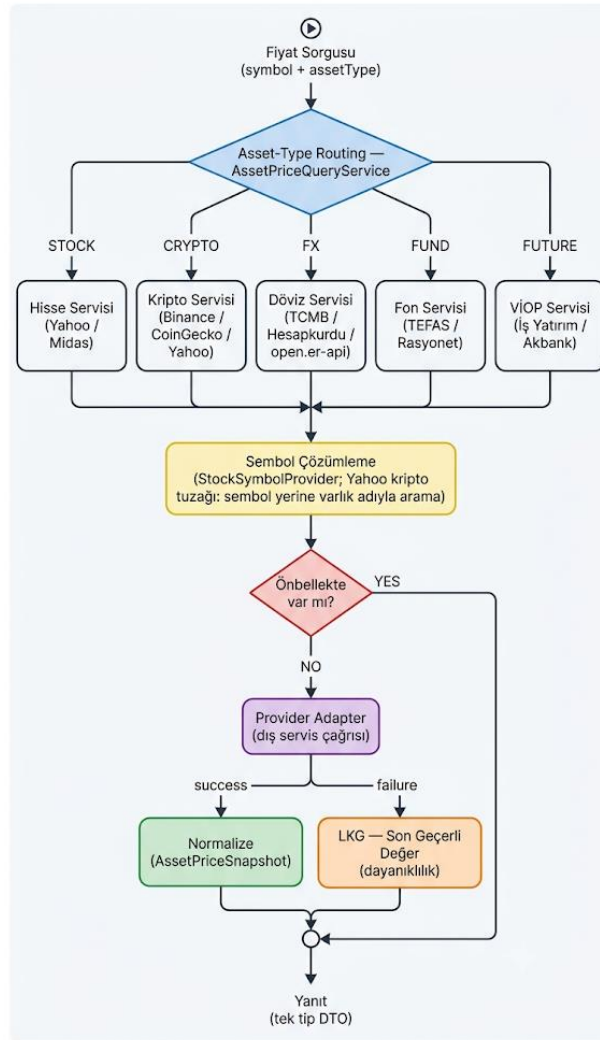
Bu desen, her varlık türünün kendine özgü kaynak ve dönüşüm mantığına sahip olmasına izin verirken, çağırın tarafa tek tip bir arayüz (`AssetPriceSnapshot`) sunar.

4.3.3 Sembol Çözümleme (Symbol Resolution)

Dış kaynaklar aynı varlığı farklı sembollerle adlandırabilir; bu, özellikle Yahoo Finance ve kripto kaynaklarında **sembol çakışması** riskine yol açar. Sistem, bu durumu çözmek için sembol sağlayıcıları (`StockSymbolProvider`, `FuturesSymbolProvider`, `CommoditySymbol`) ve varlık adına göre arama yapan çözümleyiciler kullanır.

Tasarım kararı (kripto sembol tuzağı): Yahoo Finance'te bir kripto sembolü doğrudan `{TICKER}-USD` biçiminde sorgulandığında, ticker çakışması nedeniyle yanlış bir token dönebilir. Bu nedenle çözümleyici, ticker yerine **varlığın adıyla** arama yaparak doğru sembolü belirler. Bu tür kaynağa özgü tuzaklar, ilgili adapter içinde izole edilmiştir ve iş mantığına sızmaz.

Şekil 6: Dış Veri Entegrasyonu ve Sembol Yönlendirme Akışı



Şekil 6 — Dış Veri Entegrasyonu ve Sembol Yönlendirme Akışı

4.4 Grafik Veri Stratejisi (Chart Strategy)

Tarihsel grafik verisi, fiyat anlık görüntüsünden farklı bir zorluk içerir: her varlık türü ve zaman aralığı için farklı kaynaklar farklı kapsam ve çözünürlük sunar. Sistem bu sorunu, varlık türüne özel **grafik servisleri** ve kripto için **çok kaynaklı toplama (aggregation) ve geri çekilme (fallback)** stratejisiyle çözer.

4.4.1 Varlık Türüne Özel Grafik Servisleri

Servis / Bileşen	Varlık	Kaynak Yaklaşımı
`CryptoChartAggregator` (Binance/Yahoo/CoinGecko)	Kripto	Çok kaynaklı toplama ve fallback (bkz. 4.4.2)
`YahooChartClient` → `StockChartResponse`	Hisse / Endeks	Yahoo Finance grafik verisi (gün/saat çözünürlük)
`FundPriceHistoryResponse` (TEFAS)	Yatırım Fonu	TEFAS resmi fiyat geçmişi (5 yıla kadar)
`EurobondChartPoint` (Business Insider)	Eurobond	Business Insider grafik verisi
`TcmbFxHistoryClient` → `FxHistory`	Döviz	TCMB gün-gün kur geçmişi
`ViopChartService` (İş Yatırım)	ViOP	İş Yatırım grafik verisi
`EconomyChartService` (EVDS/FRED)	Ekonomi	TCMB EVDS ve FRED makro serileri

***Not (döviz grafiği tasarım kararı):** TCMB kur geçmişi gün-gün HTTP çağrısı gerektirdiğinden maliyetlidir; bu nedenle `@Cacheable` + LKG ile önbelleklenir ve uzun aralıklarda (5Y/10Y) üst sınır uygulanır.*

4.4.2 Kripto Grafiği için Kaynak Zinciri ve Toplama

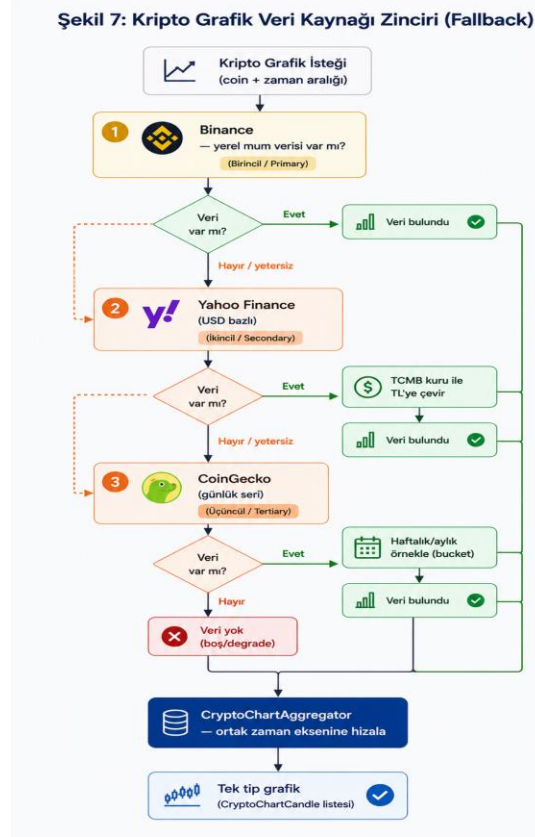
Kripto grafiği, en zengin ve en kırılğan veri alanıdır. Bir coin'in farklı zaman aralıklarındaki (1 gün – tüm zamanlar) verisi tek bir kaynaktan elde edilemeyebilir. Bu nedenle `CryptoChartAggregator` bileşeni, birden çok kaynağı sıralı bir öncelikte dener ve birleştirir:

- Birincil:** Binance yerel mum verisi (en güvenilir, TL paritesi varsa doğrudan).
- İkincil:** Yahoo Finance (USD bazlı; TCMB döviz kuruyla TL'ye çevrilir) — Binance'te bulunmayan veya uzun geçmiş gereken durumlar.
- Üçüncül:** CoinGecko (günlük seri; haftalık/aylık olarak örneklenir) — diğer kaynaklar yetersizse.

Aggregator ayrıca; farklı çözünürlükteki serileri ortak bir zaman eksenine hizalar, haftalık/aylık kovalara (bucket) örnekler ve tek tip bir `CryptoChartCandle` listesi üretir. Bu strateji, bir kaynağın eksik veya erişilemez olduğu durumlarda bile kullanıcıya kesintisiz grafik sunulmasını sağlar.

4.4.3 Tasarım Gerekeçesi

Bu strateji deseni; her varlık türünün kendine özgü veri kaynağı davranışını (kapsam, para birimi, çözünürlük) kapsülleyerek, grafik tüketen üst katmanların (controller, frontend) bu karmaşıklıktan habersiz kalmasını sağlar. Yeni bir kaynak veya çözünürlük kuralı eklemek, yalnızca ilgili grafik servisini veya aggregator'ı etkiler.



Şekil 7 — Kripto Grafik Kaynak Zinciri

4.5 Dayanıklılık Tasarımı (Resilience)

Sistem, 20'den fazla dış kaynağa bağımlı olduğundan, bu kaynakların kesinti, yavaşlama veya kota sınırı durumlarına karşı çok katmanlı bir dayanıklılık tasarımı uygular. Üç mekanizma birlikte çalışır: **Last Known Good (LKG)**, **Resilience4j (Retry + Circuit Breaker)** ve **ön bellek ısıtma (warm-up)**.

4.5.1 Last Known Good (LKG) — Birincil Dayanıklılık

Sistemin ana dayanıklılık deseni **Last Known Good (stale-if-error)** yaklaşımıdır ve `LastKnownGoodCache` bileşeni tarafından sağlanır. LKG, kod tabanında **24 farklı veri ad alanına** (crypto, economy, fx, fund vb.) bağlıdır ve **100'den fazla çağrı noktasında** kullanılır.

Çalışma mantığı:

1. Bir dış kaynaktan veri çekme denenir (`fetch`).
2. Çekim **başarılı ve boş değilse**, değer normal önbelleğe ve ayrıca uzun TTL'li bir `lkg:` anahtarına yazılır, sonra döndürülür.
3. Çekim **hata verir veya boş dönerse**, en son başarılı `lkg:` değeri servis edilir (eldeki "bayat ama geçerli" veri).

Bu yaklaşım; bir dış kaynak tamamen çökse bile, sistemin kullanıcıya en son geçerli veriyi sunarak çalışmaya devam etmesini sağlar. LKG anahtarları için TTL, kaynağın değişim hızına göre ayarlanır (hızlı değişen veri için ~3 gün, yavaş için ~14 gün).

4.5.2 Resilience4j — Yeniden Deneme ve Devre Kesici

Özellikle kırılgan ve yavaş kaynaklar için (örn. Yahoo Finance, kripto grafik) Resilience4j kütüphanesi kullanılır:

- **Yeniden Deneme (`@Retry`)**: Geçici hatalarda (timeout, anlık ağ sorunu) çağrı sınırlı sayıda tekrarlanır (kod tabanında 6 kullanım).
- **Devre Kesici (`@CircuitBreaker`)**: Bir kaynak ardışık olarak başarısız olduğunda, devre "açılır" ve bir süre boyunca o kaynağa çağrı yapılmaz; bu, çöken bir kaynağa boşuna istek yağdırmayı önler ve sistemi korur (kod tabanında 4 kritik noktada — Yahoo ve kripto istemcilerinde).

Resilience4j yapılandırması, kırılgan kaynaklar için adlandırılmış örnekler (instance) tanımlar. Örneğin `yahooApi` örneği; devre kesici için 20 boyutlu kayan pencere (sliding window) ve yeniden deneme için en fazla 2 deneme (maxAttempts) ile yapılandırılmıştır.

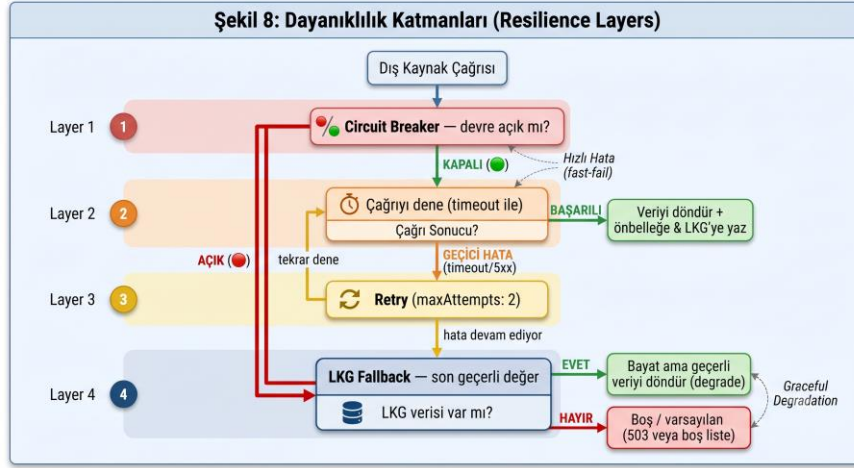
LKG ve Resilience4j birbirini tamamlar: Resilience4j çağrı seviyesinde geçici hataları yönetirken (retry) ve kalıcı arızalarda hızlı başarısızlık sağlarken (circuit breaker), LKG bu başarısızlık durumunda kullanıcıya servis edilecek bir veri (son geçerli değer) garanti eder.

4.5.3 Önbellek Isıtma (Cache Warm-up)

Büyük listelerin (tahvil ~1000+ kayıt, fon listeleri) ilk erişimde soğuk yük gecikmesini önlemek için `MarketListCacheWarmupService` ve `StockCacheWarmupService` periyodik ısıtma yapar:

- **Başlangıçta**: Uygulama hazır olduğunda (`ApplicationReadyEvent`) kritik listeler önceden yüklenir.
- **Periyodik**: Zamanlanmış görevlerle önbellek tazelenir. Isıtma; piyasa hareketleri (movers ~90 sn), fonlar (~8 dk), tahviller (~2 saat), ekonomi ve döviz (fx) listelerini kapsar (toplam 5 liste tipi).

Tasarım invaryantı: Önbellek ısıtma periyodu, ilgili önbelleğin TTL süresinden **kısa** tutulur; böylece veri TTL nedeniyle düşmeden önce yeniden ısıtılır ve kullanıcı hiçbir zaman soğuk önbellekle karşılaşmaz.



Şekil 8 — Dayanıklılık Katmanları

4.6 Hata Yönetimi ve İstisnalar

Sistem, hataları merkezi ve tutarlı biçimde yönetmek için **global istisna işleme (Global Exception Handling)** yaklaşımı kullanır. Tüm denetleyiciler genelinde tek bir `GlobalExceptionHandler`` (Spring `@RestControllerAdvice``) bileşeni, fırlatılan istisnaları yakalar ve standart bir yanıt formatına dönüştürür. Toplam 10 istisna işleyici (`@ExceptionHandler``) tanımlıdır.

4.6.1 İstisna → HTTP Durum Eşlemesi

İstisna	HTTP Durumu	Anlamı
<code>MethodArgumentNotValidException`</code>	400 Bad Request	İstek gövdesi doğrulama hatası (alan kuralları)
<code>IllegalArgumentException`</code>	400 Bad Request	Geçersiz parametre
<code>ResourceNotFoundException`</code>	404 Not Found	İstenen kaynak bulunamadı
<code>NoResourceFoundException`</code> / <code>NoHandlerFoundException`</code>	404 Not Found	Tanımsız uç nokta
<code>ResponseStatusException`</code>	İlgili durum kodu	Programatik olarak fırlatılan HTTP durumu (404 dışındakiler loglanır)
<code>UnsupportedViopContractException`</code>	422 Unprocessable Entity	Desteklenmeyen ViOP sözleşmesi
<code>AdminPolicyException`</code>	(yönetim kuralı ihlali)	Yönetici işlem politikası ihlali
<code>ExternalApiException`</code>	503 Service Unavailable	Dış servis erişilemez/başarısız

İstisna	HTTP Durumu	Anlamı
`Exception` (genel)	500 Internal Server Error	Yakalanmayan tüm beklenmeyen hatalar (catch-all)

4.6.2 Özel İstisna Sınıfları

Sistem, alan-özü hata durumlarını anlamlı biçimde temsil etmek için özel istisna sınıfları tanımlar:

- **`ResourceNotFoundException`**: Var olmayan bir kaynağa (portföy, alarm vb.) erişim.
- **`ExternalApiException`**: Bir dış veri kaynağına erişimin başarısız olması; LKG devreye girmediğinde 503 ile yansıtılır.
- **`UnsupportedViopContractException`**: Sistemde tanımlı olmayan bir ViOP sözleşmesi.
- **`AdminPolicyException`**: Yönetici işlemlerinde politika ihlali (örn. kendini banlama girişi).

4.6.3 Standart Hata Yanıtı ve Kimlik/Yetki Hataları

Tüm hatalar, başarı yanıtlarıyla aynı sarmalayıcı (`ApiResponse`) içinde döndürülür. Bu, ön yüzün her uç nokta için aynı işleme mantığını kullanmasını sağlar. Hata yanıtı; başarı bayrağı (`success: false`), açıklayıcı bir mesaj ve (doğrulama hatalarında) alan bazlı hata detaylarını içerir. Yanıt sözleşmesinin ayrıntıları Bölüm 7'de (API Tasarımı) verilmiştir.

Kimlik/Yetki Hataları: 401 (kimlik doğrulanmamış) ve 403 (yetkisiz) yanıtları, global istisna işleyicide değil, güvenlik katmanında (`SecurityConfig` — `AuthenticationEntryPoint` ve `AccessDeniedHandler`) üretilir. Bu, güvenlik kararlarının istek iş mantığına ulaşmadan, filtre seviyesinde verildiğini gösterir (bkz. Bölüm 8).

Tasarım ilkesi: Hassas bilgiler (yığın izleri, iç hata ayrıntıları, sır değerler) istemciye yansıtılmaz; bunlar yalnızca sunucu tarafı loglarına (yapılandırılmış log) yazılır.

4.7 Önbellekleme ve Zamanlanmış İşler

4.7.1 Önbellek Tasarımı (Redis)

Sistem, dış kaynaklardan okunan verilerin tekrar tekrar çekilmesini önlemek için Redis tabanlı, çok ad alanlı (50+ namespace) bir önbellek kullanır. Önbellek erişimi, Spring Cache soyutlaması (`@Cacheable`) üzerinden yapılır; her ad alanının **TTL süresi, verinin değişim hızına göre** ayrı ayrı yapılandırılır.

Önbellek Ad Alanı (örnek)	TTL	Gerekçe
Kripto fiyat (`market.crypto`)	60 sn	Çok hızlı değişen veri
Emtia spot (`market.commodity.spot`)	60 sn	Hızlı değişen
Hisse (`market.stocks`)	10 dk	Orta hızda değişen
Fon (`market.funds`)	10 dk	Günde birkaç kez değişen
Haber (`newsCache`)	5 dk	Akış verisi

Önbellek Ad Alanı (örnek)	TTL	Gerekçe
Korku & Hirs (`market.feargreed`)	2 saat	Günde bir güncellenir
TCMB döviz (`market.fx.tcmb`)	6 saat	Günde bir güncellenir
Fon geçmişi (`market.funds.history`)	6 saat	Tarihsel, seyrek değişir

Tüm TTL değerleri yapılandırma (env) üzerinden ayarlanabilir. Önbellek, Bölüm 4.5'teki LKG mekanizmasıyla ikinci bir dayanıklılık katmanı oluşturur (önbellekte yoksa LKG son geçerli değeri sunar).

4.7.2 Zamanlanmış Görevler (Schedulers)

Sistem, periyodik işleri **31 zamanlanmış görevle** (22 sınıfa dağılmış) otomatikleştirir. Bunlar iki türdür:

Sabit aralıklı (fixedDelay) görevler — önbellek ısıtma ve alarm:

Görev	Aralık	Sorumluluk
Alarm değerlendirme (`AlarmEvaluationService`)	60 sn	Aktif alarmları güncel fiyatla kontrol
Margin alarm değerlendirme (`MarginAlarmEvaluator`)	Periyodik	Marj/teminat alarmlarını kontrol
Piyasa hareketleri ısıtma	90 sn	Movers önbelleğini tazele
Fon listesi ısıtma	8 dk	Fon önbelleğini tazele
Tahvil/döviz ısıtma	2 saat	Tahvil ve FX önbelleğini tazele
Ekonomi ısıtma	4 saat	Makro veri önbelleğini tazele
Metal/emtia ısıtma (`GoldMarketService` vb.)	Periyodik	Altın/gümüş/değerli metal önbelleğini tazele

Zamanlı (cron) görevler — bakım ve bildirim:

Görev	Zaman (cron)	Sorumluluk
TCMB DİBS sınıflandırma	Her gün 06:20	Tahvil kategorilerini TCMB'den tazele
Eurobond ISIN tazeleme	Ayın 21'i 06:00	HMB ISIN listesini güncelle
Bülten özeti	Her gün 08:00	Günlük/haftalık/aylık dijest gönderimi
Tahvil/Eurobond/VİOP vade kapatma	Her gün 00:30–00:35	Vadesi dolan pozisyonları kapat (`ViopMaturityScheduler` vb.)
Geçici ban süresi dolumu (`TemporaryBanExpiryService`)	Periyodik	Süresi dolan geçici banları otomatik kaldır

4.7.3 Dağıtık Ortamda Görev Kilitleme (ShedLock)

Sistem birden fazla kopya (replica) ile çalıştığında, bir zamanlanmış görevin **yalnızca tek bir kopyada** çalışması gerekir (örn. bülten iki kez gönderilmemeli). Bu, **ShedLock** ile sağlanır: kod tabanında **23 görev** Redis tabanlı dağıtık kilitte korunur. Bir kopya görevi alırken kilidi tutar; diğer kopyalar o görevi atlar.

4.7.4 Zamanlayıcı İş Parçacığı Havuzu (Thread Pool)

Varsayılan Spring zamanlayıcısı tek iş parçacığı (single-thread) kullanır; bu, uzun süren bir görevin (örn. büyük liste ısıtma) diğer görevleri (örn. 60 sn'lik alarm kontrolü) bloke etmesine yol açabilir. Bu nedenle `SchedulerConfig` içinde **çok iş parçacıklı bir `ThreadPoolTaskScheduler`** tanımlanmıştır; böylece bağımsız görevler paralel çalışır ve birbirini geciktirmez. Havuz boyutu yapılandırılabilir olup varsayılan değeri 5 iş parçacığıdır (`scheduling.pool-size`).

4.8 Konfigürasyon ve Sır (Secret) Yönetimi

Sistem, "yapılandırma odaklı tasarım" (configuration-driven design) ilkesini benimser: tüm ortam-bağımlı değerler ve sırlar koddan ayrıştırılmıştır. Bu, aynı imajın farklı ortamlarda (geliştirme/üretim) yeniden derlenmeden çalıştırılmasını sağlar.

4.8.1 Profil Tabanlı Yapılandırma

Yapılandırma, Spring Profiles ile katmanlı biçimde yönetilir:

Dosya	Amaç
`application.yml`	Tüm ortamlar için ortak ayarlar ve varsayılanlar
`application-dev.yml`	Geliştirme ortamı (yüksek log seviyesi, yerel servisler)
`application-prod.yml`	Üretim ortamı (güvenli ayarlar, zorunlu sırlar)
`application-test.yml`	Test ortamı (Testcontainers vb.)

Aktif profil, `SPRING\_PROFILES\_ACTIVE` ortam değişkeniyle belirlenir. Kod tabanında **106'dan fazla ayar** ortam değişkeniyle (`\${ENV\_VAR:varsayılan}` deseni) geçersiz kılınabilir.

4.8.2 Sır (Secret) Yönetimi

Hassas değerler (API anahtarları, veritabanı şifreleri) **hiçbir zaman kaynak koda veya sürüm kontrolüne** yazılmaz. Ortama göre iki yöntem kullanılır:

- **Geliştirme:** Sırlar, sürüm kontrolüne dahil edilmeyen (gitignore) bir `.env.local` dosyasında tutulur; uygulama bunu `optional:file:.env.local` olarak isteğe bağlı içe aktarır. Bir API anahtarı boşsa, ilgili özellik zarif biçimde devre dışı kalır (örn. sohbet asistanı "şu an kullanılamıyor" yanıtı verir) — uygulama çökmez.

- **Üretim:** Sırlar, Kubernetes Secret nesneleri olarak yönetilir ve konteynerlere ortam değişkeni olarak (`secretKeyRef`) enjekte edilir (PostgreSQL, Keycloak, Grafana ve backend için tanımlı).

4.8.3 Üretimde Güvenli Başlatma (Fail-Loud)

Üretim profilinde, kritik kimlik bilgileri ve bağlantı adresleri için **varsayılan değer tanımlanmaz**. İlgili ortam değişkeni sağlanmamışsa, uygulama sessizce hatalı bir yapılandırma ile çalışmak yerine **başlangıçta bilinçli olarak başarısız olur (fail-loud)**. Bu, üretim ortamının yanlış/eksik yapılandırma ile ayağa kalkmasını önler.

Ayrıca, **15 yapılandırma sınıfı** (`@ConfigurationProperties`) ile ayarlar tip-güvenli biçimde gruplanır ve başlangıçta doğrulanır.

5. FRONTEND TASARIMI

Bu bölüm, React tabanlı ön yüz uygulamasının mimari yapısını; katman organizasyonunu, yönlendirme, durum yönetimi, grafik bileşenleri, tema ve uluslararasılaştırma (i18n) tasarımını açıklar.

5.1 Uygulama Mimarisi

Ön yüz; React 19, Vite ve React Router üzerine kurulu, **özellik tabanlı (feature-based)** bir tek sayfa uygulamasıdır (SPA). Kod, teknik türe göre değil, işlevsel özellik alanlarına göre organize edilmiştir.

5.1.1 Katman Organizasyonu

Katman (`src/`)	Sorumluluk
`features`	Özellik alanlarına göre sayfalar ve bileşenler (9 alan, 47 sayfa)
`app`	Uygulama kabuğu: yerleşim (`AppLayout`), başlık/altbilgi, erişim koruması (`ProtectedRoute`), ortak parçalar (bildirim zili, market şeridi, AI sohbet penceresi)
`components`	Özellik bağımsız paylaşılan UI bileşenleri (`common`, `instrument`, `shared` alt grupları)
`context`	Global durum sağlayıcıları (React Context)
`api`	Backend REST çağrı katmanı (Axios istemcisi)
`router`	Sayfa yönlendirme tanımları
`hooks`	Yeniden kullanılabilir özel kancalar (`useCardOrder`, `useChartDrawings`, `useSortable`, `useYAxisWheelZoom`)
`i18n`	Çok dilli metin yönetimi
`config`	Ortam ve uygulama yapılandırması
`lib` / `utils`	Yardımcı fonksiyonlar ve biçimlendiriciler

5.1.2 Özellik Alanları (Features)

Özellik Alanı	Sayfa Sayısı	İçerik
`market`	31	Tüm varlık türleri (hisse, kripto, döviz, fon, tahvil, ViOP, emtia, altın, endeks, ekonomi) — liste ve detay sayfaları
`auth`	4	Giriş, kayıt, e-posta doğrulama, callback
`admin`	3	Kullanıcı yönetimi panelleri
`portfolio`	3	Portföy, holdings, analiz
`news`	2	Haber listesi ve detay
`dashboard`	1	Özelleştirilebilir kontrol paneli
`alarms`	1	Alarm yönetimi
`notifications`	1	Bildirim merkezi
`profile`	1	Kullanıcı profili ve tercihler

Toplam 47 sayfa bileşeni, yönlendiricide 54 rota tanımıyla sunulur (bazı rotalar kimlik/yönlendirme amaçlıdır). `market` alanı, her varlık türü için ayrı bir alt klasör (bonds, crypto, fx, funds, futures, gold, commodities, economy, precious) içerir; bu, varlık türüne özel bileşenlerin izole kalmasını sağlar.

5.1.3 Uygulama Kabuğu ve Sağlayıcı Zinciri

Uygulama, giriş noktasında (App.jsx) iç içe geçmiş **7 Context sağlayıcısı** ile sarmalanır.

Sağlayıcılar, dıştan içe şu sırayla yerleştirilmiştir:

...

LanguageProvider

└─ AuthProvider

└─ ToastProvider

└─ ConfirmProvider

└─ WatchlistProvider

└─ PreferencesProvider

└─ ThemeProvider

...

En dışta dil sağlayıcısının bulunması, kimlik dahil tüm alt bileşenlerin çok dilli metinlere erişebilmesini sağlar. Sağlayıcıların altında, ortak yerleşimi (`AppLayout`: başlık, market şeridi, altbilgi, AI sohbet penceresi) sağlayan kabuk ve yönlendirici (router) yer alır.

5.2 Yönlendirme ve Sayfa Haritası

Uygulama, React Router ile istemci tarafı yönlendirme kullanır. Rotalar, erişim seviyesine göre **üç koruma katmanına** ayrılmıştır; bu, Bölüm 3.1'deki üç aktör (Misafir / Kayıtlı / Admin) modeliyle birebir örtüşür. Yönlendiricide toplam 50 rota tanımı bulunur.

5.2.1 Erişim Koruma Katmanları

Koruma Bileşeni	Erişim	Kapsam
(korumasız, `AppLayout` içinde)	Misafir + herkes	Haber, tüm piyasa sayfaları, giriş/kayıt
`ProtectedRoute` `requireEmailVerified={false}`	Kimliği doğrulanmış	Profil (e-posta doğrulaması gerekmeden)
`ProtectedRoute`	Kayıtlı kullanıcı	Kontrol paneli, portföy, alarm, bildirim
`AdminRoute`	Yönetici	Yönetici panelleri

Yönlendirme davranışı: `ProtectedRoute`, kullanıcı kimliği doğrulanmamışsa `/login`'e (geldiği sayfayı `state` ile koruyarak), e-posta doğrulaması gerekiyor ve yapılmamışsa `/verify-email`'e yönlendirir. `AdminRoute` ek olarak `isAdmin` kontrolü yapar; yönetici değilse `/unauthorized` sayfasına yönlendirir. Bu kademeli kontrol, yetkisiz erişimi istemci tarafında da engeller (asıl yetki kontrolü her zaman backend'de yapılır — bkz. Bölüm 8).

5.2.2 Genel (Public) Rotalar

Giriş gerektirmeyen, misafir kullanıcının erişebildiği rotalar — uygulamanın çoğunluğu:

Rota	Sayfa
`/` → `/news`	Açılışta haberlere yönlendirir
`/news`, `/news/:id`	Haber listesi ve detay
`/login`, `/register`, `/verify-email`, `/auth/callback`	Kimlik akışı
`/market/stocks`, `/market/stocks/:symbol`, `/market/stocks/compare`	Hisse listesi/detay/karşılaştırma
`/market/indices/:code`	Endeks detay
`/market/crypto`, `/market/crypto/:coinId`	Kripto liste/detay
`/market/futures`, `/market/futures/:symbol`	ViOP liste/detay
`/market/funds`, `/market/tefas`, `/market/tefas/:code`, `/market/tefas/compare`	Fon liste/detay/karşılaştırma
`/market/fx`, `/market/fx/:symbol`, `/market/fx/compare`	Döviz liste/detay/karşılaştırma

Rota	Sayfa
`/market/bonds`, `/market/bonds/:symbol`, `/market/bonds/global/:isin`	Tahvil ve Eurobond detay
`/market/economy`, `/market/economic-calendar`	Ekonomi ve takvim
`/market/kredi-hesaplama`, `/market/mevduat-hesaplama`	Kredi/mevduat hesaplayıcı
`/market/gold`, `/market/silver`, `/market/platinum`, `/market/palladium`	Kıymetli metaller
`/market/commodities`, `/market/commodities/:symbol`, `/market/commodities/compare`	Emtia liste/detay/karşılaştırma
`/market/cizimlerim`	Kullanıcı grafik çizimleri
`*`	404 (yerleşim içinde)

5.2.3 Korumalı (Kayıtlı Kullanıcı) Rotalar

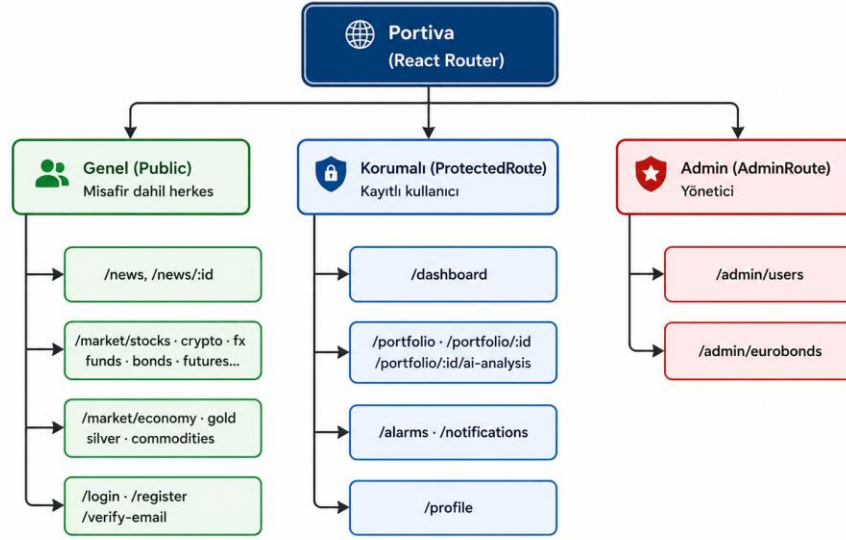
Rota	Sayfa
`/dashboard`	Özelleştirilebilir kontrol paneli
`/portfolio`, `/portfolio/:id`, `/portfolio/:id/ai-analysis`	Portföy, detay, AI analiz
`/alarms`	Alarm yönetimi
`/notifications`	Bildirim merkezi
`/profile`	Profil (e-posta doğrulaması olmadan da erişilebilir)

5.2.4 Yönetici (Admin) Rotalar

Rota	Sayfa
`/admin` → `/admin/users`	Yönetim paneli (kullanıcılara yönlendirir)
`/admin/users`	Kullanıcı yönetimi
`/admin/eurobonds`	Eurobond ISIN yönetimi

5.2.5 Geriye Dönük Uyumluluk

Eski `/market/compare` rotası, sorgu dizesini (query string) koruyarak yeni `/market/fx/compare` rotasına yönlendirilir (`FxCompareRedirect`). Bu, eski yer imlerinin ve bağlantıların kırılmamasını sağlar.



Şekil 9: Sayfa Haritası / Route Ağacı

5.3 Durum Yönetimi ve Veri Çekme Katmanı

Uygulama, global durumu **React Context** ile, sunucu verisini ise merkezi bir **HTTP istemci katmanı** ile yönetir. Harici bir durum yönetimi kütüphanesi (Redux vb.) yerine, yerleşik Context API tercih edilmiştir; bu, uygulamanın ölçeği için yeterli ve daha az bağımlılık sağlar.

5.3.1 Global Durum (Context)

Yedi Context sağlayıcısı, farklı global durum alanlarını yönetir:

Context	Sorumluluk
<code>`AuthContext`</code>	Kimlik durumu (<code>`isAuthenticated`</code> , <code>`isAdmin`</code> , <code>`emailVerified`</code>), oturum yaşam döngüsü
<code>`ThemeContext`</code>	Tema (açık/koyu) durumu ve uygulanması
<code>`LanguageContext`</code>	Aktif dil (TR/EN) ve metin çevirisi
<code>`PreferencesContext`</code>	Sunucuda saklanan arayüz tercihlerini (kontrol paneli düzeni, dil, ticker, grafik düzenleri) uygulama render edilmeden önce çeken hidrasyon (yükleme kapısı)
<code>`WatchlistContext`</code>	İzleme listesi öğelerini bir kez yükler; <code>`varlık türü, sembol`</code> → kayıt eşlemesi tutar (liste sayfalarındaki yıldız durumları için)
<code>`ToastContext`</code>	Geçici bildirim (toast) gösterimi
<code>`ConfirmContext`</code>	Tarayıcının yerleşik <code>`window.confirm`</code> yerine site temasına uygun (M3) onay diyalogu

Tasarım notu (tercih hidrasyonu): `PreferencesContext`, kullanıcının sunucudaki tercihlerini bileşenler ekrana gelmeden önce yükler (kısa bir yükleme kapısı). Böylece bileşenler ilk render'da senkron olarak doğru değerleri (örn. kontrol paneli düzeni) okur; bu, cihazlar arası tutarlı bir arayüz sağlar.

5.3.2 Veri Çekme Katmanı (API)

Sunucu çağrıları, özellik alanına göre ayrılmış **13 API modülünden** yapılır: `marketApi`, `portfolioApi`, `alarmApi`, `newsApi`, `assistantApi`, `notificationApi`, `newsletterApi`, `supportApi`, `adminApi`, `authApi`, `meApi`, `ipoApi`, `prefs`. Her modül, kendi alanının uç noktalarını sarmalayan fonksiyonlar içerir. Tüm istekler, merkezi bir Axios istemcisinden (`src/lib/http.js`) geçer.

5.3.3 Kimlik ve Token Yönetimi (Interceptor)

Merkezi HTTP istemcisi, kimlik akışını şeffaf biçimde yöneten interceptor'lar içerir:

- **İstek interceptor'ı:** Her korumalı isteğe geçerli erişim belirtecini (access token) `Authorization: Bearer` başlığı olarak ekler. Belirtecini süresi dolmuş veya dolmak üzereyse, istek gönderilmeden önce belirteç sessizce yenilenir.
- **Yanıt interceptor'ı (401 kurtarma):** Bir istek 401 (yetkisiz) dönerse, kullanıcı hemen çıkışa zorlanmaz; bunun yerine belirteç **bir kez sessizce yenilenir ve istek otomatik tekrarlanır** (`\_retried` bayrağıyla sonsuz döngü önlenir). Yenileme de başarısız olursa oturum sonlandırılır ve kullanıcı giriş sayfasına yönlendirilir; e-posta doğrulaması gereken durumda `/verify-email` sayfasına yönlendirilir.
- **Tek-uçuşan yenileme (single-flight refresh):** Aynı anda birden çok istek belirteç yenilemesi gerektirdiğinde, hepsi **tek bir yenileme çağrısını paylaşır**. Bu, eşzamanlı isteklerin çoklu (ve yarış halinde) yenileme tetiklemesini ve belirteç rotasyonunda tutarsızlık oluşmasını önler.

Belirteçler tarayıcı yerel depolamasında (localStorage) üç anahtarla saklanır: erişim belirteci (`auth\_token`), kimlik belirteci (`id\_token`) ve yenileme belirteci (`refresh\_token`); yenileme belirteci rotasyonu desteklenir.

5.4 Grafik Bileşenleri Mimarisi

Sistem, finansal bir uygulama olarak yoğun biçimde grafik kullanır. Farklı grafik ihtiyaçları için **üç ayrı grafik kütüphanesi** kullanılır; her biri kendi güçlü olduğu alanda tercih edilmiştir.

5.4.1 Grafik Kütüphaneleri ve Kullanım Alanları

Kütüphane	Kullanım	Tercih Gerekçesi
klinecharts (18 bileşen)	Varlık detay sayfalarındaki finansal mum (candlestick) ve çizgi grafikleri	Profesyonel finansal grafik özellikleri: teknik indikatörler, çizim araçları, etkileşim

Kütüphane	Kullanım	Tercih Gerekçesi
ECharts (41 bileşen)	Karşılaştırma/kıyas sayfaları ve çok serili grafikler	Esnek, performanslı, çok eksenli karşılaştırma
Recharts (19 bileşen)	İstatistiksel ve analitik grafikler (AI analiz, portföy dağılımı, hesaplayıcılar)	React-yerel, basit deklaratif API

5.4.2 Finansal Grafikler (klinecharts)

Varlık detay sayfaları (hisse, kripto, döviz, fon, tahvil, altın, emtia), klinecharts tabanlı bir mum grafiği etrafında kurgulanmıştır. Bu grafikler şu yetenekleri sunar:

- **Teknik indikatörler:** Hareketli Ortalama (MA), RSI, MACD, Bollinger Bantları (BOLL) gibi standart göstergeler (`createIndicator`).
- **Çizim araçları (overlay):** Kullanıcı, grafik üzerine trend çizgisi, seviye vb. çizebilir (`createOverlay`).
- **Zaman aralığı seçimi:** 1G – Tüm zamanlar arası dönem değiştirme.

5.4.3 Grafik Çizim Kalıcılığı (useChartDrawings)

Kullanıcının grafik üzerine yaptığı çizimler (overlay), `useChartDrawings` özel kancası ile kalıcı hale getirilir. Her çizim değişikliğinde (pointer hareketi sonu), grafikteki tüm canlı overlay'ler okunur ve tek bir JSON listesi olarak saklanır:

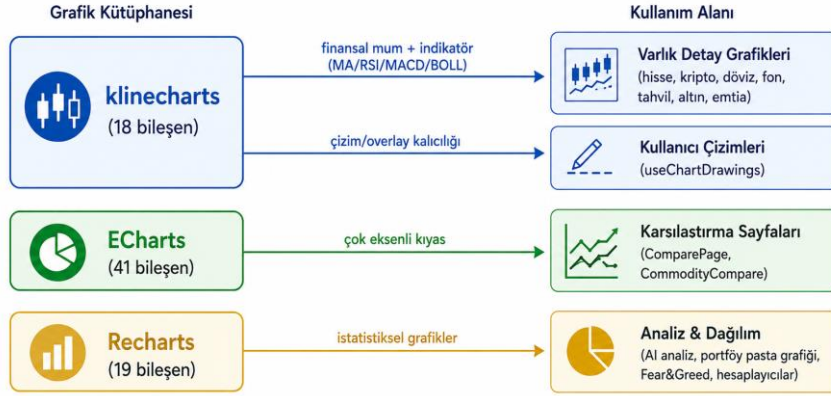
- **Yerel:** Tarayıcı yerel depolamasına (`chart-overlays:<varlık>` anahtarıyla).
- **Sunucu (giriş yapılmışsa):** Kullanıcı tercihleri senkronizasyonu üzerinden, böylece çizimler cihazlar arasında korunur.

Bu sayede kullanıcı, bir varlığın grafiğine çizdiği analizi daha sonra (hatta başka bir cihazda) yeniden görebilir. Çizilen grafikler ayrıca özel bir "Çizimlerim" sayfasında (`/market/cizimlerim`) toplanır.

5.4.4 İstatistiksel ve Karşılaştırmalı Grafikler

- **Karşılaştırma (ECharts):** Birden çok enstrümanın aynı grafikte kıyaslandığı sayfalar (örn. `ComparePage`, `CommodityComparePage`) çok eksenli ECharts grafikleri kullanır.
- **Analiz (Recharts):** Portföy varlık dağılımı (pasta), AI analiz ekranları, Korku & Hırs grafiği ve hesaplayıcı çıktıları Recharts ile çizilir.

Şekil: Grafik Bileşen Mimarisi (Kütüphane → Kullanım Alanı)



Şekil 10: Grafik Bileşen Mimarisi

5.5 Tema Sistemi

Uygulama, açık (light) ve koyu (dark) tema desteği sunar. Tema, **CSS değişkenleri (design token)** tabanlı bir yaklaşımla uygulanır.

5.5.1 Tema Uygulama Mekanizması

Tema, kök `<html>` elemanına `dark` sınıfının eklenip çıkarılmasıyla değiştirilir. `index.css` içindeki `.dark` kuralları, bu durumda devreye giren bir CSS değişkenleri kümesini tanımlar.

5.5.2 Tasarım Token'ları (Design Tokens)

Renk ve biçim değerleri doğrudan bileşenlere yazılmaz; merkezi CSS değişkenleri (token) üzerinden tanımlanır. Başlıca token'lar: `--background`, `--foreground`, `--primary`, `--card`, `--muted`, `--border`, `--ring`, `--radius`. Bu yaklaşım; tema değişiminin tek noktadan yönetilmesini ve görsel tutarlılığı sağlar.

5.5.3 Flaş Önleme (FOUC) ve Kalıcılık

Tema tercihi, `app_theme` anahtarıyla tarayıcı yerel depolamasına anında (senkron) yazılır. Sayfa yüklenirken, React render başlamadan **önce** (main.jsx içinde erken çağrı) tema okunur ve uygulanır; böylece kullanıcı, yanlış temanın bir an görünüp düzelmesi (Flash of Unstyled Content / FOUC) sorununu yaşamaz. Tema tercihi ayrıca kullanıcı tercihleri senkronizasyonu ile cihazlar arasında taşınır.

5.6 Uluslararasılaştırma (i18n) Mimarisi

Uygulama, Türkçe (TR) ve İngilizce (EN) dillerini destekler. Çok dillilik, harici bir i18n kütüphanesi yerine **modüler, namespace tabanlı kendi çeviri altyapısı** ile yönetilir.

5.6.1 Çeviri Organizasyonu

Çeviri metinleri, özellik alanına göre **14 namespace dosyasına** bölünmüştür: `common`, `header`, `footer`, `layout`, `home`, `market`, `portfolio`, `news`, `dashboard`, `auth`, `profile`, `admin`, `analysis`, `extra`. Her namespace, kendi TR ve EN çevirilerini bir arada tutar. Bu modüler yapı; çevirilerin ilgili özelliklerle birlikte bakımını ve büyük bir tek dosyanın yönetilmesini önlemeyi sağlar.

Aktif dil `LanguageContext` ile yönetilir ve bileşenler, bir çeviri fonksiyonu üzerinden namespace ve anahtara göre metinlere erişir.

5.6.2 Yerel Biçimlendirme (Locale Formatting)

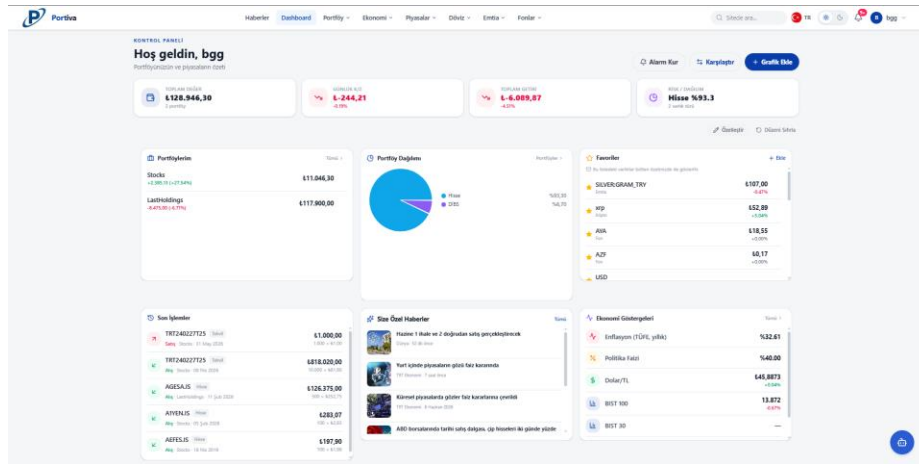
Dilin yanı sıra, sayı, para birimi ve tarih biçimlendirmesi de yerelleştirilir. Yardımcı modüller (`numberFormat`, `dateUtils`, varlık-özel biçimlendiriciler), `toLocaleString` ve `Intl` API'leri ile `tr-TR` / `en-US` yerellerine uygun biçimlendirme yapar (örn. binlik ayraç, ondalık, para birimi sembolü, tarih formatı). Bu, finansal değerlerin her dilde doğru ve okunabilir gösterilmesini sağlar.

5.7 Temsili Ekran Görüntüleri

Bu bölüm, yukarıda tasarımı açıklanan ön yüz mimarisinin çalışan uygulamadaki somut karşılığını gösterir. Aşağıdaki ekran görüntüleri, sistemin gerçek (canlı) ortamından alınmıştır.

5.7.1 Kontrol Paneli (Dashboard)

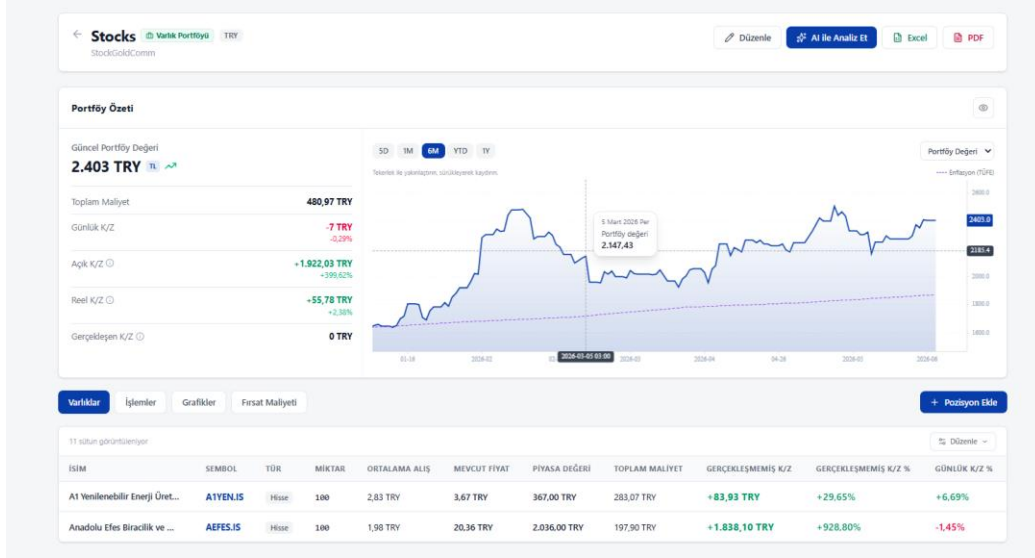
Özelleştirilebilir, sürükle-bırak ızgara tabanlı kontrol paneli — kullanıcının piyasa kartlarını ve portföy özetini düzenleyebildiği ana ekran.



Şekil 11: Kullanıcı Dashboardı

5.7.2 Portföy Görünümü

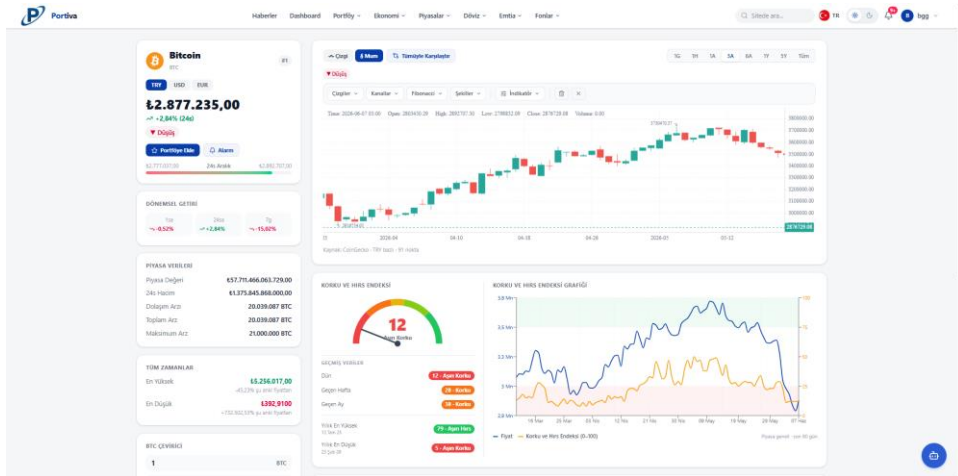
Portföy holdings tablosu; güncel değer, kâr/zarar, varlık dağılımı (pasta grafik) ve performans grafiğiyle.



Şekil 12: Kullanıcı Portföyü

5.7.3 Varlık Detay ve Grafik

Bir varlığın (örn. hisse veya kripto) detay sayfası; klinecharts tabanlı mum grafiği, teknik indikatörler ve zaman aralığı seçimiyle.



Şekil 13: Enstünman Detay Sayfası

5.7.4 Yapay Zekâ Portföy Analizi

Portföyün yapay zekâ destekli analiz ekranı; risk/sağlık skoru, çeşitlendirme ve istatistiksel grafiklerle.

AI Yorum Raporu

Teşhis

Bu portföy, hisse senetlerine %100 yoğunlaşmış ve tek bir varlık olan Anadolu Efes'te %84.73 gibi yüksek bir oranda toplanmış olması nedeniyle oldukça agresif bir yapı sergilemektedir. Bu aşırı yoğunlaşma, portföyün yıllık volatilitisini %38.77 gibi yüksek bir seviyeye taşımış ve 2008 Küresel Finans Krizi'nde %50.00, Mart 2020 Covid Çöküşü'nde ise %29.32 gibi ciddi kayıplara yol açabileceğini göstermiştir. Nominal olarak %399.62 getiri sağlamasına rağmen, yüksek enflasyon (%388.02) nedeniyle reel getirisi yalnızca %2.38'de kalmış ve BIST 100, S&P 500 ve Altın gibi önemli benchmark'ların gerisinde kalmıştır.

Portföy Kimliği

Portföy, %100 büyüme odaklı varlık ağırlığı ile "Agresif" bir profile sahiptir. Bu yapı, yüksek getiri potansiyeli barındırmakla birlikte, piyasa dalgalanmalarına karşı da oldukça hassastır. Genellikle yüksek risk toleransına sahip, uzun vadeli yatırım ufku olan ve sermayesinde önemli düşüşleri göze alabilen yatırımcılar için uygun bir profil çizer. Korumacı varlıklara hiç yer verilmemesi, risk iştahının çok yüksek olduğunu göstermektedir.

Güçlü Yönler

- Enflasyona Karşı Korumacı: Portföy, dönem TÜFE enflasyonu olan %388.02'yi %11.60'lık bir farkla geride bırakarak reel getiri sağlamayı başarmıştır (%2.38). Bu, enflasyonist ortamda satın alma gücünü koruma yeteneğini göstermektedir.
- Dolar ve Mevduat Getirisini Aşma: Dolar ve mevduat getirilerine kıyasla sırasıyla %112.29 ve %154.47'lik önemli farklarla önde performans göstermiştir. Bu durum, pasif yatırım araçlarına göre daha yüksek getiri potansiyelini ortaya koymaktadır.
- Yüksek Nominal Getiri: Portföyün nominal getirisi %399.62 gibi oldukça yüksek bir seviyededir. Bu, özellikle ana varlık olan Anadolu Efes'in %928.80'lik etkileyici performansından kaynaklanmaktadır.
- Pozitif Risk-Getiri Oranları: Sharpe oranı 1.12 ve Sortino oranı 1.49, alınan risk başına elde edilen getirinin makul seviyelerde olduğunu işaret etmektedir, ancak bu oranlar yüksek volatilité bağlamında değerlendirilmelidir.

Riskler & Bağlantılar

- Aşırı Yoğunlaşma Riski: Portföyün en büyük riski, %100 hisse senedi dağılımı ve ilk üç pozisyonun %100'ünü oluşturmalarıdır. Özellikle Anadolu Efes'e olan %84.73'lük yoğunlaşma, portföyü tek bir şirketin performansına ve sektör risklerine karşı son derece savunmasız hale getirmektedir.
- Yüksek Volatilité ve Maksimum Düşüş: Bu yoğunlaşma, yıllık volatilitéyi %38.77 gibi yüksek bir seviyeye çıkarmıştır. Geçmişte yaşanan %16.24'lük maksimum düşüş, portföyün kısa sürede önemli değer kayıpları yaşayabileceğini göstermektedir.
- Stres Testi Sonuçları: Tarihsel stres testleri, yoğunlaşma ve volatilité arasındaki güçlü bağlantıyı ortaya koymaktadır. 200

© Bu analiz bilgilendirme amaçlıdır, yatırım tavsiyesi değildir.

Şekil 14: Yapay Zeka Portföy Analizi Sayfası

5.8 Dışa Aktarma (Export)

Kullanıcı, çeşitli ekranlardaki veriyi ve grafikleri dışa aktarabilir. Bu, istemci tarafında (tarayıcıda) gerçekleştirilir; sunucuya ek yük bindirmez. Üç format desteklenir:

Format	Kütüphane	Kullanım
Excel (.xlsx)	`xlsx`	Portföy, liste ve yönetim verilerinin tablo olarak dışa aktarımı
PDF	`jsPDF`	Raporların/ekranların PDF belgesi olarak alınması
Görsel (PNG)	`html-to-image`	Grafiklerin ve panellerin resim olarak dışa aktarımı

Bu özellik; portföy detayı, yönetim panelleri ve grafik ekranları gibi yerlerde "Rapor AI / Dışa Aktar" eylemleriyle sunulur.

5.9 Sayfalama (Pagination)

Büyük listeler (piyasa varlık listeleri, yönetici kullanıcı listesi vb.), performans ve kullanılabilirlik için **sayfalanır**. Sayfalama iki uçta birlikte çalışır:

- **Backend:** Liste uç noktaları, Spring `Pageable`/`PageRequest` ile sayfa bazlı veri döndürür (sayfa numarası, boyut, toplam).
- **Frontend:** Ortak bir `Pagination` bileşeni (ve yönetici için `AdminUsersPagination`) tüm liste sayfalarında tutarlı bir sayfalama arayüzü sağlar.

Bu, binlerce kayıtlı listelerin (örn. tahvil ~1000+) tek seferde yüklenmesini önler.

6. VERİ TASARIMI

Bu bölüm, sistemin kalıcı veri yapısını; varlık-ilişki modelini, veritabanı şemasını, bütünlük kurallarını, şema sürümleme (migration) tasarımını ve önbellek tasarımını açıklar.

6.1 Mantıksal Veri Modeli (ER)

Sistem, kullanıcıya ait kalıcı verileri PostgreSQL'de **10 tabloda** saklar. Piyasa ve haber verileri kalıcı tutulmaz; yalnızca önbellekte geçici olarak bulunur (bkz. 6.5). Kullanıcı kimliği Keycloak/LDAP'ta yönetildiğinden, veritabanında ayrı bir kullanıcı tablosu yoktur; tablolar kullanıcıya `user\_id` (Keycloak kimliği) üzerinden mantıksal olarak bağlanır.

6.1.1 Varlıklar ve İlişkiler

Tablo	Amaç	Anahtar İlişki
`portfolio`	Kullanıcı portföyü (üst kayıt)	—
`portfolio_transaction`	Portföy işlemleri (al/sat)	→ `portfolio` (FK, CASCADE)
`watchlist_item`	İzleme listesi öğeleri	→ `portfolio` (FK, CASCADE)
`alarm`	Fiyat/değişim/hacim alarmları	— (user_id ile)
`notification`	Uygulama içi bildirimler	→ `alarm` (FK, SET NULL)
`newsletter_subscription`	Bülten abonelikleri	— (user_id ile)
`support_ticket`	Destek talepleri	— (user_id ile)
`user_preference`	Kullanıcı arayüz tercihleri	— (user_id ile)
`user_margin_alert_setting`	Marj alarm ayarı	— (user_id ile)
`user_ban_state`	Kullanıcı ban durumu	— (keycloak_user_id ile)

6.1.2 Gerçek Yabancı Anahtar (FK) İlişkileri

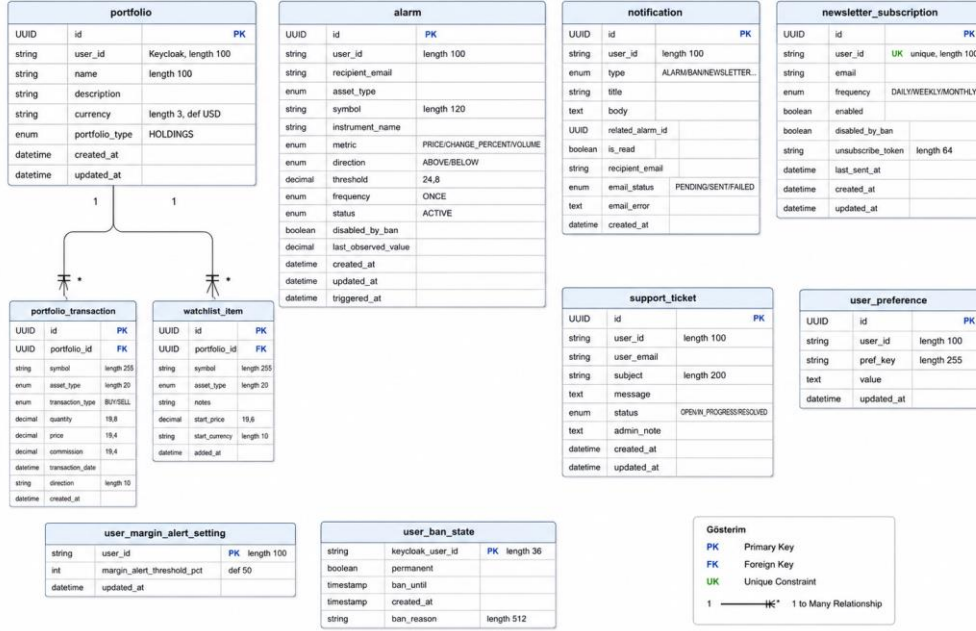
Veritabanı düzeyinde tanımlı FK ilişkileri yalnızca aynı veritabanındaki tablolar arasındadır:

- `portfolio\_transaction.portfolio\_id` → `portfolio.id` (ON DELETE CASCADE): Bir portföy silindiğinde işlemleri de silinir.
- `watchlist\_item.portfolio\_id` → `portfolio.id` (ON DELETE CASCADE): Aynı şekilde izleme öğeleri de silinir.

- `notification.related\_alarm\_id → alarm.id` (ON DELETE SET NULL): Bir alarm silinse bile bildirim kaydı korunur (ilgili alarm referansı boşaltılır).

Diğer tüm tablolar kullanıcıya `user\_id` / `keycloak\_user\_id` ile mantıksal olarak bağlıdır (Keycloak harici kimlik). Bu nedenle bu tablolar için veritabanı düzeyinde bir "kullanıcı" FK'sı tanımlanmaz.

Veritabanı Varlık-İlişki (ER) Diyagramı



Şekil 15: ER Diyagramı

6.2 Veritabanı Şeması

Veritabanı şeması; UUID birincil anahtarlar, uygun veri tipleri, bütünlük kısıtları (constraint) ve performans indeksleri ile tasarlanmıştır.

6.2.1 Tasarım Standartları

- **Birincil Anahtar:** Tüm tablolarda UUID kullanılır; `pgcrypto` eklentisi ile `gen\_random\_uuid()` varsayılan değer olarak atanır.
- **Parasal/Miktar Değerleri:** `NUMERIC` (sabit nokta) kullanılır — kayan nokta yuvarlama hatalarını önlemek için. Örneğin miktar `NUMERIC(19,8)` (8 ondalık — krypto hassasiyeti), fiyat `NUMERIC(19,4)`.
- **Zaman Damgaları:** `TIMESTAMP` tipi; `created\_at` için `CURRENT\_TIMESTAMP` varsayılanı.
- **Enum Değerleri:** `VARCHAR` olarak saklanır (örn. `asset\_type`, `transaction\_type`, alarm durumu) — uygulama katmanında enum ile eşlenir.

6.2.2 Örnek Tablo Şeması (`portfolio\_transaction`)

```sql

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

portfolio_id UUID NOT NULL REFERENCES portfolio(id) ON DELETE CASCADE,

symbol VARCHAR(20) NOT NULL,

asset_type VARCHAR(20) NOT NULL, -- STOCK/FUND/FX/FUTURE/CRYPTO...

transaction_type VARCHAR(10) NOT NULL, -- BUY/SELL

quantity NUMERIC(19,8) NOT NULL CHECK (quantity > 0),

price NUMERIC(19,4) NOT NULL CHECK (price > 0),

commission NUMERIC(19,4) NOT NULL DEFAULT 0 CHECK (commission >= 0),

transaction_date TIMESTAMP NOT NULL,

created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

...
```

Bu örnek; `CHECK` kısıtlarının (miktar/fiyat pozitif, komisyon negatif olamaz) veri bütünlüğünü veritabanı düzeyinde nasıl güvence altına aldığını gösterir.

### 6.2.3 İndeksler

Sorgu performansı için **17 indeks** tanımlıdır. Başlıca indeksler:

| İndeks                                                        | Tablo                 | Amaç                                     |
|---------------------------------------------------------------|-----------------------|------------------------------------------|
| `idx_portfolio_user_id`                                       | portfolio             | Kullanıcının portföylerini listeleme     |
| `ux_portfolio_user_name` (UNIQUE)                             | portfolio             | Kullanıcı başına benzersiz portföy adı   |
| `idx_portfolio_user_type`                                     | portfolio             | Tür bazlı portföy filtreleme             |
| `idx_tx_portfolio_id`                                         | portfolio_transaction | Portföy işlemlerini çekme                |
| `idx_tx_portfolio_asset_symbol`                               | portfolio_transaction | Varlık/sembol bazlı işlem sorgusu        |
| `idx_tx_portfolio_date`                                       | portfolio_transaction | Tarihe göre işlem sıralama               |
| `idx_watchlist_portfolio_id`                                  | watchlist_item        | İzleme öğelerini çekme                   |
| `idx_alarm_user_id`, `idx_alarm_status`                       | alarm                 | Kullanıcı/durum bazlı alarm sorgusu      |
| `idx_notification_user_id`,<br>`idx_notification_user_unread` | notification          | Bildirim listeleme /<br>okunmamış sayısı |

| İndeks                                | Tablo                   | Amaç                              |
|---------------------------------------|-------------------------|-----------------------------------|
| `idx_newsletter_enabled`              | newsletter_subscription | Aktif aboneleri çekme             |
| `idx_user_ban_state_temporary_expiry` | user_ban_state          | Süresi dolan geçici banları bulma |

### 6.3 Veri Bütünlüğü Kuralları

Veri bütünlüğü, hem veritabanı kısıtları (constraint) hem de uygulama katmanı doğrulamasıyla iki seviyede güvence altına alınır.

#### 6.3.1 Değer Kısıtları (CHECK)

Anlamsız veri değerlerini veritabanı düzeyinde engellemek için `CHECK` kısıtları tanımlanmıştır:

| Kısıt                                                | Tablo                     | Kural                        |
|------------------------------------------------------|---------------------------|------------------------------|
| `quantity > 0`                                       | portfolio_transaction     | İşlem miktarı pozitif olmalı |
| `price > 0`                                          | portfolio_transaction     | İşlem fiyatı pozitif olmalı  |
| `commission >= 0`                                    | portfolio_transaction     | Komisyon negatif olamaz      |
| `transaction_type IN ('BUY','SELL','COUPON_INCOME')` | portfolio_transaction     | Geçerli işlem türü           |
| `portfolio_type IN ('HOLDINGS','WATCHLIST')`         | portfolio                 | Geçerli portföy türü         |
| `margin_alert_threshold_pct BETWEEN 0 AND 100`       | user_margin_alert_setting | Eşik yüzde 0–100 arası       |

*\*\*Not:\*\* Alarm/bildirim gibi diğer enum alanlarının (durum, metrik, yön) değer kontrolü veritabanında `CHECK` yerine uygulama katmanındaki enum eşlemesiyle yapılır; bu alanlar `VARCHAR` olarak saklanır.*

#### 6.3.2 Benzersizlik Kısıtları (UNIQUE)

| Kısıt                                | Tablo                   | Anlamı                                       |
|--------------------------------------|-------------------------|----------------------------------------------|
| `(user_id, name)`                    | portfolio               | Kullanıcının iki aynı isimli portföyü olamaz |
| `(portfolio_id, symbol, asset_type)` | watchlist_item          | Aynı varlık izleme listesinde tekrarlanamaz  |
| `(user_id, pref_key)`                | user_preference         | Her tercih anahtarı kullanıcı başına tek     |
| `user_id`                            | newsletter_subscription | Kullanıcı başına tek bülten aboneliği        |
| `unsubscribe_token`                  | newsletter_subscription | Benzersiz abonelik iptal anahtarı            |

#### 6.3.3 İki Katmanlı Doğrulama

Veri bütünlüğü iki seviyede sağlanır:

- **Uygulama katmanı:** Gelen istekler DTO doğrulaması (alan kuralları) ve iş kuralı kontrollerinden geçer (örn. aşırı-satış kontrolü). Bu, kullanıcıya anlamlı hata mesajları döndürür.
- **Veritabanı katmanı:** Yukarıdaki CHECK/UNIQUE/FK kısıtları, uygulama katmanı atlansa bile (örn. doğrudan veritabanı erişimi) verinin tutarlı kalmasını garanti eder.

Bu çift katman, "derinlemesine savunma" (defense in depth) ilkesini veri bütünlüğüne uygular.

## 6.4 Şema Sürümleme ve Migration (Flyway)

Veritabanı şeması, **Flyway** ile sürüm kontrollü biçimde yönetilir. Her şema değişikliği, sıralı ve değiştirilemez bir migration betiği (SQL) olarak tanımlanır; böylece tüm ortamlarda (geliştirme/test/üretim) şema aynı adımlarla, deterministik biçimde oluşur. Migration betikleri `db/migration` dizininde tutulur.

### 6.4.1 Migration Yaklaşımı

- Migration betikleri sürüm öneki ile adlandırılır (`V<sürüm>\_\_<açıklama>.sql`) ve **sıralı** uygulanır.
- Uygulanmış migration'lar Flyway'in şema geçmiş tablosunda kaydedilir; bir betik uygulandıktan sonra **değiştirilmez** (yeni değişiklik = yeni betik).
- Bu yaklaşım, şema değişikliklerinin **izlenebilir** ve **geri takip edilebilir** olmasını sağlar.

### 6.4.2 Şema Evrimi

Kod tabanında **17 migration betiği** bulunur. Bunların kronolojisi, sistemin nasıl evrildiğini gösterir:

- **V1–V3:** Temel — eklentiler (pgcrypto), portföy ve işlem tabloları.
- **V4–V7:** Portföy türü, izleme listesi, izleme başlangıç fiyatı, sembol genişletme.
- **V8–V12:** Ban durumu, alarm/bildirim, bülten, ban gerekçesi ve cascade (disabled\_by\_ban) alanları.
- **V20260525+ (tarih bazlı):** Kullanıcı tercihleri, destek talepleri, kupon geliri işlem türü, VIOP yönü ve marj alarm ayarı.

Sürüm numaralandırmasının tarih tabanlı biçime (`V20260525\_01`) geçmesi, eşzamanlı geliştirme sırasında sürüm çakışmalarını önler.

Şekil: Flyway Migration Zaman Çizelgesi (Şema Evrimi)



Şekil 16: Flyway Migration

## 6.5 Önbellek Tasarımı (Redis)

Sistem, Redis'i iki amaçla kullanır: **performans önbelleği** (dış kaynak verilerini hızlandırma) ve **dağıtık kilit** (ShedLock — `RedisLockProvider`). Kalıcı kullanıcı verisi PostgreSQL'de; geçici/okuma-ağırlıklı veri ise Redis'tedir.

### 6.5.1 Önbellek Yapılandırması

Önbellek, Spring `RedisCacheManager` ile yönetilir. Değerler **JSON formatında** serileştirilir (`GenericJackson2JsonRedisSerializer`), anahtarlar düz metin (`StringRedisSerializer`) olarak tutulur. Redis bağlantı bilgileri (host, port) ortam değişkenleriyle yapılandırılır.

### 6.5.2 Anahtar (Key) Desenleri

Önbellek anahtarları, ad alanı (namespace) ve parametre bazlı bir desenle üretilir. Spring Cache, anahtar metot parametrelerinden (SpEL ifadeleriyle) türetir. Örnekler:

| Ad Alanı        | Anahtar Deseni                                   | Açıklama                         |
|-----------------|--------------------------------------------------|----------------------------------|
| Eurobond detay  | `#isin`                                          | ISIN bazlı                       |
| Eurobond grafik | `#isin + ':' + #range`                           | ISIN + zaman aralığı             |
| ViOP grafik     | `#instrumentCode + '_' + #period`                | Sözleşme + periyot               |
| Kripto grafik   | `#coinId + ':v5:' + #days + ':' + #currency`     | Coin + sürüm + gün + para birimi |
| Metal grafik    | `#metal.name() + ':' + #range + ':' + #currency` | Metal + aralık + para birimi     |

Bu desen, aynı parametrelerle yapılan tekrar sorguların önbellekten karşılanmasını sağlar.

### 6.5.3 İki Katmanlı Önbellek (Normal + LKG)

Önbellek iki katmanlı çalışır (Bölüm 4.5 ile ilişkili):

1. **Normal önbellek katmanı** (`@Cacheable``): Kısa-orta TTL'li, ad alanına özel önbellek. İlk savunma hattı — performans için.
2. **Son Geçerli Değer katmanı** (`!kg: ` önekli`): Uzun TTL'li dayanıklılık önbelleği. İkinci savunma hattı — dış kaynak çöktüğünde son geçerli veriyi sunar.

Bir veri talep edildiğinde önce normal önbelleğe, oradan da gerekirse dış kaynağa gidilir; dış kaynak başarısız olursa LKG katmanı devreye girer. Bu iki katman, hem hızı (normal cache) hem de kesinti dayanıklılığını (LKG) birlikte sağlar.

**Anahtar versiyonlama:** Bazı önbellek anahtarları, veri formatı değiştiğinde eski önbelleğin otomatik geçersiz kalması için sürüm öneki içerir (örn. kripto grafik anahtarlarında ``v4:``, ``v5:``). Format güncellendiğinde sürüm artırılır; eski anahtarlar TTL ile düşer.

### 6.5.4 Önbellek Geçersiz Kılma (Invalidation)

Önbellek dört yöntemle güncel tutulur:

- **TTL tabanlı (otomatik):** Her anahtar, ad alanının TTL süresi dolunca otomatik düşer (temel strateji).
- **Aktif geçersiz kılma** (`@CacheEvict``): Veri değiştiren işlemlerde ilgili önbellek kaydı anında temizlenir (kod tabanında 10 kullanım) — bayat veri gösterimini önler.
- **Önbellek ısıtma** (`@CachePut``): Kritik listeler periyodik olarak yeniden hesaplanıp önbelleğe yazılır (14 kullanım; bkz. 4.5.3).
- **Yönetimsel temizleme:** Veri kaynağı sınıflandırması değiştiğinde (örn. TCMB tahvil kategorileri), önbellek bir yönetici uç noktasıyla (`EvdsBondAdminController``) veya doğrudan Redis komutuyla temizlenebilir.

## 7. ARAYÜZ VE API TASARIMI

Bu bölüm, sistemin iç REST API sözleşmesini; standart yanıt formatını, uç nokta kataloğunu, kimlik doğrulama akışını ve önemli etkileşim senaryolarını açıklar.

### 7.0 API Standartları ve Yanıt Formatı

#### 7.0.1 Genel İlkeler

Sistemin REST API'si tutarlı bir sözleşmeye dayanır:

- **Sürümlenme:** Tüm uç noktalar `/api/v1/...`` öneki altında sunulur; bu, gelecekte geriye dönük uyumsuz değişiklikler için sürüm ayırımına izin verir.
- **HTTP Semantiği:** Standart HTTP metotları kullanılır — ``GET`` (okuma), ``POST`` (oluşturma), ``PUT`` (güncelleme), ``DELETE`` (silme).



- **JSON:** İstek ve yanıt gövdeleri JSON formatındadır.
- **Standart Sarmalayıcı:** Tüm yanıtlar tek tip bir ``ApiResponse`` zarfı içinde döner.

Kod tabanında, 36 denetleyici (controller) üzerinden **124 REST uç noktası** sunulur ve tüm denetleyiciler istisnasız ``/api/v1`` öneki altında konumlanır.

### 7.0.2 Standart Yanıt Sarmalayıcısı (ApiResponse)

Tüm uç noktalar, başarı ve hata durumlarında aynı sarmalayıcıyı (``ApiResponse<T>``) döndürür. Bu, ön yüzün her uç nokta için aynı işleme mantığını kullanmasını sağlar. Sarmalayıcı dört alandan oluşur:

| Alan                     | Tip         | Açıklama                                      |
|--------------------------|-------------|-----------------------------------------------|
| <code>`success`</code>   | boolean     | İşlemin başarılı olup olmadığı                |
| <code>`message`</code>   | string      | İnsan-okunabilir durum mesajı                 |
| <code>`data`</code>      | T (generic) | Asıl veri yükü (hata durumunda null olabilir) |
| <code>`timestamp`</code> | ISO-8601    | Yanıtın üretildiği zaman                      |

Sarmalayıcı, statik fabrika metotlarıyla üretilir: başarı için ``success(data)`` / ``success(data, message)``, hata için ``error(message)`` / ``error(message, data)`` ve bunlara eşdeğer ``failure(message)`` (anlamsal okunabilirlik için, özellikle dış servis hatalarında kullanılır). Her örnek, oluşturulduğunda ``timestamp`` alanını otomatik atar.

#### Başarılı yanıt örneği:

```
```json
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { / ... / },
  "timestamp": "2026-06-07T10:15:30"
}
```
```

#### Hata yanıtı örneği:

```
```json
{
  "success": false,
```

```
"message": "Portföy bulunamadı",  
"data": null,  
"timestamp": "2026-06-07T10:15:30" } ``
```

7.0.3 HTTP Durum Kodları

Sarmalayıcının `success` alanına ek olarak, anlamlı HTTP durum kodları kullanılır (ayrıntılı eşleme Bölüm 4.6'da):

- **2xx**: Başarılı (200 OK, 201 Created).
- **400/422**: İstemci hatası (doğrulama, geçersiz veri).
- **401/403**: Kimlik doğrulama / yetkilendirme hatası.
- **404**: Kaynak bulunamadı.
- **503**: Dış servis erişilemez.
- **500**: Beklenmeyen sunucu hatası.

7.0.4 API Dokümantasyonu (OpenAPI / Swagger UI)

API, springdoc-openapi ile otomatik olarak belgelenir. Çalışan sistemde, `/swagger-ui.html` adresinden erişilebilen etkileşimli bir **Swagger UI** sunulur; OpenAPI 3 spesifikasyonu `/v3/api-docs` üzerinden yayımlanır. Arayüz, JWT tabanlı kimlik doğrulama şemasını da içerir (korunmalı uç noktalar Swagger üzerinden token ile denenebilir). Bu, API'nin her zaman güncel ve keşfedilebilir bir dokümantasyona sahip olmasını sağlar.

admin-support-controller	
GET	/api/v1/admin/users/{userId}/tickets
PATCH	/api/v1/admin/support/tickets/{id}/status
admin-user-controller	
GET	/api/v1/admin/users
GET	/api/v1/admin/users/{userId}
POST	/api/v1/admin/users/{userId}/login
POST	/api/v1/admin/users/{userId}/logout
alarm-controller	
DELETE	/api/v1/alarms/{alarmId}
GET	/api/v1/alarms
GET	/api/v1/alarms/{alarmId}
PATCH	/api/v1/alarms/{alarmId}
POST	/api/v1/alarms
assistant-controller	
POST	/api/v1/assistant/chat
auth-controller	
POST	/api/v1/auth/register
bank-currency-controller	
GET	/api/v1/market/currency/banks
GET	/api/v1/market/currency/banks/{bankName}

Şekil 17: Swagger UI

7.1 REST API Sözleşmesi (Endpoint Kataloğu)

Sistemin 124 uç noktası, 36 denetleyiciye ve işlevsel alanlara göre gruplanmıştır. Aşağıdaki katalog, denetleyici bazında base path, uç nokta sayısı ve erişim seviyesini özetler. (Erişim seviyesi: **Genel** = misafir dahil herkes, **Kimlikli** = giriş gerekli, **Admin** = yönetici gerekli.)

7.1.1 Piyasa ve Haber Uç Noktaları (Genel Erişim)

Base Path	Uç Nokta	Denetleyici	İçerik
`/api/v1/market/crypto`	11	CryptoController	Kripto liste/detay/grafik/fear-greed
`/api/v1/market/funds`	6	MarketFundController	Yatırım fonları
`/api/v1/market/futures`	6	MarketFuturesController	ViOP sözleşmeleri
`/api/v1/market/stocks`	5	MarketStockController	Hisse senetleri
`/api/v1/market/economy`	5	EconomyController	Makroekonomik veri
`/api/v1/news`	5	NewsController	Haber listesi, detay, kişiselleştirilmiş ("size özel")
`/api/v1/market/bonds/evds`	4	EvdsBondController	DİBS tahviller
`/api/v1/market/fx`, `/indices`, `/bonds/global`	3'er	MarketFx/Index/Eurobond	Döviz, endeks, Eurobond
`/api/v1/commodities`, `/gold`, `/silver`, `/precious-metals`, `/currency`	2–3	Emtia/Metal/Kur Controller'ları	Emtia, kıymetli metaller, banka kurları
`/api/v1/market/ipo`, `/indicators`, `/economy/calendar`, `/viop`	1–2	Ipo/Indicator/Calendar/Viop	Halka arz, gösterge, takvim, ViOP

7.1.2 Kullanıcı Verisi Uç Noktaları (Kimlik Gerekli)

Base Path	Uç Nokta	Denetleyici	İçerik
`/api/v1/portfolios`	17	PortfolioController	Portföy, işlem, izleme, analiz, what-if, AI analiz
`/api/v1/alarms`	5	AlarmController	Alarm CRUD ve yönetimi
`/api/v1/notifications`	5	NotificationController	Bildirim listeleme/okuma
`/api/v1/me`	4	MeController	Kullanıcı bilgisi
`/api/v1/support`	4	SupportTicketController	Kullanıcı destek talepleri
`/api/v1/newsletter`	3	NewsletterController	Bülten aboneliği

Base Path	Uç Nokta	Denetleyici	İçerik
`/api/v1/me/preferences`	—	UserPreferenceController	Kullanıcı arayüz tercihleri senkronu

7.1.3 Yönetici Uç Noktaları (Admin Gerekli)

Base Path	Uç Nokta	Denetleyici	İçerik
`/api/v1/admin/users`	4	AdminUserController	Kullanıcı yönetimi, ban
`/api/v1/admin/support`	—	AdminSupportController	Destek talebi durum yönetimi
`/api/v1/admin/eurobonds`	2	EurobondAdminController	Eurobond ISIN yönetimi
`/api/v1/admin/bonds/evds`	1	EvdsBondAdminController	Tahvil önbellek temizleme

7.1.4 Diğer Uç Noktalar

Base Path	Erişim	İçerik
`/api/v1/assistant`	Genel (sınırlı)	Yapay zekâ sohbet asistanı
`/api/v1/auth`	Genel	Kimlik akışı (token)
`/api/v1/proxy`	Genel	Görsel proxy (CORS/karışık içerik için)
`/api/v1` (health)	Genel	Sağlık kontrolü

Örnek uç nokta sözleşmesi (Portföy oluşturma):

Alan	Değer
Metot / Yol	`POST /api/v1/portfolios`
Erişim	Kimlik gerekli (JWT)
İstek gövdesi	{ "name": "...", "portfolioType": "HOLDINGS" }
Başarı yanıtı	`200` + `ApiResponse<PortfolioDto>`
Hata	`400` (doğrulama), `401` (kimlik yok)

7.2 Kimlik Doğrulama Akışı (OAuth2/OIDC)

Sistem, kimlik doğrulamayı Keycloak'a devreder ve OAuth2/OpenID Connect (OIDC) standartlarını kullanır. Backend, **durumsuz (stateless) bir OAuth2 Resource Server** olarak yapılandırılmıştır; oturum durumu sunucuda tutulmaz (CSRF korumasına gerek kalmaz), her istek kendi JWT'sini taşır.

7.2.1 Giriş Akışı (Authorization Code + PKCE)

Ön yüz, en güvenli tarayıcı tabanlı akış olan **Authorization Code + PKCE (S256)** akışını kullanır:

1. Kullanıcı giriş başlatır; ön yüz bir PKCE doğrulayıcısı (`code_verifier`) üretir, bundan **SHA-256 ile bir meydan okuma** (`code_challenge`, **S256 yöntemi**) hesaplar ve durum (`state`) değeriyle birlikte oturum deposunda (`sessionStorage`) saklar, ardından kullanıcıyı Keycloak giriş sayfasına yönlendirir.

2. Kullanıcı Keycloak'ta kimlik bilgilerini (gerekirse 2FA/TOTP) girer.

3. Keycloak, kullanıcıyı bir yetkilendirme kodu (`code`) ile ön yüzün `/auth/callback` adresine geri yönlendirir.

4. Ön yüz, bu kodu ve PKCE doğrulayıcısını Keycloak token uç noktasına göndererek **erişim belirteci (access token)**, **kimlik belirteci (id token)** ve **yenileme belirteci (refresh token)** alır.

5. Belirteçler `localStorage`'da saklanır ve sonraki API isteklerinde `Authorization: Bearer` başlığıyla kullanılır.

Geçici PKCE doğrulayıcısı ve durum değeri `sessionStorage`'da; uzun ömürlü belirteçler ise `localStorage`'da tutulur.

7.2.2 Belirteç Doğrulama (Backend — Yerel JWKS)

Backend, gelen her istekte JWT'yi **yerel olarak** doğrular; her istekte Keycloak'a gitmez:

1. Backend, Keycloak'ın açık anahtarlarını (JWKS) `jwt-set-uri` üzerinden alır ve ön belleğe alır.

2. Gelen JWT'nin imzası, bu ön belleğe alınmış anahtarlarla doğrulanır; süre ve veren (issuer) kontrol edilir.

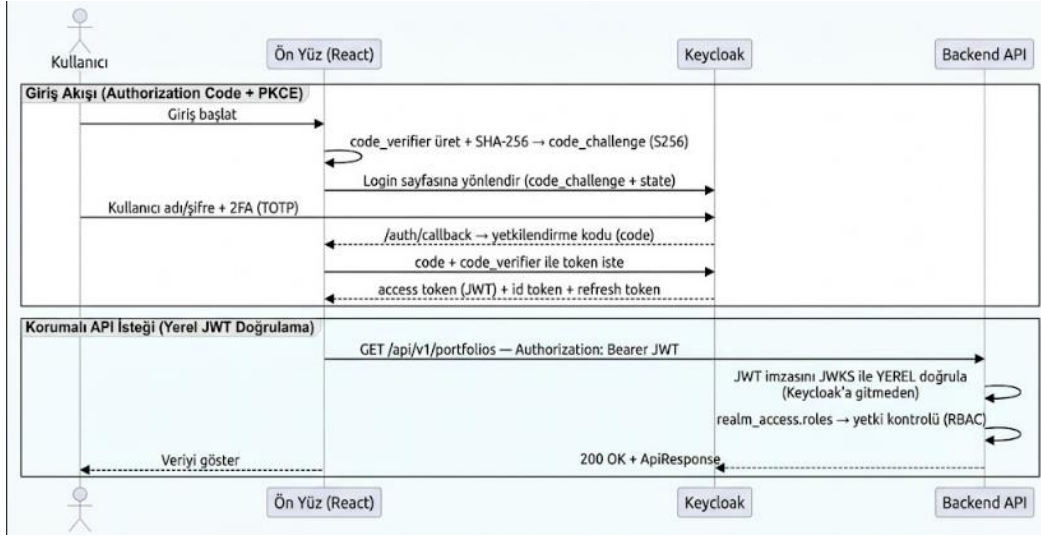
3. JWT geçerliyse, içindeki roller (`realm_access.roles` claim'i) `keycloakRealmRoleConverter` ile Spring Security yetkilerine (authority) dönüştürülür.

4. Uç nokta düzeyinde rol/yetki kontrolü uygulanır (bkz. Bölüm 8).

Bu "yerel doğrulama" yaklaşımı; her istekte Keycloak'a bağımlılığı ortadan kaldırarak performansı ve dayanıklılığı artırır. Bearer belirteç çözümleyici yalnızca `Authorization` başlığı mevcut olduğunda JWT doğrulaması dener; böylece genel (public) uç noktalar, belirteç olmadan da sorunsuz çalışır.

7.2.3 Belirteç Yenileme

Erişim belirtecinin süresi dolmadan önce, ön yüz interceptor'ı yenileme belirtecini kullanarak belirteci arka planda sessizce yeniler (bkz. Bölüm 5.3.3). Bu, kullanıcının oturumunun kesintisiz devam etmesini sağlar.



Şekil 18: Kimlik Doğrulama Sıra Diyagramı

7.2.4 Keycloak Giriş Ekranı

Giriş işlemi, ön yüz uygulamasının kendi formunda değil, doğrudan Keycloak'ın güvenli giriş sayfasında gerçekleşir (OIDC yönlendirmesiyle). Bu sayfa; kullanıcı adı/parola girişini, iki adımlı doğrulama (TOTP) adımı ve "parolamı unuttum" gibi hesap akışlarını yönetir. Kimlik bilgileri hiçbir zaman uygulama sunucusundan geçmez.

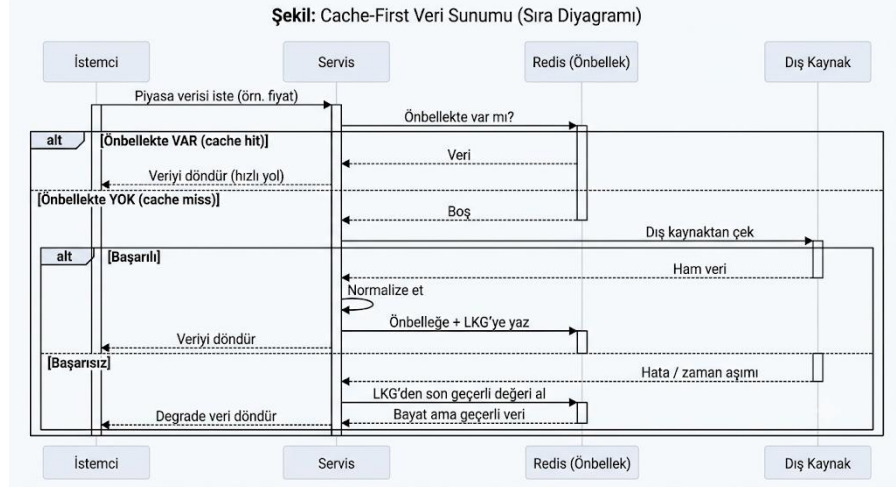
7.3 Önemli Etkileşim Senaryoları

Bu bölüm, sistemin en önemli ve çok bileşenli iş akışlarını sıralı etkileşim (sequence) düzeyinde açıklar. Bu senaryolar, ayrı bileşenlerin (controller, servis, port, dış sistem) birlikte nasıl çalıştığını gösterir.

7.3.1 Cache-First Veri Sunumu

En sık akış, bir piyasa verisinin sunulmasıdır:

1. İstemci bir piyasa verisi ister (örn. hisse fiyatı).
2. Servis, önce Redis önbelleğine bakar (cache-first).
3. **Önbellekte varsa:** veri doğrudan döndürülür (hızlı yol).
4. **Önbellekte yoksa:** dış kaynaktan çekilir, normalize edilir, önbelleğe ve LKG'ye yazılır, döndürülür.
5. **Dış kaynak başarısızsa:** LKG'deki son geçerli değer döndürülür (degrade).



Şekil 19: Cache-First Veri Sunumu

7.3.2 Portföy Değerleme

Bir portföy görüntülendiğinde, güncel değer ve kâr/zarar hesaplanır:

1. İstemci portföy detayını ister (JWT ile).
2. Servis, portföy işlemlerini veritabanından okur ve net pozisyonları (holdings) hesaplar.
3. Her varlık için güncel fiyat, `HoldingMarketEnrichmentPort` üzerinden market modülünden alınır (paralel çekim).
4. Güncel değer, maliyet ve kâr/zarar hesaplanır; sonuç istemciye döner.

Güncel değer/kâr-zarar gibi türetilmiş veriler veritabanında saklanmaz; her istekte güncel fiyatla hesaplanır (bkz. 6.1).

7.3.3 Alarm → Bildirim → E-posta

Zamanlanmış alarm değerlendirme akışı:

1. `AlarmEvaluationService`, her 60 saniyede aktif alarmları değerlendirir (`evaluateActiveAlarms`).
2. Her alarm için güncel metrik (fiyat/değişim/hacim) market modülünden alınır.
3. Alarm koşulu sağlanırsa: alarm durumu güncellenir ve `AlarmNotificationPort` üzerinden bildirim üretilir.

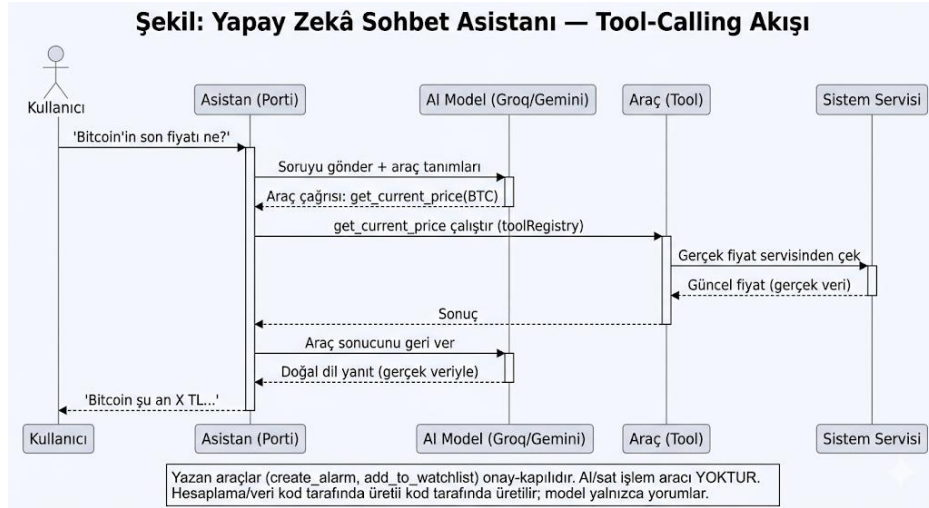
4. Bildirim hem uygulama içi (notification kaydı) hem de e-posta (SMTP) olarak gönderilir; e-posta gönderim durumu kayıta tutulur

7.3.4 Yapay Zekâ Sohbet Asistanı (Tool-Calling)

Sohbet asistanı, kullanıcı sorusuna göre gerçek veriye erişmek için **araç çağırma (tool-calling)** kullanır:

1. Kullanıcı doğal dilde soru sorar (örn. "Bitcoin'in son fiyatı ne?").
2. Yapay zekâ modeli, soruyu analiz eder ve uygun aracı seçer.
3. Asistan, ilgili aracı çalıştırır (araç kayıt defteri `toolRegistry` üzerinden). Mevcut 9 araç: `get\_current\_price`, `get\_price\_history`, `get\_price\_at\_date`, `get\_news`, `get\_portfolio\_summary`, `get\_economy\_indicator`, `simulate\_portfolio\_scenario`, `create\_alarm`, `add\_to\_watchlist`.
4. Araç, sistemin gerçek servislerinden veri çeker (örn. fiyat servisi) ve sonucu modele geri verir.
5. Model, gerçek veriyle nihai yanıtı üretir ve kullanıcıya sunar.

Tasarım notu: Yazan (write) araçlar (`create\_alarm`, `add\_to\_watchlist`) onay-kapılıdır; al/sat gibi finansal işlem aracı yoktur (sistem portföy takibi yapar, işlem yapmaz). Sağlayıcı zinciri (Groq/Gemini), kota dolduğunda otomatik bir sonrakine geçer.



Şekil 20: Yapay Zekâ Sohbet Asistanı

7.3.5 Senaryo Simülasyonu (What-If)

Sohbet asistanının en gelişmiş aracı, portföy üzerinde "ne olurdu?" simülasyonu yapar:

1. Kullanıcı bir senaryo sorar (örn. "Dolar %10 yükselse portföyüm ne olur?").
2. Yapay zekâ, senaryoyu tasarlar (hangi varlığa ne kadar çok uygulanacağını belirler).
3. **Hesaplama kod tarafında yapılır** (`ScenarioSimulationTool`/`WhatIfScenario`) — yapay zekâ sayı üretmez, yalnızca senaryoyu tanımlar.
4. Hesaplanan sonuç yapay zekâya geri verilir; yapay zekâ bunu yorumlar ve kullanıcıya açıklar.

Bu ayırım (AI tasarlar → kod hesaplar → AI yorumlar), finansal hesaplamaların doğruluğunu garanti ederken yapay zekânın doğal dil gücünden yararlanır.

7.3.6 Kullanıcı Kayıt Akışı

Yeni kullanıcı kaydı, Keycloak ile koordineli yürütülür:

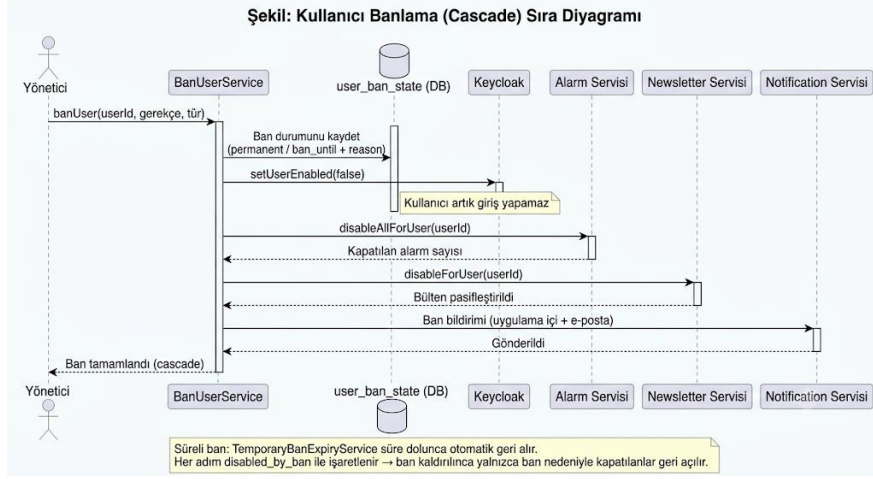
1. Kullanıcı kayıt formunu doldurur; ön yüz `register` isteği gönderir.
2. Backend, `UserRegistrationPort` üzerinden Keycloak Admin API'sini çağırarak kullanıcıyı oluşturur.
3. Kullanıcıya varsayılan `USER` rolü atanır.
4. Keycloak, e-posta doğrulama bağlantısı gönderir; kullanıcı doğrulayana kadar bazı işlemler kısıtlıdır.
5. Doğrulama tamamlandığında kullanıcı tam erişim kazanır.

7.3.7 Kullanıcı Banlama (Cascade)

Yönetici bir kullanıcıyı banladığında, birden çok modülü etkileyen bir basamaklı (cascade) akış tetiklenir (`BanUserService`):

1. Yönetici, gerekçeyle birlikte ban işlemi başlatır (kalıcı veya süreli).
2. **Keycloak'ta** kullanıcı devre dışı bırakılır (`setUserEnabled(false)`) — artık giriş yapamaz.
3. **Alarm modülü:** Kullanıcının aktif alarmları devre dışı bırakılır (`disabled\_by\_ban`).
4. **Bülten modülü:** Bülten aboneliği devre dışı bırakılır.
5. **Bildirim modülü:** Kullanıcıya ban bildirimi (uygulama içi + e-posta) gönderilir.

Bu akış, tek bir yönetici eyleminin ilgili tüm sistemlerde tutarlı biçimde uygulanmasını sağlar. Süreli banlarda, `TemporaryBanExpiryService` süre dolduğunda durumu otomatik geri alır.



7.3.8 Bülten Dijesti (Newsletter Digest)

Periyodik özet e-postası akışı:

1. `NewsletterDigestService` her gün belirlenen saatte çalışır.
2. O gün gönderilmesi gereken abonelikler (DAILY/WEEKLY/MONTHLY sıklığına göre) belirlenir.
3. Her abone için kişisel bir "kontrol paneli özeti" (portföy + piyasa verileri) derlenir (`NewsletterDigestPort`).
4. Özet, "dashboard görünümü" bir HTML e-posta olarak gönderilir (`EmailSenderPort`).
5. Gönderim zamanı kaydedilir (aynı dönemde tekrar gönderilmez).

7.3.9 ViOP Vade Kapatma

Gece çalışan zamanlanmış görev, vadesi dolan ViOP pozisyonlarını kapatır:

1. `ViopMaturityScheduler` her gece çalışır (cron, 00:35).
2. Vadesi dolan açık FUTURE pozisyonları bulunur.
3. Her pozisyon için settlement fiyatı belirlenir ve pozisyon kendi yönünü taşıyan bir kapanış işlemiyle kapatılır (realized P&L hesaplanır).
4. İşlem idempotenttir (tekrar çalışsa bile çift kapatma yapmaz) ve dağıtık kilitle (ShedLock) korunur.

7.4 Dış Entegrasyon Sözleşmeleri ve Degrade Davranışı

Sistem 20'den fazla dış servisle bütünleştiğinden, bu entegrasyonlar standart bir sözleşme ve koruma katmanıyla yönetilir.

7.4.1 Entegrasyon Koruma Katmanı

Her dış çağrı, aşağıdaki korumalarla sarmalanır:

Koruma	Açıklama
Zaman aşımı (timeout)	Bağlantı/okuma/yazma için sınırlar (örn. 5 sn); yavaş kaynak tüm isteği bloke edemez
Anti-ban gate (throttle)	Hassas kaynaklara (örn. EVDS, scrape kaynakları) art arda istekler arasında minimum bekleme uygulanır; kaynağın istek limitini aşmamak için
Yeniden deneme / Devre kesici	Geçici hatalarda Resilience4j retry; kalıcı arızada circuit breaker (bkz. 4.5)
Degrade / LKG	Çağrı başarısızsa son geçerli değer (LKG) veya boş/varsayılan sonuç döndürülür

7.4.2 Degrade (Aşamalı Bozulma) Davranışı

Sistem, bir dış kaynak erişilemez olduğunda **tamamen çökmek yerine aşamalı olarak bozulur (graceful degradation)**:

- **LKG mevcutsa:** Son geçerli veri servis edilir (kullanıcı çoğu zaman farkı bile anlamaz).
- **LKG yoksa:** Boş liste veya anlamlı bir varsayılan döndürülür; istemci "veri şu an alınamıyor" durumunu yönetir.
- **Kritik olmayan özellik:** İlgili özellik geçici devre dışı kalır (örn. yapay zekâ anahtarı yoksa asistan "kullanılamıyor" der), ana akış etkilenmez.

7.4.3 Entegrasyon Loglama ve Gözlemlenebilirlik

Tüm dış çağrılar, merkezi bir entegrasyon loglama altyapısıyla (`CentralIntegrationLogService`) izlenir:

- Her çağrı için kaynak, süre (latency), sonuç (başarı/başarısızlık) ve hata türü yapılandırılmış log olarak kaydedilir.
- Hassas veriler (API anahtarları, kişisel bilgiler) loglanmadan önce temizlenir (`IntegrationLogMetadataSanitizer`).
- Bu loglar, dış kaynak sağlığını izleyen Grafana panolarını besler (bkz. Bölüm 10).

7.4.4 Rate Limiting (Hız Sınırlama)

Sistemin kendi kaynaklarını korumak için, özellikle anonim (misafir) kullanıcıların kötüye kullanımı önlenir: örneğin yapay zekâ asistanı için anonim kullanıcılara IP bazlı bir kullanım limiti uygulanır (yük dengeleyici atlanarak gerçek istemci IP'si `X-Forwarded-For` üzerinden belirlenir).

8. GÜVENLİK TASARIMI

Bu bölüm, sistemin güvenlik mimarisini; kimlik ve erişim yönetimini, koruma önlemlerini, ban mekanizmasını ve tehdit modelini OWASP ASVS çerçevesi referans alınarak açıklar.

8.1 Kimlik ve Yetki Modeli

8.1.1 Kimlik Yönetimi (Keycloak + LDAP)

Sistem, kimlik doğrulamayı kendi içinde yapmaz; uzman bir kimlik sağlayıcısı olan **Keycloak**'a devreder. Keycloak:

- Kullanıcı kimlik bilgilerini, oturum ve token yaşam döngüsünü yönetir.
- Bir **LDAP dizini (ApacheDS)** ile federasyon kurar; kurumsal kullanıcılar LDAP'tan doğrulanabilir (realm'deki `federationLink` ile bağlanır).
- İki adımlı doğrulama (TOTP tabanlı 2FA), şifre politikaları ve e-posta doğrulamasını sağlar.

Bu yaklaşım (externalized identity), kimlik güvenliğini olgun bir sisteme bırakarak uygulama kodunu bu sorumluluktan arındırır.

8.1.2 Yetkilendirme Modeli (RBAC)

Sistem, **rol tabanlı erişim kontrolü (RBAC)** kullanır. İki uygulama rolü vardır:

Rol	Yetki
`USER`	Kişisel kaynaklara (portföy, alarm, bildirim, bülten, destek, tercih) erişim
`ADMIN`	Yukarıdakilere ek olarak yönetici uç noktalarına (`/api/v1/admin/**`) erişim

Roller Keycloak'ta tanımlıdır ve JWT içindeki `realm\_access.roles` claim'i üzerinden taşınır.

8.1.3 Merkezi Yetkilendirme (SecurityConfig)

Tüm erişim kuralları, **tek bir merkezi yapılandırmada** (`SecurityConfig`) tanımlanır; dağınık metot-seviyesi anotasyonlar (`@PreAuthorize`) kullanılmaz. Bu, güvenlik kararlarının tek yerden denetlenebilmesini sağlar. Erişim matrisi:

Uç Nokta Grubu	Erişim
`/api/v1/market/`, `/news/`, `/gold/`, `/assistant/`, `/auth/`, `/proxy/`	Genel (permitAll)
`/api/v1/newsletter/unsubscribe`	Genel (public iptal bağlantısı)
`/api/v1/portfolios/`, `/alarms/`, `/notifications/`, `/me/`, `/support/`, `/newsletter/`	Kimlik gerekli (authenticated)
`/api/v1/admin/**`	Yalnızca `ADMIN` rolü
`/swagger-ui/**`, `/actuator/health`, `/actuator/prometheus`	Genel (dokümantasyon/sağlık)

8.1.4 Kaynak Sahipliği Kontrolü

Rol kontrolüne ek olarak, kullanıcıya ait kaynaklarda **sahiplik doğrulaması** yapılır: bir kullanıcı yalnızca kendi `user\_id`'sine ait portföy, alarm vb. kaynaklara erişebilir. Bu, JWT'den alınan kullanıcı kimliği ile kaynağın sahibi karşılaştırılarak servis katmanında uygulanır; böylece bir kullanıcı başka birinin verisine (yetkili token'la bile) erişemez.

8.2 Ban Mekanizması Tasarımı

Sistemin en kapsamlı güvenlik/yönetim akışlarından biri, kullanıcı banlama mekanizmasıdır. Bu mekanizma, tek bir yönetici eylemini birden çok modülde tutarlı biçimde uygular (cascade).

8.2.1 Ban Türleri ve Durum Saklama

Ban durumu `user\_ban\_state` tablosunda tutulur (`keycloak\_user\_id` birincil anahtar):

Alan	Açıklama
`permanent`	Kalıcı ban mı (true) yoksa süreli mi (false)
`ban_until`	Süreli ban için bitiş zamanı (`TIMESTAMPZ`)
`ban_reason`	Ban gerekçesi (yöneticinin girdiği)

İki ban türü vardır:

- **Kalıcı (PERMANENT):** Süresiz; yalnızca yönetici geri alabilir.
- **Süreli (TEMPORARY):** `BanDurationCalculator` ile bitiş zamanı hesaplanır; süre dolunca otomatik kaldırılır.

8.2.2 Banlama Akışı (Cascade)

`BanUserService.banUser()` çağrıldığında sıralı bir basamaklı işlem yürütülür:

1. **Doğrulama ve kayıt:** Gerekçe alınır, ban türü belirlenir, `user\_ban\_state` tablosuna yazılır.
2. **Kimlik kapatma:** Keycloak'ta kullanıcı devre dışı bırakılır (`setUserEnabled(false)`) — kullanıcı artık giriş yapamaz, mevcut oturumları geçersizleşir.
3. **Alarm devre dışı:** Kullanıcının tüm aktif alarmları kapatılır (`disableAllForUser`); kaç alarmın etkilendiği kayıt altına alınır.
4. **Bülten devre dışı:** Bülten aboneliği pasifleştirilir.
5. **Bilgilendirme:** Kullanıcıya ban bildirimi (uygulama içi `NotificationType.BAN` + e-posta) gönderilir.

Bu adımlar; alarm, bülten ve bildirim modüllerini koordine eden bir orkestrasyondur. Her adım `disabled\_by\_ban` bayrağıyla işaretlenir; böylece ban kaldırıldığında **yalnızca ban nedeniyle kapatılanlar** geri açılır (kullanıcının kendi kapattıkları korunur).

8.2.3 Ban Kaldırma (Cascade Geri Alma)

Ban kaldırma iki yolla olur:

- **Manuel (`UnbanUserService`):** Yönetici banı kaldırır; Keycloak'ta kullanıcı yeniden etkinleştirilir ve `disabled\_by\_ban` ile kapatılan alarm/bülten kayıtları geri açılır.
- **Otomatik (`TemporaryBanExpiryService`):** Süreli banlarda, `ban\_until` zamanı geçtiğinde zamanlanmış görev banı otomatik kaldırır (bu nedenle `user\_ban\_state` tablosunda süresi dolan banları hızlı bulmak için kısmi (partial) bir indeks tanımlıdır).

8.3 Güvenlik Önlemleri

Kimlik/yetki modeline ek olarak, sistem çeşitli katmanlarda koruyucu önlemler uygular.

8.3.1 Aktarım Güvenliği (HTTPS/TLS)

Üretim ortamında tüm istemci-sunucu iletişimi **HTTPS/TLS** üzerinden şifrelenir. Kubernetes (GKE) ortamında, ingress üzerinde **GKE ManagedCertificate** (otomatik Let's Encrypt sertifikası) kullanılır; sertifika otomatik sağlanır ve yenilenir. Böylece veri aktarımı dinlemeye karşı korunur.

8.3.2 Köken Politikası (CORS)

Tarayıcı tabanlı erişimde, yalnızca izin verilen kökenlerin (origin) API'ye erişebilmesi için **CORS** yapılandırılır. İzinli kökenler (`allowed-origins`) ortam değişkeniyle (`CORS\_ALLOWED\_ORIGINS`) tanımlanır; geliştirmede yerel adresler, üretimde yalnızca gerçek ön yüz adresi izinlidir. Bu, yetkisiz bir web sitesinin kullanıcı adına istek yapmasını engeller.

8.3.3 Girdi Doğrulama ve Temizleme (Validation & Sanitization)

- **Doğrulama (Validation):** Gelen istekler, Bean Validation anotasyonlarıyla (kod tabanında 80+ kullanım: `@Valid`, `@NotNull`, `@Email`, `@Size` vb.) doğrulanır. Geçersiz girdi, iş mantığına ulaşmadan reddedilir.
- **Temizleme (Sanitization):** Loglara veya dış sistemlere yazılan kullanıcı girdileri temizlenir (`BusinessLogMetadataSanitizer`, `IntegrationLogMetadataSanitizer`); bu, **log enjeksiyonu** ve hassas veri sızıntısını önler. Yapay zekâ ve kayıt akışlarında da girdi temizleme uygulanır.

8.3.4 Hız Sınırlama (Rate Limiting)

Kötüye kullanımı önlemek için, özellikle giriş yapmamış (anonim) kullanıcıların yapay zekâ asistanına erişimi sınırlanır (`assistant.anon-message-limit`). Anonim kullanıcının kimliği IP adresinden belirlenir; gerçek istemci IP'si, yük dengeleyicinin eklediği başlıklar atlanarak `X-Forwarded-For` zincirinden güvenli biçimde çıkarılır (istemcinin sahte başlık göndererek limiti aşması engellenir).

8.3.5 Sır (Secret) Yönetimi

Hassas değerler asla kaynak koda yazılmaz (bkz. 4.8): geliştirmede `.env.local` (gitignore), üretimde Kubernetes Secret. Loglarda da sırlar maskelenir. Üretim profili, kritik sır eksikse başlatmayı durdurur (fail-loud).

8.3.6 Durumsuz Oturum ve CSRF

Backend durumsuz (stateless) olduğundan ve oturum çerezi yerine `Authorization` başlığıyla JWT kullandığından, **CSRF** saldırı yüzeyi ortadan kalkar; bu nedenle CSRF koruması bilinçli olarak devre dışıdır (token tabanlı kimlik doğrulamada gereksizdir).

8.4 Tehdit Modeli (STRIDE) ve ASVS Uyumu

Bu bölüm, sistemin güvenlik tasarımını iki standart çerçeveye değerlendirir: tehditleri sınıflandırmak için **STRIDE**, doğrulama seviyesi için **OWASP ASVS**.

8.4.1 STRIDE Tehdit Analizi

STRIDE, altı tehdit kategorisinde sistemin önlemlerini değerlendirir:

Tehdit (STRIDE)	Risk	Sistemdeki Önlem
Spoofing (Kimlik Taklidi)	Başkası gibi davranma	Keycloak/OIDC kimlik doğrulama, JWT imza doğrulama, 2FA (TOTP)
Tampering (Kurcalama)	Veri/istek değiştirme	JWT imza bütünlüğü, HTTPS/TLS, veritabanı kısıtları (CHECK/FK), girdi doğrulama
Repudiation (İnkâr)	Eylemi inkâr etme	Merkezi AUDIT loglama (kim, ne, ne zaman); yapılandırılmış loglar OpenSearch'te
Information Disclosure (Bilgi İfşası)	Hassas veri sızması	HTTPS, sahiplik kontrolü, hata mesajlarında detay gizleme, log sanitizasyonu, secret yönetimi
Denial of Service (Hizmet Reddi)	Aşırı yük/kötüye kullanım	Rate limiting (anonim), dış çağrı timeout/circuit breaker, HPA ile ölçekleme
Elevation of Privilege (Yetki Yükseltme)	Yetkisiz erişim	RBAC (USER/ADMIN), merkezi yetkilendirme, kaynak sahipliği kontrolü, admin uç noktalarının ayrılması

8.4.2 OWASP ASVS Referansı

Sistem, OWASP Application Security Verification Standard (ASVS) ilkeleriyle değerlendirildiğinde, bir web uygulaması için makul ve uygulanabilir bir güvenlik seviyesi (genel olarak **Seviye 1–2**) hedefler. Başlıca ASVS alanlarındaki karşılıklar:

ASVS Alanı	Sistemdeki Karşılık
V1 — Mimari	Kimlik dışsallaştırma (Keycloak), katmanlı mimari, merkezi güvenlik yapılandırması

ASVS Alanı	Sistemdeki Karşılık
V2 — Kimlik Doğrulama	OIDC/OAuth2, 2FA (TOTP), güçlü şifre politikası (Keycloak), token yenileme
V3 — Oturum Yönetimi	Durumsuz JWT, kısa ömürlü erişim belirteci + yenileme belirteci rotasyonu
V4 — Erişim Kontrolü	RBAC, kaynak sahipliği doğrulaması, varsayılan-kapalı yetkilendirme
V5 — Doğrulama/Kodlama	Bean Validation, girdi temizleme, log enjeksiyonu önleme
V6 — Saklanan Kriptografi	Sırların kod dışı yönetimi, kimlik bilgilerinin Keycloak'ta saklanması
V7 — Hata/Loglama	Merkezi yapılandırılmış loglama, hata mesajlarında bilgi sızdırmama
V9 — İletişim	HTTPS/TLS (ManagedCertificate), güvenli token aktarımı

8.4.3 Bilinçli Kapsam Sınırları

Sistem, hedeflenen kapsam ve gerçekçilik dahilinde tasarlanmıştır. Bazı ileri güvenlik önlemleri (örn. tam ASVS Seviye 3, donanımsal HSM, gelişmiş anomali tespiti) mevcut kapsamın dışındadır ve gerektiğinde ileride eklenebilir. Bu, güvenlik tasarımının **risk-orantılı** (risk-proportionate) bir yaklaşımla yapıldığını gösterir.

9. DAĞITIM TASARIMI

Bu bölüm, sistemin nasıl paketlenildiğini ve çalıştırıldığını; konteynerizasyon, Kubernetes topolojisi, ağ/TLS yapılandırması, ölçekleme ve sürekli entegrasyon/dağıtım (CI/CD) hattını açıklar.

9.1 Konteynerizasyon (Docker)

Tüm bileşenler Docker konteyneri olarak paketlenir. Uygulama imajları, derleme ve çalıştırma ortamlarını ayıran **çok aşamalı (multi-stage) yapı** ile oluşturulur; bu, küçük ve güvenli üretim imajları sağlar.

9.1.1 Backend İmajı

Aşama	Temel İmaj	İşlev
1. Derleme	`maven:3.9-eclipse-temurin-21`	Bağımlılıkları indirir (önbelleğe alınır), kaynağı derler ve jar üretir
2. Çalıştırma	`eclipse-temurin:21-jre`	Yalnızca jar'ı ve JRE'yi içerir (derleme araçları içermez)

Çalıştırma aşaması, uygulamayı **OpenTelemetry Java agent** ile başlatır (`-javaagent`); bu, kod değişikliği gerektirmeden otomatik enstrümantasyon (iz/metrik) sağlar. Agent, derleme zamanında resmi sürümden (`v2.4.0`) imaja indirilir (`ADD`). Konteyner 8080 portunu açar.

***Tasarım notu:** Bağımlılık indirme (`dependency:go-offline`) ayrı bir katmanda yapılır; böylece yalnızca kaynak kod değiştiğinde bağımlılıklar yeniden indirilmez (Docker katman önbelleği). Derleme sırasında testler atlanır (`-DskipTests`); testler CI sürecinde ayrıca çalıştırılır.*

9.1.2 Frontend İmajı

Aşama	Temel İmaj	İşlev
1. Derleme	`node:20-alpine`	`npm ci` ile bağımlılıkları kurar, Vite ile statik üretim derlemesi yapar
2. Sunum	`nginx:1.27-alpine`	Statik dosyaları Nginx ile servis eder

Build-arg taşınabilirliği: Keycloak adresi gibi ortam-bağımlı değerler (`VITE\_KEYCLOAK\_URL`, `VITE\_KEYCLOAK\_REALM`), derleme zamanında **build-arg** olarak imaja gömülür. Bulut dağıtımında gerçek HTTPS adresi geçilir; verilmezse koddaki yerel varsayılanlar devreye girer (yerel derleme bozulmaz).

Nginx yapılandırması iki kritik iş yapar:

1. `/api/` isteklerini backend'e yönlendirir (`proxy\_pass http://backend:8080`).
2. Diğer tüm istekleri `index.html`'e yönlendirerek React Router'ın istemci-tarafı yönlendirmesini destekler (SPA fallback: `try\_files \$uri \$uri /index.html`).

Ayrıca SPA kabuğu (`index.html`) **önbelleğe alınmaz**; böylece yeni dağıtımda tarayıcı eski bundle'ı tutmaz (hash'li asset adları değiştiğinden bu önemlidir).

9.1.3 Diğer İmajlar

- **Keycloak:** Özelleştirilmiş bir imaj (realm yapılandırması ile).
- **Log Consumer:** Kafka'dan log tüketip OpenSearch'e yazan, kendi çok aşamalı imajıyla (maven-alpine → jre-alpine) paketlenmiş özel Java servisi.

9.2 Kubernetes Topolojisi (GKE)

Üretim ortamında sistem, Google Kubernetes Engine (GKE) üzerinde çalışır. Tüm kaynaklar `portiva` ad alanında (namespace), **katmanlı bir manifest yapısıyla** organize edilir; bu yapı, dağıtım sırasını ve bağımlılık yönünü de yansıtır.

9.2.1 Katmanlı Manifest Organizasyonu

Katman	İçerik	Bileşenler
`00-base`	Temel	Namespace (`portiva`)

Katman	İçerik	Bileşenler
`01-data`	Veri katmanı	PostgreSQL, Redis, Kafka, ApacheDS (LDAP)
`02-app`	Uygulama katmanı	Backend, Frontend, Keycloak, Ingress
`03-monitoring`	Gözlemlenebilirlik	OpenSearch, OTel Collector, Tempo, Prometheus, Grafana, Log Consumer

Bu katmanlama, dağıtımın doğru sırada yapılmasını sağlar: önce temel ve veri katmanı, sonra uygulama, en son izleme.

9.2.2 İş Yüğü Türleri (Deployment vs StatefulSet)

Kaynaklar, durum (state) ihtiyacına göre iki türde tanımlanır:

- **StatefulSet (6 adet):** Kalıcı disk (PVC, `standard-rwo` depolama sınıfı) ve sabit kimlik gerektiren durumlu (stateful) bileşenler — **PostgreSQL, Kafka, OpenSearch, Prometheus, Tempo, ApacheDS.** `volumeClaimTemplates` ile kalıcı disk talep ederler.
- **Deployment (7 adet):** Durumsuz veya durumu kritik olmayan bileşenler — **Backend, Frontend, Keycloak, Grafana, OTel Collector, Log Consumer ve Redis.** (Redis bir önbellek olduğundan kalıcılık kritik değildir; bu nedenle Deployment olarak çalışır.)

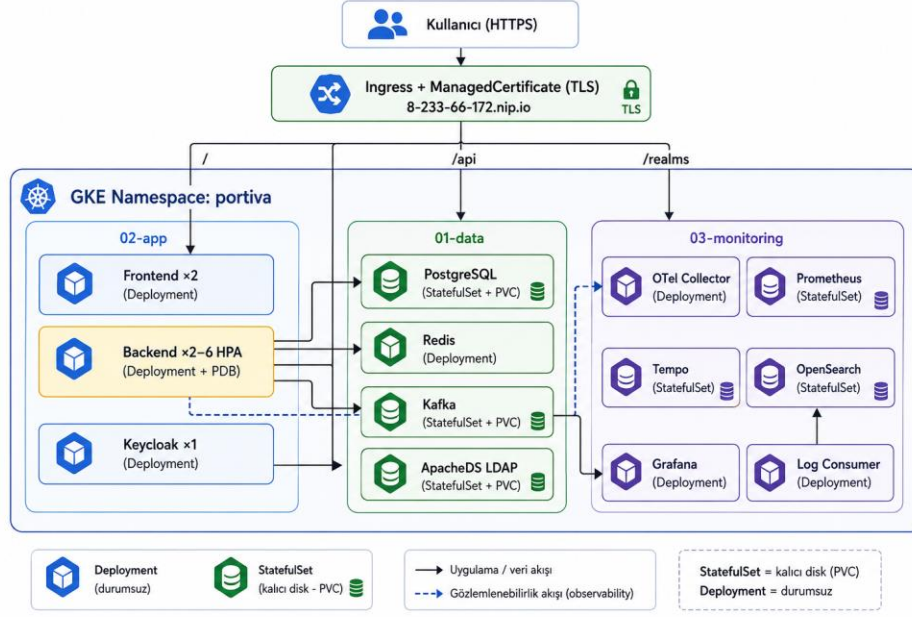
9.2.3 Servis Keşfi ve İç İletişim

Konteynerler birbirine Kubernetes **Service** nesneleri üzerinden, sabit DNS adlarıyla erişir (örn. backend, postgres, redis). Bu, bir konteynerin IP'si değişse bile diğerlerinin onu adıyla bulmasını sağlar.

Tüm dahili Service'ler `ClusterIP` türündedir (yalnızca küme içinden erişilebilir); sisteme dış erişim **yalnızca Ingress** üzerinden sağlanır. Hiçbir veri veya uygulama servisi doğrudan dışarıya (LoadBalancer/NodePort) açık değildir — bu, saldırı yüzeyini küçültür.

9.2.4 Backend Yapılandırması (BackendConfig)

Backend ve Keycloak için, GKE'ye özgü `BackendConfig` nesneleri tanımlıdır; bu, sağlık kontrolü (health check) yolları ve yük dengeleyici davranışı gibi GKE-özel ayarları yapılandırır.



Şekil 22: Kubernetes Dağıtım Topolojisi

9.3 Ağ, Ingress ve TLS

9.3.1 Ingress Yönlendirme (Same-Origin)

Sisteme dış erişimin tek kapısı, bir **GKE Ingress**'tir. Tüm trafik tek bir ana bilgisayar adı ('8-233-66-172.nip.io') üzerinden gelir ve yol (path) bazında ilgili servise yönlendirilir:

Yol (Path)	Hedef Servis	Amaç
`/api`	backend:8080	REST API çağrıları
`/realms`, `/resources`	keycloak:8080	Kimlik doğrulama (OIDC token ve kaynakları)
`/` (diğer tümü)	frontend:80	Web arayüzü (SPA)

Bu **aynı köken (same-origin)** yaklaşımı; ön yüz, backend ve Keycloak'ın tarayıcı açısından tek bir kökenden geliyormuş gibi görünmesini sağlar. Bu, CORS ve çerez karmaşıklığını azaltır.

9.3.2 TLS Sonlandırma

Ingress, **HTTPS/TLS sonlandırması** yapar. Sertifika, GKE **ManagedCertificate** ('portiva-cert') ile otomatik sağlanır ve yenilenir (Let's Encrypt). Böylece dışarıdan gelen tüm trafik şifrelidir; iç küme trafiği güvenli ağ içinde kalır.

9.4 Ölçekleme, Yüksek Erişilebilirlik ve Kaynaklar

9.4.1 Yatay Otomatik Ölçekleme (HPA)

Backend, **HorizontalPodAutoscaler** ile otomatik ölçeklenir:

- **Kopya aralığı:** En az 2, en fazla 6 kopya.
- **Metrik:** CPU kullanımı; ortalama %50 hedefi aşıldığında ölçek artırılır.
- **Tepki davranışı:** Ölçek artırma hızlıdır (15 sn stabilizasyon — ani yüke çabuk yanıt). Ölçek azaltma temkinlidir (120 sn stabilizasyon penceresi, her 60 sn'de en fazla 1 pod azaltma); bu, geçici yük düşüşlerinde gereksiz dalgalanmayı (flapping) önler.

9.4.2 Kesinti Bütçesi ve Yedeklilik

- **Backend:** Bir **PodDisruptionBudget** tanımlıdır (`minAvailable: 1``); bu, küme bakımı/güncellemesi sırasında bile en az bir backend kopyasının her zaman çalışır kalmasını garanti eder.
- **Frontend:** Yedeklilik için 2 kopya çalıştırır.
- **Keycloak:** Realm içe aktarımının tutarlılığı için tek kopya (`replicas: 1``) olarak yapılandırılmıştır.

9.4.3 Kaynak Talepleri ve Sağlık Kontrolleri

- **Kaynak yönetimi:** Backend için kaynak talebi (request: 768Mi bellek, 500m CPU) ve üst sınırı (limit: 1.5Gi bellek, 2 CPU) tanımlıdır; bu, adil kaynak paylaşımını ve istikrarı sağlar.
- **Sağlık kontrolleri:** Üç probe kullanılır — `startupProbe`` (Spring Boot başlangıç toleransı), `readinessProbe`` (trafiğe hazır mı — `/actuator/health/readiness``) ve `livenessProbe`` (canlı mı — `/actuator/health/liveness``). Probe'lar, toplu (aggregate) sağlık yerine kritik bileşen gruplarını kullanır; bu, ikincil bir bağımlılığın geçici sorununun pod'u gereksiz yere yeniden başlatmasını önler.

9.4.4 Kalıcı Depolama (PVC)

Durumlu bileşenler (PostgreSQL, Kafka, OpenSearch vb.), `volumeClaimTemplates`` ile kalıcı disk (PVC, `standard-rwo`` depolama sınıfı) talep eder; bu, pod yeniden başlasa bile verinin korunmasını sağlar.

9.5 Sürekli Entegrasyon ve Dağıtım (CI/CD)

Sistem, GitHub Actions tabanlı iki ayrı iş akışıyla (workflow) otomatikleştirilmiş bir CI/CD hattına sahiptir: **CI** (kod kalitesi) ve **CD** (imaj/dağıtım).

9.5.1 Sürekli Entegrasyon (CI)

CI iş akışı, `main`` dalına yapılan her **push** ve her **pull request** üzerinde otomatik çalışır:

- **Backend işi:** JDK 21 ile `mvn verify`` çalıştırır (derleme + tüm testler) ve JaCoCo kod kapsama raporu üretir.
- **Frontend işi:** Node 20 ile Vitest testlerini çalıştırır.

CI başarısız olursa değişiklik birleştirilmeden engellenir; bu, ana dalın her zaman çalışır durumda kalmasını sağlar. (Statik kod analizi için SonarQube ayrıca kullanılır; ayrıntılar Bölüm 14'te.)

9.5.2 Sürekli Dağıtım (CD)

CD iş akışı iki aşamalı çalışır:

1. İmaj Üretimi (otomatik — her push'ta):

- Backend ve frontend imajları derlenir.
- Her imaj iki etiketle (tag) işaretlenir: kısa **git-SHA** (izlenebilirlik için, örn. `a1b2c3d`) ve `latest`.
- İmajlar Google **Artifact Registry**'ye gönderilir (push).

2. Dağıtım (elle tetikli — `workflow\_dispatch`):

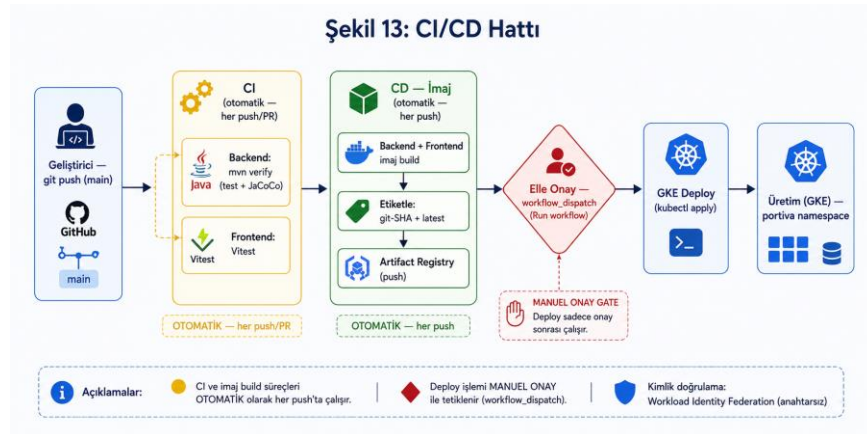
- Dağıtım adımı yalnızca **"Run workflow"** ile elle başlatılır (otomatik push'ta dağıtım yapılmaz). Bu, üretime çıkışın bilinçli bir onayla yapılmasını sağlar.
- Dağıtım, güncel imajları GKE kümesine uygular (kubectl).

9.5.3 Anahtarsız Kimlik Doğrulama (Workload Identity Federation)

CI/CD hattı, Google Cloud'a erişmek için **Workload Identity Federation (WIF)** kullanır — uzun ömürlü servis hesabı anahtarı (JSON key) saklanmaz. Bunun yerine GitHub Actions, kısa ömürlü bir OIDC token (`id-token: write`) ile kimliğini Google'a kanıtlar. Bu, sır sızıntısı riskini ortadan kaldırır, güvenli ve modern bir yaklaşımdır.

9.5.4 İzlenebilir Dağıtım

git-SHA etiketleme sayesinde, üretimde çalışan her imajın **hangi kod sürümünden** üretildiği kesin olarak bilinir. Bu, bir sorun durumunda hızlı geri alma (rollback) ve sorunun hangi değişiklikten kaynaklandığını tespit etme imkânı sağlar.



Şekil 23: CI/CD Hattı

10. GÖZLEMLENEBİLİRLİK TASARIMI

Bu bölüm, sistemin üretim ortamında nasıl izlendiğini; metrik, iz (trace) ve log toplama mimarisini, izleme panolarını ve bunların amaçlarını açıklar. Gözlemlenebilirlik, çok sayıda dış kaynağa bağımlı bu sistemde, sorunların hızlı teşhisi için kritiktir.

10.1 Üç Sütun Mimarisi (Metrik, İz, Log)

Sistem, modern gözlemlenebilirliğin üç temel sütununu (three pillars) toplar:

Sütun	Ne Cevaplar	Toplama Yolu	Depo
Metrikler (Metrics)	"Sistem ne durumda?" (oranlar, sayaçlar)	OTel Collector + doğrudan Actuator	Prometheus
İzler (Traces)	"Bir istek hangi yollardan geçti, nerede yavaşladı?"	OTel agent → OTel Collector → Tempo	Tempo
Loglar (Logs)	"Tam olarak ne oldu?" (olay detayı)	log4j2 (JSON) → Kafka → Log Consumer	OpenSearch

Bu üç sinyal birlikte, bir sorunun **ne zaman** (metrik), **nerede** (iz) ve **neden** (log) oluştuğunu yanıtlar. Grafana, üç deponun (Prometheus, Tempo, OpenSearch) tamamını veri kaynağı olarak birleştirir.

10.2 Veri Akışı ve Bileşenler

10.2.1 İz ve Metrik Akışı

Backend, OpenTelemetry Java agent ile otomatik enstrümente edilir. Agent, iz ve metrikleri **OTel Collector**'a (OTLP protokolü) gönderir. Collector, bir yönlendirme merkezi gibi çalışır:

- **İzleri** → Tempo'ya iletir (otlp/tempo).
- **Metrikleri** → Prometheus formatında dışa verir.

Metrikler iki yoldan Prometheus'a ulaşır: (1) OTel Collector üzerinden (8889 portu) ve (2) Spring Actuator'ın `/actuator/prometheus` uç noktasından doğrudan scrape ile (backend:8080). Bu, hem OTel metrikleri hem de Spring/JVM metriklerinin toplanmasını sağlar (15 sn aralıkla).

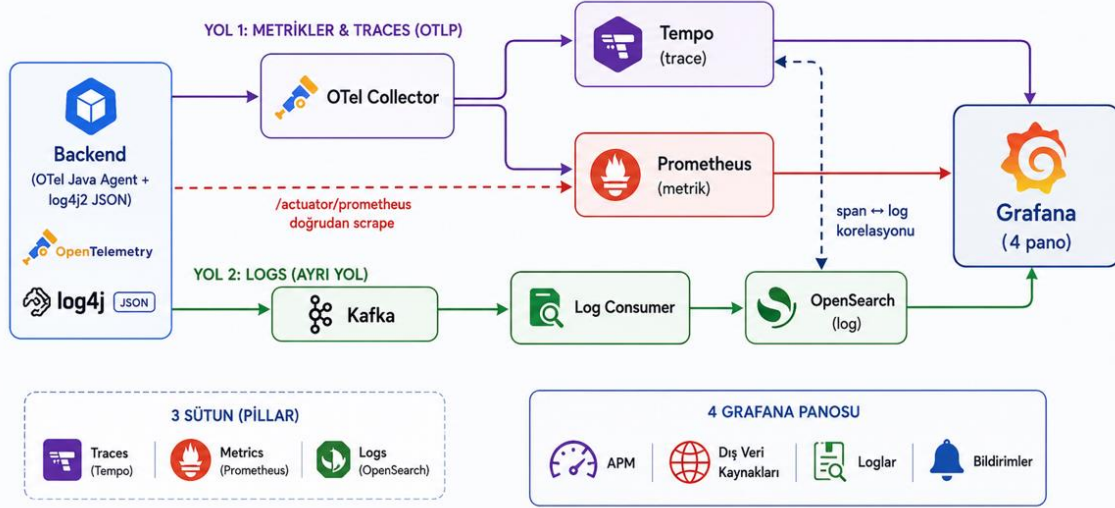
10.2.2 Log Akışı (Ayrı Hat)

Loglar, iz/metrik hattından **bağımsız** bir yolla taşınır. Uygulama logları log4j2 ile **JSON formatında** üretilir, Kafka üzerinden asenkron olarak taşınır, **Log Consumer** servisi bunları

tüketip **OpenSearch**'e yazar. Bu asenkron hat, loglamanın uygulama performansını etkilememesini sağlar. (OTel Collector log işlemez; loglama tamamen bu ayrı hatla yürür.)

10.2.3 Sinyal Korelasyonu (Trace ↔ Log)

Üç sütun birbirine bağlıdır: Grafana'da bir iz (trace) span'i incelenirken, **"bu span'e ait loglar"** bağlantısıyla doğrudan o isteğin OpenSearch'teki loglarına geçilebilir. Bu korelasyon (Tempo ↔ OpenSearch arası bağ), bir sorunu izden loga, loglardan ize takip etmeyi sağlar — teşhis süresini büyük ölçüde kısaltır.



Şekil 24: Gözlemlenebilirlik Veri Akışı

10.3 İzleme Panoları (Grafana Dashboards)

Sistem, farklı izleme ihtiyaçları için **4 özel Grafana panosu** içerir. Her pano belirli bir amaca hizmet eder.

10.3.1 APM Panosu (Uygulama Performansı)

Uygulamanın genel sağlık ve performansını izler (14 panel): istek hacmi, yanıt süresi, hata oranı, p95 gecikme, son backend izleri (traces), JVM bellek (heap) ve havuz bazlı bellek, çöp toplama (GC) süresi, CPU kullanımı, JVM iş parçacıkları, veritabanı bağlantı havuzu (HikariCP), **devre kesici durumu (Circuit Breaker State)** ve **önbellek isabet oranı (Cache Hit Rate)**.

Amaç: Uygulamanın kendi içsel sağlığını ve darboğazlarını tespit etmek. Devre kesici ve önbellek panelleri, Bölüm 4.5'teki dayanıklılık mekanizmalarının (circuit breaker, cache) canlı durumunu da gösterir.

10.3.2 Dış Veri Kaynakları Panosu (En Kritik)

Bu pano, sistemin **20'den fazla dış kaynağa bağımlılığının** doğurduğu riski yönetmek için tasarlanmıştır (12 panel). İçerdiği paneller:

- Her dış kaynak (host) için istek oranı, hata oranı ve p95 gecikme.
- **Sessiz Hatalar (Silent Failures):** Bir kaynağın hata vermeden bozuk/boş veri dönmesi gibi gizli sorunlar — türlerine göre (PARSE_FAILED / EMPTY / RATE_LIMITED) ve sağlayıcıya göre ayrıştırılır; son sessiz-hata olayları (sağlayıcı + neden) listelenir.
- **Ban Riski:** Scrape edilen kaynaklarda (örn. Business Insider, HMB ISIN) istek oranı (anti-ban throttle) ve 403/429 yanıt oranları (ban riski).

Amaç: Çok sayıda dış kaynağın metrik ve loglarını izleyerek bir kaynağın **neden çökebileceğini, çökme ihtimalinin olup olmadığını ve sessiz başarısızlıkları** önceden tespit etmek. Bu pano, dış kaynak kırılmalığının kullanıcıya yansımadan fark edilmesini sağlar — sistemin dayanıklılık tasarımının (LKG, anti-ban) izleme tarafındaki tamamlayıcısıdır.

10.3.3 Loglar Panosu

Yapılandırılmış logları görselleştirir ve filtrelenebilir kılar (19 panel): en son loglar (filtre ile / karta tıklayarak süzülür), log seviyesi dağılımı, durum kodu dağılımı, kategori dağılımı, API istek hacmi/yanıt süresi, hata sayısı/oranı ve en çok istek alan uç noktalar. **Amaç:** Bir sorun anında ilgili logları hızlıca bulmak ve örüntüleri (pattern) görmek.

10.3.4 Bildirimler ve Alarmlar Panosu

İş olaylarını (business events) izler (12 panel): tetiklenen alarmlar, gönderilen/başarısız e-postalar, başarısız e-posta oranı, yeni/iptal bülten abonelikleri, oluşturulan bildirimler, destek talepleri ve olay türü dağılımı. **Amaç:** Kullanıcıya dokunan bildirim/alarm/e-posta akışlarının sağlıklı çalıştığını doğrulamak (örn. e-posta gönderimi aksıyor mu?).

10.4 İş ve Entegrasyon Loglaması (AUDIT/INTEGRATION)

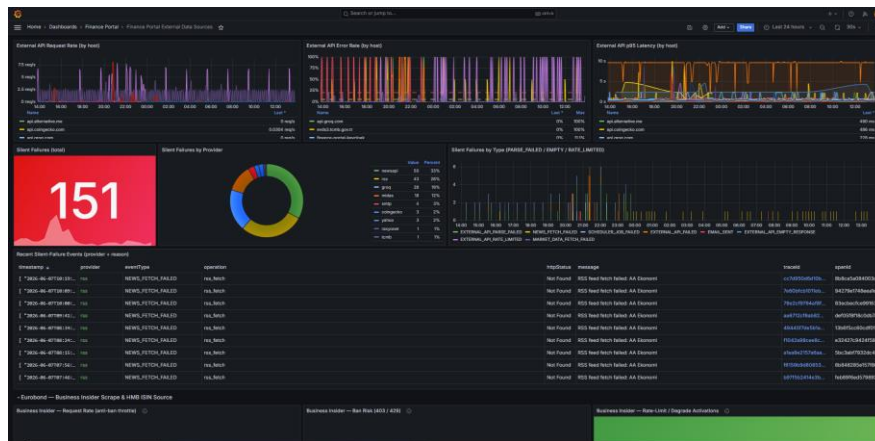
Standart teknik logların yanı sıra, sistem iki özel log türü üretir (Bölüm 4 ve 7 ile ilişkili):

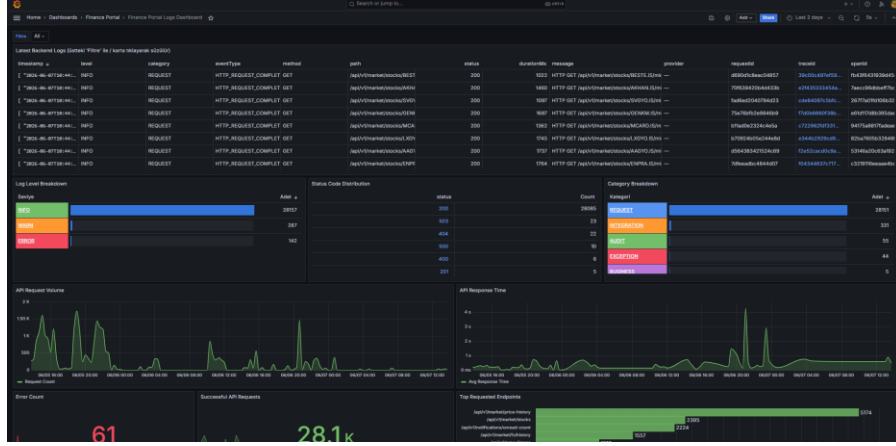
- **İş (AUDIT) logları:** Önemli kullanıcı/yönetici eylemleri (`CentralBusinessLogService`) — kim, ne, ne zaman yaptı. Bu, izlenebilirlik (repudiation önleme) için kritiktir ve Bildirimler panosunu besler.
- **Entegrasyon logları:** Dış servis çağrıları (`CentralIntegrationLogService`) — yukarıdaki Dış Veri Kaynakları panosunu besler.

Her iki log türü de yapılandırılmış (JSON) olarak OpenSearch'e akar ve ilgili panolarda görselleştirilir.

10.5 Uçtan Uca İzleme (Distributed Tracing)

OpenTelemetry agent sayesinde, her HTTP isteği bir **iz (trace)** olarak baştan sona izlenebilir. Bir istek; controller → service → port → dış servis/veritabanı/önbellek katmanları boyunca





Şekil 25: Grafana Dashboard Örnekleri

11. ÇAPRAZ KESEN KONULAR (Cross-Cutting Concerns)

Bu bölüm, belirli bir modüle ait olmayan, tüm sistemi yatay olarak kesen ortak tasarım konularını tek yerde toplar. Bu konuların ayrıntıları ilgili bölümlerde verilmiştir; burada bütünsel bir özet sunulur.

11.1 Standart Yanıt Sarmalayıcı (Response Wrapper)

Tüm API yanıtları, tek tip `ApiResponse<T>` zarfında döner (başarı bayrağı, mesaj, veri, zaman damgası). Bu, ön yüzün tüm uç noktalar için aynı işleme mantığını kullanmasını sağlar. Ayrıntı: Bölüm 7.0.

11.2 Merkezi Hata Yönetimi (Global Exception Handling)

`GlobalExceptionHandler`, tüm istisnaları yakalayıp standart hata yanıtına dönüştürür (10 istisna işleyici, anlamlı HTTP durum kodlarıyla). Hassas bilgiler istemciye sızdırılmaz. Ayrıntı: Bölüm 4.6.

11.3 Girdi Doğrulama (Validation)

Gelen istekler, Bean Validation (`@Valid`, `@NotNull`, `@Email` vb. — 80+ kullanım) ile DTO seviyesinde, ayrıca servis katmanında iş kuralı seviyesinde doğrulanır. Geçersiz girdi iş mantığına ulaşmadan reddedilir. Ayrıntı: Bölüm 8.3.

11.4 İstek Filtreleri (Filter Zinciri)

Her HTTP isteği, iş mantığına ulaşmadan önce sıralı bir filtre zincirinden geçer:

Filter (sıra)	Sorumluluk
CORS Filter (en önce)	İzinli köken (origin) kontrolü; en yüksek öncelikte çalışır

Filter (sıra)	Sorumluluk
`RequestLoggingFilter` (Order 1)	Her isteğe bir `requestId` (UUID) atar; MDC'ye requestId, metot, yol, kullanıcı, istemci IP, durum kodu ve süre (ms) koyar — istek sonunda temizler
`AdminBanStatusFilter`	Banlı bir kullanıcının korumalı işlem yapmasını engeller
`DisabledAccountFilter`	Devre dışı bırakılmış hesabın erişimini durdurur
`EmailVerifiedFilter`	E-posta doğrulaması gereken işlemleri kontrol eder

Bu filtreler, güvenlik ve loglama gibi çapraz konuları merkezi ve tutarlı biçimde uygular. (Sistemde ayrı bir AOP/Aspect katmanı yoktur; çapraz konular filtre ve interceptor desenleriyle çözülür.)

11.5 Loglama ve İz Kancaları (Logging & Tracing Hooks)

Sistem genelinde, gözlemlenebilirlik için yoğun loglama/izleme kancaları bulunur (kod tabanında 129+ `@WithSpan`/`@SpanAttribute`/MDC kullanımı):

- **İz kancaları:** Kritik metotlar `@WithSpan` ile izlenir; önemli değerler `@SpanAttribute` ile span'lere eklenir.
- **Korelasyon:** `RequestLoggingFilter`'in her isteğe atadığı `requestId` ve MDC (Mapped Diagnostic Context) alanları (kullanıcı, IP, yol, durum, süre) ile sağlanır; böylece bir isteğin tüm log satırları aynı kimlikle ilişkilendirilir.
- **Yapılandırılmış log:** Tüm loglar JSON formatında (log4j2 + JsonTemplateLayout), alan-standartlı üretilir.

Ayrıntı: Bölüm 10.

11.6 Görsel Proxy (Image Proxy)

Haber ve varlık logosu gibi dış görseller, doğrudan tarayıcıya yüklenmek yerine bir **görsel proxy** servisinden (`ImageProxyService`, `/api/v1/proxy`) geçirilir. Bunun iki nedeni vardır:

- **Karışık içerik (mixed content) ve CORS:** HTTPS sayfada HTTP görsel veya CORS kısıtlı kaynak sorunlarını önler.
- **Güvenlik (SSRF koruması):** Proxy yalnızca bir izin listesindeki (**allow-list**) ana bilgisayarlardan görsel çeker (`ALLOWED\_HOSTS`); böylece sunucunun keyfi/iç adreslere yönlendirilmesi (Server-Side Request Forgery) engellenir.

Bu, harici görsellerin güvenli ve tutarlı biçimde sunulmasını sağlayan çapraz bir bileşendir.

11.7 Performans Çapraz Konuları

Performans, tek bir yerde değil, sistem genelinde ele alınmıştır:

- **Önbellekleme:** Cache-first okuma ve LKG (Bölüm 4.5, 6.5).
- **Önbellek ısıtma:** Soğuk yük gecikmesini önleme (Bölüm 4.5.3).
- **Paralel çekim:** Portföy değerlemede çok varlığın fiyatı paralel alınır (Bölüm 7.3.2).
- **Yatay ölçekleme:** HPA ile yük altında otomatik ölçekleme (Bölüm 9.4).
- **Asenkron loglama:** Loglar Kafka ile asenkron taşınır, ana akışı bloke etmez (Bölüm 10.2.2).

12. MİMARİ KARAR KAYITLARI (ADR)

Bu bölüm, sistemin tasarımı sırasında alınan önemli mimari kararları; bağlamı, seçilen yaklaşımı, gerekçesini ve sonuçlarını kayıt altına alır (Architecture Decision Records). Bu kayıtlar, "neden böyle yapıldı?" sorusunu yanıtlar ve gelecekteki değişikliklere referans oluşturur.

ADR-01: Modüler Monolit Mimari

- **Bağlam:** Sistem çok sayıda işlevsel alan (portföy, market, alarm, haber vb.) içeriyor.
- **Karar:** Mikroservis yerine, işlevsel alanlara net biçimde bölünmüş bir **modüler monolit** seçildi.
- **Gerekçe:** Küçük ekip için mikroservislerin operasyonel karmaşıklığı (dağıtık işlem, ağ gecikmesi, servis keşfi) maliyetlidir. Modüler monolit, modülerlik avantajını dağıtım basitliğiyle birleştirir.
- **Sonuç:** Tek dağıtılabılır birim; modüller arası port-tabanlı gevşek bağıllık; gerektiğinde ileride bir modül ayrıştırılabilir.

ADR-02: Clean Architecture ve Port/Adapter

- **Bağlam:** 20'den fazla dış kaynak entegrasyonu ve değişen sağlayıcılar.
- **Karar:** Her domain Clean Architecture katmanlarına bölündü; dış bağımlılıklar **port/adapter** ile soyutlandı.
- **Gerekçe:** Dış servisler sık değişir/kırılır; iş mantığını bunlardan yalıtımak sürdürülebilirlik için kritiktir.
- **Sonuç:** Bir kaynağın değişmesi yalnızca adapter'ı etkiler; domain çerçeveden bağımsız ve test edilebilir kalır.

ADR-03: Dayanıklılık için LKG (Stale-if-Error)

- **Bağlam:** Dış kaynaklar kesilebilir, kota dolabilir, yavaşlayabilir.
- **Karar:** Birincil dayanıklılık deseni olarak **Last Known Good (LKG)** seçildi; Resilience4j (retry/circuit breaker) ile tamamlandı.
- **Gerekçe:** Bir kaynak çökse bile kullanıcıya "son geçerli veri" sunmak, hata göstermekten iyidir (graceful degradation).
- **Sonuç:** Sistem, dış kaynak kesintilerinde çalışmaya devam eder; veri tazeliği geçici olarak düşebilir.

ADR-04: Kimlik Yönetiminin Dışsallaştırılması (Keycloak)

- **Bağlam:** Güvenli kimlik doğrulama, 2FA, şifre politikası ihtiyacı.

- **Karar:** Kimlik yönetimi kendi kodumuzda değil, **Keycloak + LDAP**'a devredildi.
- **Gerekçe:** Kimlik güvenliği uzmanlık gerektirir; uygun bir çözümü yeniden yazmak hem riskli hem gereksizdir.
- **Sonuç:** Veritabanında kullanıcı tablosu yok; tablolar `user\_id` ile mantıksal bağlanır. JWT yerel doğrulanır.

ADR-05: Kalıcı Olmayan Piyasa/Haber Verisi (Cache-Only)

- **Bağlam:** Piyasa ve haber verisi sürekli değişen, "akış" niteliğinde veridir.
- **Karar:** Bu veriler PostgreSQL'de **saklanmaz**; yalnızca Redis önbelleğinde geçici tutulur.
- **Gerekçe:** Sürekli değişen veriyi kalıcı saklamak gereksiz depolama/indeks yükü ve veri-bayatlama sorunu doğurur.
- **Sonuç:** Veritabanı yalnızca kullanıcı verisini tutar (10 tablo); piyasa/haber cache-first sunulur.

ADR-06: Standart Yanıt Sarmalayıcı (ApiResponse)

- **Bağlam:** 124 uç nokta, tutarlı bir istemci-sunucu sözleşmesi gerektirir.
- **Karar:** Tüm yanıtlar tek tip `ApiResponse<T>` zarfında döner.
- **Gerekçe:** Ön yüzün her uç nokta için aynı işleme/hata mantığını kullanması, tutarlılık ve bakım kolaylığı sağlar.
- **Sonuç:** Öngörülebilir API; merkezi hata yönetimiyle uyumlu.

ADR-07: Şema Sürümleme (Flyway)

- **Bağlam:** Şema zamanla evrilir; farklı ortamlarda tutarlılık gerekir.
- **Karar:** Tüm şema değişiklikleri **Flyway** migration'larıyla yönetilir.
- **Gerekçe:** Sürüm kontrollü, deterministik, izlenebilir şema evrimi sağlar.
- **Sonuç:** 17 migration; tüm ortamlarda aynı şema; geriye dönük takip.

ADR-08: Yapay Zekâ Hesaplama Ayrımı (AI Tasarlar, Kod Hesaplar)

- **Bağlam:** Yapay zekâ finansal sayı üretirken hata yapabilir (halüsinasyon).
- **Karar:** Senaryo simülasyonu ve analizlerde, ilgili araç katmanında **hesaplamayı kod yapar, yapay zekâ yalnızca senaryoyu tasarlar/yorumlar**.
- **Gerekçe:** Finansal doğruluk kritiktir; yapay zekânın sayısal güvenilirliği yeterli değildir.
- **Sonuç:** Doğru hesaplama + doğal dil açıklaması birleşir; al/sat işlem aracı yoktur (yalnızca takip).

ADR-09: Elle Onaylı Dağıtım (Manuel CD Trigger)

- **Bağlam:** Üretime otomatik (her push'ta) çıkış riski.
- **Karar:** İmaj üretimi otomatik, ancak **dağıtım elle tetiklenir** (if: github.event_name == 'workflow_dispatch').
- **Gerekçe:** Üretime çıkışın bilinçli bir kontrol noktasından geçmesi (özellikle küçük ekipli projede) güvenlidir.
- **Sonuç:** Her push imaj üretir (izlenebilir), dağıtım onayla yapılır.

ADR-10: Anahtarsız CI/CD Kimlik Doğrulama (WIF)

- **Bağlam:** CI/CD'nin buluta erişimi için kimlik bilgisi gerekir.
- **Karar:** Uzun ömürlü servis anahtarı yerine **Workload Identity Federation** kullanıldı.
- **Gerekçe:** Saklanan anahtarlar sızabilir; kısa ömürlü OIDC token'lar daha güvenlidir.
- **Sonuç:** CI/CD'de saklanan sır yok; sızıntı riski minimum.

ADR-11: Çoklu Sağlayıcı Yapay Zekâ Zinciri

- **Bağlam:** Ücretsiz yapay zekâ API'leri kota sınırlıdır ve tek sağlayıcıya bağımlılık risklidir.
- **Karar:** Sohbet asistanı, **sıralı bir sağlayıcı zinciri** kullanır (`chain`: Gemini → Gemini-lite → Groq); bir sağlayıcı kotası dolduğunda veya başarısız olduğunda otomatik olarak bir sonrakine geçilir.
- **Gerekçe:** Tek bir ücretsiz sağlayıcının kota/erişim sınırı, asistanın tamamen durmasına yol açabilir; zincir yaklaşımı süreklilik sağlar.
- **Sonuç:** Anahtarı olmayan sağlayıcı adımı atlanır; asistan, bir sağlayıcı çökse bile çalışmaya devam eder (dayanıklılık deseninin yapay zekâ tarafındaki uygulaması).

13. RİSKLER VE ÖNLEMLER

Bu bölüm, sistemin karşı karşıya olduğu teknik riskleri kategorize eder, her risk için alınmış önlemleri belirtir ve bunları bir öncelik matrisinde (olasılık × etki) konumlandırır.

13.1 Risk Kategorileri ve Önlemler

13.1.1 Entegrasyon Riskleri (Dış Kaynak)

Risk	Etki	Önlem
Dış kaynak kesintisi / kota dolması	Veri sunulamaz	LKG (son geçerli veri), çoklu kaynak zinciri, degrade davranışı
Scrape kaynağı ban'ı (403/429)	Eurobond/VİOP verisi kesilir	Anti-ban throttle (istekler arası bekleme), ban-risk izleme panosu
Sessiz hata (bozuk/boş veri)	Yanlış veri gösterimi	Sessiz-hata tespiti ve izleme (Dış Kaynaklar panosu), içerik doğrulama
Sağlayıcı API formatı değişimi	Ayrıştırma (parse) hatası	Port/adapter izolasyonu (yalnızca adapter etkilenir), parse-hata izleme

13.1.2 Performans ve Yük Riskleri

Risk	Etki	Önlem
Soğuk önbellek gecikmesi	Büyük listelerde ilk erişim yavaş	Önbellek ısıtma (warm-up), TTL > ısıtma periyodu invaryantı

Risk	Etki	Önlem
Ani yük artışı	Yanıt süresi artışı	HPA ile otomatik ölçekleme (2–6 kopya)
Zamanlanmış görev yoğunluğu	Görevlerin birbirini bloke etmesi	Çok iş parçacıklı zamanlayıcı havuzu, ShedLock

13.1.3 Güvenilirlik Riskleri

Risk	Etki	Önlem
Önbellek bayatlaması (stale data)	Eski veri gösterimi	TTL stratejisi, aktif geçersiz kılma (@CacheEvict), tazelik göstergesi (asOf)
Tek kopya hatası	Hizmet kesintisi	Çoklu kopya (backend ≥ 2), PodDisruptionBudget, sağlık probe'ları
Veritabanı tutarsızlığı	Bozuk veri	DB kısıtları (CHECK/FK/UNIQUE), iki katmanlı doğrulama

13.1.4 Güvenlik Riskleri

Risk	Etki	Önlem
Yetkisiz erişim	Veri ihlali	RBAC, kaynak sahipliği, JWT doğrulama (bkz. Bölüm 8)
Sır sızıntısı	Kimlik bilgisi ifşası	Kod-dışı secret yönetimi, log sanitizasyonu, WIF
Kötüye kullanım (anonim)	Kaynak tükenmesi	Rate limiting (IP bazlı), anti-ban

13.1.5 Operasyonel ve Veri-Kalitesi Riskleri

Risk	Etki	Önlem
Hatalı yapılandırma (üretim)	Sistem yanlış çalışır	Fail-loud (eksik sırda başlamama), profil ayrımı
Eski imaj dağıtımı	Yeni özellik canlıya çıkmaz	git-SHA etiketleme, imaj doğrulama
Tutarsız işlem girişi (aşırı-satış)	Negatif/bozuk pozisyon	Aşırı-satış tespiti: uyarı verip işlemi atlama (warn+skip), hesaplamayı bozmama
İzleme veri gürültüsü	Maliyet/karmaşa	Yapılandırılmış log, kategori filtreleme, kritik metrik odağı

13.2 Risk Öncelik Matrisi

Riskler, gerçekleşme olasılığı ve etkisine göre bir matriste konumlandırılarak önceliklendirilir. Bu, hangi risklere öncelikli odaklanılması gerektiğini gösterir.

Matris özeti :

	Etki: Düşük	Etki: Orta	Etki: Yüksek
Olasılık: Yüksek	İzleme gürültüsü	Önbellek bayatlama	Dış kaynak kesintisi / scrape ban
Olasılık: Orta	Yapılandırma hatası	Sessiz hata, yük artışı	Sağlayıcı format değişimi
Olasılık: Düşük	—	Tek kopya hatası	Güvenlik ihlali

Yüksek olasılık/yüksek etki bölgesindeki riskler (dış kaynak kesintisi, scrape ban), sistemin en güçlü önlemlerinin (LKG, anti-ban, izleme) yöneldiği önceliklerdir.

14. TASARIM DOĞRULAMA (V&V) VE NFR ÖZETİ

Bu bölüm, tasarımın nasıl doğrulandığını; test stratejisini, kullanılan araçları, kalite kapısını ve fonksiyonel olmayan gereksinimlerin (NFR) tasarımdaki karşılıklarını açıklar.

14.1 Test Stratejisi

Sistem, test piramidine uygun olarak çok seviyeli bir test stratejisiyle doğrulanır. Kod tabanında toplam **328 test dosyası** (216 backend birim + 12 entegrasyon + 100 frontend) bulunur.

Test Türü	Araç	Kapsam
Birim (Unit)	JUnit Jupiter (backend), Vitest (frontend)	Tekil sınıf/fonksiyon mantığı, izole
Entegrasyon (Integration)	Spring Boot Test + Testcontainers	Gerçek PostgreSQL/Redis ile çalışan akışlar
Arayüz/API (Controller)	Spring MockMvc	REST uç noktası istek/yanıt davranışı
Güvenlik (Security)	spring-security-test	Kimlik doğrulama, yetkilendirme, erişim kontrolü
Bileşen (Component)	React Testing Library + jsdom	Ön yüz bileşen render/etkileşim
Dış Servis Mock	WireMock	Dış API'lerin taklit edilerek izole testi
Performans / Yük	k6	Yük altında yanıt süresi ve davranış (load-test, cloud job)

14.2 Kod Kalitesi Araçları

- **JaCoCo:** Test çalıştırmasında satır/dal (branch) kapsama ölçümü yapılır; rapor CI'da üretilir (`mvn verify`). Kapsama dışı tutulan alanlar (DTO, domain, config) bilinçli olarak hariç tutulur.
- **SonarQube:** Statik kod analizi ve kalite kapısı (quality gate) için kullanılır. Sistem **self-hosted** bir SonarQube örneğiyle, geliştirme sırasında elle çalıştırılır (`mvn sonar:sonar`); JaCoCo kapsama raporunu girdi olarak okur.
- **Vitest Coverage (v8):** Frontend test kapsama ölçümü.

14.3 Sürekli Doğrulama (CI)

Tüm testler, CI hattında otomatik çalışır (Bölüm 9.5): her push/PR'de backend `mvn verify` (derleme + testler + JaCoCo) ve frontend Vitest. CI başarısız olursa değişiklik engellenir; bu, ana dalın her zaman doğrulanmış kalmasını sağlar.

14.4 NFR Özet Tablosu (Gereksinim → Tasarım Karşılığı)

Aşağıdaki tablo, Analiz Dökümanında tanımlanan fonksiyonel olmayan gereksinimlerin (NFR) bu teknik tasarımda nasıl karşılandığını özetler:

NFR (Kalite Niteliği)	Tasarımdaki Karşılığı
Performans	Redis cache-first + 50+ ad alanı + warm-up; paralel veri çekimi (Bölüm 4.5, 4.7)
Ölçeklenebilirlik	Kubernetes HPA (2–6 kopya, %50 CPU); durumsuz backend (Bölüm 9.4)
Kullanılabilirlik / Erişilebilirlik	Çoklu kopya, PDB, sağlık probe'ları; degrade davranışı (Bölüm 9.4, 7.4)
Dayanıklılık	LKG (100+ nokta), Resilience4j (retry/CB), çoklu kaynak zinciri (Bölüm 4.5)
Güvenlik	Keycloak/OIDC, RBAC, 2FA, HTTPS/TLS, secret yönetimi (Bölüm 8)
Sürdürülebilirlik	Clean Architecture, modülerlik, port/adapter, Flyway (Bölüm 3, 4, 6)
Gözlemlenebilirlik	OpenTelemetry 3-sütun, 4 Grafana panosu, yapılandırılmış log (Bölüm 10)
Taşınabilirlik	Docker konteyner, ortam-bağımsız yapılandırma (Bölüm 4.8, 9.1)
Test Edilebilirlik	Katmanlı yapı, 328 test, Testcontainers/WireMock (Bölüm 14.1)
Uluslararasılaştırma	TR/EN i18n, locale biçimlendirme (Bölüm 5.6)

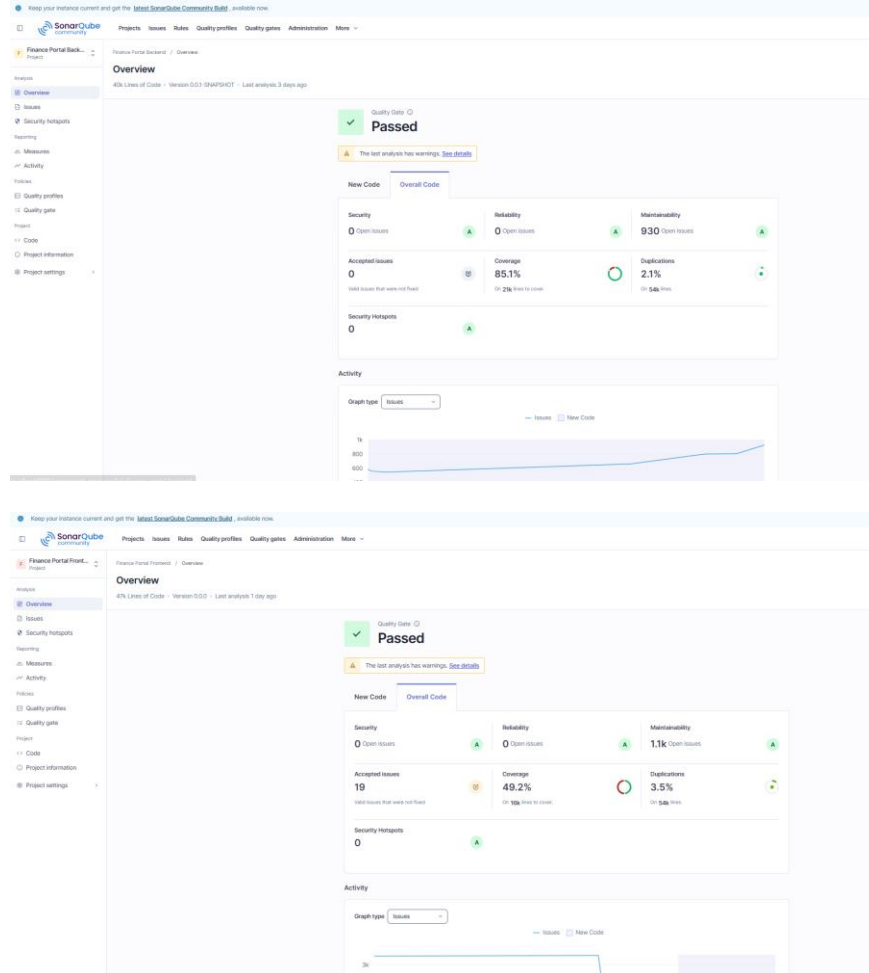
Bu tablo, Analiz Dökümanı (NFR) ile bu Teknik Döküman (tasarım) arasındaki izlenebilirliği sağlar.

14.5 Kalite Ekranları

Aşağıdaki ekran görüntüleri, sistemin çalışan kod kalitesi altyapısından alınmıştır ve doğrulama sürecinin somut çıktılarını gösterir.

14.5.1 SonarQube Kalite Kapısı

Statik kod analizi sonuçları: kalite kapısı (quality gate) durumu, hata (bug), güvenlik açığı (vulnerability), kod kokusu (code smell) sayıları ve kapsama (coverage) oranı.



Şekil 26: Grafana Dashboard Örnekleri

15. EKLER

Ek A — Gereksinim İzlenebilirlik Matrisi (RTM)

Bu matris, Analiz Dökümanındaki fonksiyonel gereksinim gruplarının (FR) bu Teknik Dökümandaki tasarım bölümleriyle ilişkisini gösterir. Böylece her gereksinimin nerede ve nasıl tasarlandığı izlenebilir — Analiz (NE) ile Teknik (NASIL) arasındaki bağ kurulur.

Analiz FR Grubu	Tasarım Bölümü	İlgili Tasarım Bileşenleri
FR-AUTH (Kimlik/Yetki)	7.2, 8.1	Keycloak/OIDC, JWT yerel doğrulama, SecurityConfig, RBAC
FR-DASH (Dashboard)	5.1, 5.3, 5.7	DashboardPage, PreferencesContext, GridBoard
FR-MKT (Piyasa Verileri)	4.3, 4.7, 7.1	Asset-type routing, provider adapter, cache-first, MarketController'lar
FR-DTL (Varlık Detay/Grafik)	4.4, 5.4	Chart strategy, klinecharts, indikatörler, çizim kalıcılığı
FR-PRT (Portföy)	6.1, 7.3.2	Portfolio/PortfolioTransaction, holdings hesaplama, değerlendirme
FR-WL (İzleme Listesi)	5.3, 6.1	WatchlistContext, watchlist_item tablosu
FR-ALR (Alarmlar)	4.7, 7.3.3	AlarmEvaluationService, alarm tablosu, scheduler
FR-AI (AI Analiz/Simülasyon)	7.3.4, 7.3.5	Tool-calling, ScenarioSimulation, AI-hesaplama ayrımı
FR-CHAT (Sohbet Asistanı)	7.3.4, ADR-11	Assistant, 9 araç, çoklu sağlayıcı zinciri
FR-DRW (Kaydedilmiş Grafikler)	5.4.3	useChartDrawings, overlay kalıcılığı
FR-NEWS (Haberler)	3.3.1, 7.1	News modülü, çok kaynaklı toplama, kişiselleştirme
FR-PRF (Profil/Tercih)	5.3, 6.1	user_preference, PreferencesContext, MeController
FR-ADM (Yönetim)	8.2	BanUserService, cascade, AdminUserController
FR-NL (Bülten)	7.3.8	NewsletterDigestService, newsletter_subscription
FR-SUP (Destek)	7.1	SupportTicketController, support_ticket tablosu

Ek B — Görsel ve Diyagram Dizini

Bu dökümanda yer alan başlıca diyagram ve görseller:

No	Görsel	Bölüm
Ş-1	C4 Seviye 1 — Sistem Bağlam Diyagramı	3.1
Ş-2	C4 Seviye 2 — Konteyner Diyagramı	3.2
Ş-3	C4 Seviye 3 — Bileşen Diyagramı	3.3
Ş-4	Clean Architecture Katman Diyagramı	3.4
Ş-5	Modüller Arası Bağımlılık Diyagramı	4.2
Ş-6	Dış Veri Entegrasyonu / Sembol Yönlendirme	4.3
Ş-7	Kripto Grafik Kaynak Zinciri	4.4
Ş-8	Dayanıklılık Katmanları (Retry/CB/LKG)	4.5
Ş-9	Kimlik Doğrulama Sıra Diyagramı (OIDC/PKCE)	7.2
Ş-10	Etkileşim Sıra Diyagramları (cache-first, alarm, AI, ban)	7.3
Ş-11	Ban Cascade Akış Diyagramı	8.2
Ş-12	Kubernetes Dağıtım Topolojisi	9.2
Ş-13	CI/CD Hattı	9.5
Ş-14	Gözlemlenebilirlik Veri Akışı	10.2
Ş-15	Risk Öncelik Matrisi	13.2

Ekran görüntüleri: Uygulama UI (5.7 — dashboard/portföy/detay/AI analiz), Keycloak giriş (7.2.4), Swagger UI (7.0.4), Grafana/OpenSearch/Tempo izleme panoları (10.6), SonarQube/JaCoCo/k6 kalite ekranları (14.5).

Ek C — Teknik Terimler ve Kısaltmalar

Dökümanda kullanılan teknik terim ve kısaltmaların tam listesi, Bölüm 1.4'te (Tanımlar ve Kısaltmalar) verilmiştir. Finansal terimler için Analiz Dökümanının sözlüğüne başvurulabilir.

Ek D — İlgili Dökümanlar

- Toyota 32Bit Finans Portalı (Portiva) Analiz Dökümanı (SRS):** Bu dökümanda tasarımı yapılan iş ve kullanıcı gereksinimlerinin tanımlandığı doküman.
- Kaynak kod deposu:** Tasarımın somut gerçekleştirimi.
- Dayanılan standartlar:** IEEE 1016-2009, ISO/IEC/IEEE 42010:2022, C4 Model, OWASP ASVS (Bölüm 1.5).

Ek E — Kapsam Dışı ve Gelecek Çalışmalar

Aşağıdaki konular mevcut tasarım kapsamının dışındadır ve gerektiğinde ileride ele alınabilir:

- **Mikroservis ayrıştırması:** Mevcut modüller monolit, gerekirse modül bazında ayrıştırılabilir (ADR-01).
- **Gelişmiş güvenlik:** Tam ASVS Seviye 3, donanımsal anahtar yönetimi (HSM), gelişmiş anomali tespiti (Bölüm 8.4.3).
- **Gerçek al/sat işlemleri:** Sistem portföy takibi yapar; gerçek emir iletimi/ödeme altyapısı kapsam dışıdır.
- **İleri analitik:** Monte Carlo simülasyonu, otomatik portföy dengeleme, hedef-tabanlı planlama (mevcut AI analizinin ötesinde).